

beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is another solution to this [latex.org forum query](#).

Contents

1	Implementation	2
1.1	Package declarations	2
1.2	Overview	2
1.3	Facility layer: definitions and naming	3
1.3.1	Debug	4
1.3.2	Facility layer: Functions	5
1.3.3	logging	5
1.3.4	Facility layer: Variables	6
1.3.5	Regex	12
1.3.6	Token lists	12
1.3.7	Strings	16
1.3.8	Sequences	18
1.3.9	Integers	19
1.4	Debug facilities	20
1.5	Local variables	20
1.6	Infinite loop management	22
1.7	Overlay specification	23
1.7.1	Registration	23
1.7.2	Data model	23
1.7.3	Low level IPK model functions	23
1.7.4	Loops	27
1.7.5	Path function	29
1.7.6	Low level IP model functions	30
1.7.7	Low level I model functions	30
1.7.8	Getting	30
1.7.9	Circular dependencies	35
1.7.10	Spring cleaning	35

1.7.11 The provide mode	37
1.7.12 Initialize	39
1.8 Regular expressions	40
1.9 End group setters	41
1.10 beamer.cls interface	42
1.11 Utilities	43
1.11.1 Split utilities	43
1.11.2 Match utilities	44
1.12 Scanner	46
1.12.1 Base grammar	47
1.12.2 Storage	49
1.12.3 AST exportation policy	51
1.12.4 Branch functions	53
1.12.5 Begin and end of scanning processes	58
1.12.6 Main scanners	60
1.12.7 Scan sessions	61
1.12.8 Delimited units	63
1.12.9 Raw units	67
1.12.10 $\langle csl(\langle item \rangle) \rangle$ units	70
1.12.11 VAZL units	72
1.12.12 Lists	79
1.12.13 ISR unit	80
1.12.14 ISR extension units	88
1.12.15 Running an AST	91
1.12.16 Run units	92
1.13 Definition scanner	94
1.13.1 AST	94
1.13.2 Next section	94

1 Implementation

1.1 Package declarations

```

1 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
2 \ProvidesExplPackage
3   {beanoves}
4   {2025/06/03}
5   {1.0}
6   {Named overlay specifications for beamer}

```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string **beanoves** or containing the case insensitive string **bnvs** delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into beamer specifications based on this data model.

The development is made easier due to a facility layer over `expl`.

1.3 Facility layer: definitions and naming

In order to make the code shorter and easier to read during development, we add a layer over $\text{\LaTeX}3$. The `c` and `v` argument specifiers take a slightly different meaning when used in a function which name contains `bnvs` or `BNVS`. Where $\text{\LaTeX}3$ would transform `l__bnvs_ref_tl` into `\l__bnvs_ref_tl`, `bnvs` will directly transform `ref` into `\l__bnvs_ref_tl`. The type of the local variable used depends on the context and may be `seq` or `int` for example. There are however a pair of exceptions mentioned below. For a better reading experience, “`ref`” will generally stand for `\l__bnvs_ref_tl`, whereas “`path sequence`” will generally stand for `\l__bnvs_path_seq`. Other similar shortcuts are used as well.

Functions with `BNVS` in their names are management functions. They belong to a deeper layer and do not really contain any logic specific to the `beanoves` package.

<code>\BNVS:c</code>	<code>\BNVS:c {<cs core name>}</code>
<code>\BNVS_l:cn</code>	<code>\BNVS_l:cn {<local variable core name>} {<type>}</code>
<code>\BNVS_g:cn</code>	<code>\BNVS_g:cn {<global variable core name>} {<type>}</code>

These are naming functions internally used to focus on the discriminating part of variable or function names.

```

7 \cs_new:Npn \BNVS:c #1 { __bnvs_#1 }
8 \cs_new:Npn \BNVS_l:cn #1 #2 { l__bnvs_#1_#2 }
9 \cs_new:Npn \BNVS_g:cn #1 #2 { g__bnvs_#1_#2 }
```

<code>\BNVS_use_Raw:N</code>	<code>\BNVS_use_Raw:N <cs></code>
<code>\BNVS_use_Raw:c</code>	<code>\BNVS_use_Raw:c {<cs name>}</code>
<code>\BNVS_use_Raw:Nc</code>	<code>\BNVS_use_Raw:Nc <function> {<cs name>}</code>
<code>\BNVS_use_Raw:nc</code>	<code>\BNVS_use_Raw:nc {<tokens>} {<cs name>}</code>
<code>\BNVS_use:c</code>	<code>\BNVS_use:c {<cs core>}</code>
<code>\BNVS_use:Nc</code>	<code>\BNVS_use:Nc <function> {<cs core>}</code>
<code>\BNVS_use:nc</code>	<code>\BNVS_use:nc {<tokens>} {<cs core>}</code>

`\BNVS_use_Raw:c` is a convenient wrapper over `\use:c`. possibly prepended with some code, for debugging and testing. It needs 3 expansion steps just like `\BNVS_use:c`. The other are used to expand `\use:c` enough before usage by `<function>` or `<tokens>`. The first argument of `<function>` has type `N`. The next token after `<tokens>` will have type `N` too. `<cs name>` is a full cs name whereas `<cs core>` will be prepended with the appropriate prefix specific to the `beanoves` package.

```

10 \cs_new:Npn \BNVS_use_Raw:N #1 { #1 }
11 \cs_new:Npn \BNVS_use_Raw:c #1 {
12   \exp_last_unbraced:No
13   \BNVS_use_Raw:N { \cs:w #1 \cs_end: }
14 }

15 \cs_new:Npn \BNVS_use:c #1 {
16   \BNVS_use_Raw:c { \BNVS:c { #1 } }
17 }

18 \cs_new:Npn \BNVS_use_Raw:NN #1 #2 {
19   #1 #2
20 }
```

```

21 \cs_new:Npn \BNVS_use_Raw:nN #1 #2 {
22   #1 #2
23 }
24 \cs_new:Npn \BNVS_use_Raw:Nc #1 #2 {
25   \exp_args:Nnc \BNVS_use_Raw:NN #1 { #2 }
26 }
27 \cs_new:Npn \BNVS_use_Raw:nc #1 #2 {
28   \exp_last_unbraced:Nno
29   \BNVS_use_Raw:nN { #1 } { \cs:w #2 \cs_end: }
30 }
31 \cs_new:Npn \BNVS_use:Nc #1 #2 {
32   \BNVS_use_Raw:Nc #1 { \BNVS:c { #2 } }
33 }
34 \cs_new:Npn \BNVS_use:nc #1 #2 {
35   \BNVS_use_Raw:nc { #1 } { \BNVS:c { #2 } }
36 }
37 \cs_new:Npn \BNVS_tl_use_unbraced:nv #1 #2 {
38   \exp_last_unbraced:Nnf \use:n { #1 } { \BNVS_tl_use:nv {\use:n} { #2 } }
39 }
40 \cs_new:Npn \BNVS_tl_use:nvv #1 #2 {
41   \BNVS_tl_use:nv { \BNVS_tl_use:nv { #1 } { #2 } }
42 }
43 \cs_new:Npn \BNVS_tl_use:nvvv #1 #2 {
44   \BNVS_tl_use:nvv { \BNVS_tl_use:nv { #1 } { #2 } }
45 }
46 \cs_new:Npn \BNVS_tl_use:nvvvv #1 #2 {
47   \BNVS_tl_use:nvvv { \BNVS_tl_use:nv { #1 } { #2 } }
48 }
49 \cs_new:Npn \BNVS_log:n #1 { }
50 \cs_generate_variant:Nn \BNVS_log:n { x }

```

1.3.1 Debug

	<code>\BNVS_DEBUG_ON:</code>	<code>\BNVS_DEBUG_ON:</code>
	<code>\BNVS_DEBUG_OFF:</code>	<code>\BNVS_DEBUG_OFF:</code>
	<code>\BNVS_DEBUG_push:nn</code>	<code>\BNVS_DEBUG_push:nn {⟨types on⟩} {⟨types off⟩}</code>
	<code>\BNVS_DEBUG_pop:</code>	<code>\BNVS_DEBUG_pop:</code>

These functions activate or deactivate the debug mode. They are only active in the `beanoves-debug` package. Manage debug messaging for one given `⟨type⟩` or `⟨types⟩`. The implementation is not publicly exposed. The `⟨type⟩` is a single letter or `**`.

1.3.2 Facility layer: Functions

`\BNVS_new:cpn` `\BNVS_new:cpn` is like `\cs_new:cpn` except that the name argument is tagged for beanoves package. Similarly for `\BNVS_set:cpn`.

```

51 \cs_new:Npn \BNVS_new:cpn #1 {
52   \cs_new:cpn { \BNVS:c { #1 } }
53 }

54 \cs_new:Npn \BNVS_set:cpn #1 {
55   \cs_set:cpn { \BNVS:c { #1 } }
56 }

57 \cs_generate_variant:Nn \cs_generate_variant:Nn { c }
58 \cs_new:Npn \BNVS_generate_variant:cn #1 {
59   \cs_generate_variant:cn { \BNVS:c { #1 } }
60 }
```

1.3.3 logging

`\BNVS_warning:n` `\BNVS_warning:n {<message>}`
`\BNVS_warning:x` `\BNVS_error:n {<message>}`
`\BNVS_error:n` `\BNVS_fatal:n {<message>}`
`\BNVS_error:x`
`\BNVS_fatal:n` Very rudimentary. No long message available for error recovery.
`\BNVS_fatal:x`

```

61 \msg_new:nnn { beanoves } { :n } { #1 }
62 \msg_new:nnn { beanoves } { :nn } { #1~(#2) }

63 \cs_new:Npn \BNVS_warning:n {
64   \msg_warning:nnn { beanoves } { :n }
65 }
66 \cs_new:Npn \BNVS_warning:x {
67   \msg_warning:nnx { beanoves } { :n }
68 }

69 \cs_new:Npn \BNVS_error:n {
70   \msg_error:nnn { beanoves } { :n }
71 }
72 \cs_generate_variant:Nn \BNVS_error:n { x }

73 \cs_new:Npn \BNVS_fatal:n {
74   \msg_fatal:nnn { beanoves } { :n }
75 }
76 \cs_new:Npn \BNVS_fatal:x {
77   \msg_fatal:nnx { beanoves } { :n }
78 }
79 \cs_new:Npn \BNVS_fatal_unreachable: {
80   \BNVS_fatal:n { Unreachable }
81 }
82 \cs_new:Npn \BNVS_fatal_unreachable: { }
```

1.3.4 Facility layer: Variables

```
\BNVS_N_new:c \BNVS_N_new:c {<type>}
\BNVS_v_new:c \BNVS_v_new:c {<type>}
```

Creates a collection of typed utility functions, see usage below. These functions are undefined when no longer used. *<type>* is one of *tl*, *seq*...

```
83 \cs_new:Npn \BNVS_N_new:c #1 {
84   \cs_new:cpn { BNVS_#1:c } ##1 {
85     1 \BNVS:c{ ##1 } \tl_if_empty:nF { ##1 } { _ } #1
86   }
87   \cs_new:cpn { BNVS_#1_new:c } ##1 {
88     \use:c { #1_new:c } { \use:c { BNVS_#1:c } { ##1 } }
89   }
90   \cs_new:cpn { BNVS_#1_use:c } ##1 {
91     \use:c { \cs:w BNVS_#1:c \cs_end: { ##1 } }
92   }
93   \cs_new:cpn { BNVS_#1_use:Nc } ##1 ##2 {
94     \BNVS_use_Raw:Nc
95     ##1 { \cs:w BNVS_#1:c \cs_end: { ##2 } }
96   }
97   \cs_new:cpn { BNVS_#1_use:nc } ##1 ##2 {
98     \BNVS_use_Raw:nc
99     { ##1 } { \cs:w BNVS_#1:c \cs_end: { ##2 } }
100  }
101 }

102 \cs_new:Npn \BNVS_v_new:c #1 {
103   \cs_new:cpn { BNVS_#1_use:Nv } ##1 ##2 {
104     \exp_args:Nv ##1
105     { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
106   }
107   \cs_new:cpn { BNVS_#1_use:cv } ##1 ##2 {
108     \exp_args:Nv \BNVS_use:c { ##1 }
109     { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
110   }
111   \cs_new:cpn { BNVS_#1_use:nv } ##1 ##2 {
112     \exp_args:Nv \use:n { ##1 }
113     { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
114   }
115 }

116 \BNVS_N_new:c { bool }
117 \BNVS_N_new:c { int }
118 \BNVS_v_new:c { int }
119 \BNVS_N_new:c { tl }
120 \BNVS_v_new:c { tl }
121 \cs_new:Npn \BNVS_tl_use:Nvv #1 #2 {
122   \BNVS_tl_use:nv { \BNVS_tl_use:Nv #1 { #2 } }
123 }

124 \cs_new:Npn \BNVS_tl_use:Nvvv #1 #2 {
125   \BNVS_tl_use:nvv { \BNVS_tl_use:Nv #1 { #2 } }
126 }
```

```

127 \cs_new:Npn \BNVS_tl_use:Nvvvv #1 #2 {
128   \BNVS_tl_use:nvvv { \BNVS_tl_use:Nv #1 { #2 } }
129 }

130 \BNVS_N_new:c { str }
131 \BNVS_v_new:c { str }
132 \BNVS_N_new:c { seq }
133 \BNVS_v_new:c { seq }
134 \cs_undefine:N \BNVS_N_new:c

```

\BNVS_use:Ncn \BNVS_use:Ncn *<function>* *{<core name>}* *{<type>}*

```

135 \cs_new:Npn \BNVS_use:Ncn #1 #2 #3 {
136   \BNVS_use_Raw:c { BNVS_#3_use:Nc }   #1   { #2 }
137 }

138 \cs_new:Npn \BNVS_use:ncn #1 #2 #3 {
139   \BNVS_use_Raw:c { BNVS_#3_use:nc } { #1 } { #2 }
140 }

141 \cs_new:Npn \BNVS_use:Nvn #1 #2 #3 {
142   \BNVS_use_Raw:c { BNVS_#3_use:Nv }   #1   { #2 }
143 }

144 \cs_new:Npn \BNVS_use:nvn #1 #2 #3 {
145   \BNVS_use_Raw:c { BNVS_#3_use:nv } { #1 } { #2 }
146 }

147 \cs_new:Npn \BNVS_use:Ncncn #1 #2 #3 {
148   \BNVS_use:ncn {
149     \BNVS_use:Ncn   #1   { #2 } { #3 }
150   }
151 }

152 \cs_new:Npn \BNVS_use:ncncn #1 #2 #3 {
153   \BNVS_use:ncn {
154     \BNVS_use:ncn { #1 } { #2 } { #3 }
155   }
156 }

157 \cs_new:Npn \BNVS_use:Nvncn #1 #2 #3 {
158   \BNVS_use:ncn {
159     \BNVS_use:Nvn   #1   { #2 } { #3 }
160   }
161 }

162 \cs_new:Npn \BNVS_use:nvncn #1 #2 #3 {
163   \BNVS_use:ncn {
164     \BNVS_use:nvn { #1 } { #2 } { #3 }
165   }
166 }

167 \cs_new:Npn \BNVS_use:Ncncncn #1 #2 #3 #4 #5 {
168   \BNVS_use:ncn {
169     \BNVS_use:Ncncn   #1   { #2 } { #3 } { #4 } { #5 }
170   }
171 }

```

```

172 \cs_new:Npn \BNVS_use:ncncncn #1 #2 #3 #4 #5 {
173   \BNVS_use:ncn {
174     \BNVS_use:ncncn { #1 } { #2 } { #3 } { #4 } { #5 }
175   }
176 }

```

\BNVS_new_c:cn \BNVS_new_c:nc {<type>} {<core name>}

```

177 \cs_new:Npn \BNVS_new_c:nc #1 #2 {
178   \BNVS_new:cpn { #1_#2:c } {
179     \BNVS_use_Raw:c { BNVS_#1_use:nc } { \BNVS_use_Raw:c { #1_#2:N } }
180   }
181 }

182 \cs_new:Npn \BNVS_new_cn:nc #1 #2 {
183   \BNVS_new:cpn { #1_#2:cn } ##1 {
184     \BNVS_use:ncn { \BNVS_use_Raw:c { #1_#2:Nn } } { ##1 } { #1 }
185   }
186 }

187 \cs_new:Npn \BNVS_new_cnn:ncN #1 #2 #3 {
188   \BNVS_new:cpn { #2:cnn } ##1 {
189     \BNVS_use:Ncn { #3 } { ##1 } { #1 }
190   }
191 }

192 \cs_new:Npn \BNVS_new_cnn:nc #1 #2 {
193   \BNVS_use_Raw:nc {
194     \BNVS_new_cnn:ncN { #1 } { #1_#2 }
195   } { #1_#2:Nnn }
196 }

197 \cs_new:Npn \BNVS_new_cnv:ncN #1 #2 #3 {
198   \BNVS_new:cpn { #2:cnv } ##1 ##2 {
199     \BNVS_tl_use:nv {
200       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
201     }
202   }
203 }

204 \cs_new:Npn \BNVS_new_cnv:nc #1 #2 {
205   \BNVS_use_Raw:nc {
206     \BNVS_new_cnv:ncN { #1 } { #1_#2 }
207   } { #1_#2:Nnn }
208 }

209 \cs_new:Npn \BNVS_new_cnx:ncN #1 #2 #3 {
210   \BNVS_new:cpn { #2:cnx } ##1 ##2 {
211     \exp_args:Nnx \use:n {
212       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
213     }
214   }
215 }

```



```

216 \cs_new:Npn \BNVS_new_cnx:nc #1 #2 {
217   \BNVS_use_Raw:nc {
218     \BNVS_new_cnx:ncN { #1 } { #1_#2 }
219   } { #1_#2:Nnn }
220 }

221 \cs_new:Npn \BNVS_new_cc:ncNn #1 #2 #3 #4 {
222   \BNVS_new_cpn { #2:cc } ##1 ##2 {
223     \BNVS_use:Ncncn #3 { ##1 } { #1 } { ##2 } { #4 }
224   }
225 }

226 \cs_new:Npn \BNVS_new_cc:ncn #1 #2 {
227   \BNVS_use_Raw:nc {
228     \BNVS_new_cc:ncNn { #1 } { #1_#2 }
229   } { #1_#2:NN }
230 }

231 \cs_new:Npn \BNVS_new_cc:nc #1 #2 {
232   \BNVS_new_cc:ncn { #1 } { #2 } { #1 }
233 }

234 \cs_new:Npn \BNVS_new_cn:ncNn #1 #2 #3 #4 {
235   \BNVS_new_cpn { #2:cn } ##1 {
236     \BNVS_use:Ncn #3 { ##1 } { #1 }
237   }
238 }

239 \cs_new:Npn \BNVS_new_cn:ncn #1 #2 {
240   \BNVS_use_Raw:nc {
241     \BNVS_new_cn:ncNn { #1 } { #1_#2 }
242   } { #1_#2:Nn }
243 }

244 \cs_new:Npn \BNVS_new_cv:ncNn #1 #2 #3 #4 {
245   \BNVS_new_cpn { #2:cv } ##1 ##2 {
246     \BNVS_use:nvn {
247       \BNVS_use:Ncn #3 { ##1 } { #1 }
248     } { ##2 } { #4 }
249   }
250 }

251 \cs_new:Npn \BNVS_new_cv:ncn #1 #2 {
252   \BNVS_use_Raw:nc {
253     \BNVS_new_cv:ncNn { #1 } { #1_#2 }
254   } { #1_#2:Nn }
255 }

256 \cs_new:Npn \BNVS_new_cv:nc #1 #2 {
257   \BNVS_new_cv:ncn { #1 } { #2 } { #1 }
258 }

259 \cs_new:Npn \BNVS_l_use:Ncn #1 #2 #3 {
260   \BNVS_use_Raw:Nc #1 { \BNVS_l:cn { #2 } { #3 } }
261 }

```

```

262 \cs_new:Npn \BNVS_l_use:ncn #1 #2 #3 {
263   \BNVS_use_Raw:nc { #1 } { \BNVS_l:cn { #2 } { #3 } }
264 }

265 \cs_new:Npn \BNVS_g_use:Ncn #1 #2 #3 {
266   \BNVS_use_Raw:Nc #1 { \BNVS_g:cn { #2 } { #3 } }
267 }

268 \cs_new:Npn \BNVS_g_use:ncn #1 #2 #3 {
269   \BNVS_use_Raw:nc { #1 } { \BNVS_g:cn { #2 } { #3 } }
270 }

```

```

\BNVS_new_conditional:cpnn \BNVS_new_conditional:cpnn {<core>} <parameter> {<conditions>} {<code>}

```

```

271 \cs_generate_variant:Nn \prg_new_conditional:Npnn { c }

272 \cs_new:Npn \BNVS_new_conditional:cpnn #1 {
273   \prg_new_conditional:cpnn { \BNVS:c { #1 } }
274 }

275 \cs_generate_variant:Nn \prg_generate_conditional_variant:Nnn { c }
276 \cs_new:Npn \BNVS_generate_conditional_variant:cnn #1 {
277   \prg_generate_conditional_variant:cnn { \BNVS:c { #1 } }
278 }

279 \cs_new:Npn \BNVS_new_conditional_vn:cNnn #1 #2 #3 #4 {
280   \BNVS_new_conditional:cpnn { #1:vn } ##1 ##2 { #4 } {
281     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
282       \prg_return_true:
283     } {
284       \prg_return_false:
285     }
286   }
287 }

288 \cs_new:Npn \BNVS_new_conditional_vn:cnn #1 #2 {
289   \BNVS_use:nc {
290     \BNVS_new_conditional_vn:cNnn { #1 }
291   } { #1:nn TF } { #2 }
292 }

293 \cs_new:Npn \BNVS_new_conditional_vc:cNnn #1 #2 #3 #4 {
294   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #4 } {
295     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
296       \prg_return_true:
297     } {
298       \prg_return_false:
299     }
300   }
301 }

302 \cs_new:Npn \BNVS_new_conditional_vc:cnn #1 {
303   \BNVS_use:nc {
304     \BNVS_new_conditional_vc:cNnn { #1 }
305   } { #1:ncTF }
306 }

```

```

307 \cs_new:Npn \BNVS_new_conditional_vvc:cNnnn #1 #2 #3 #4 #5 {
308   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #5 } {
309     \BNVS_use:nvn {
310       \BNVS_use:Nvn #2 { ##1 } { #3 }
311     } { ##2 } { #4 } { ##3 } {
312       \prg_return_true:
313     } {
314       \prg_return_false:
315     }
316   }
317 }

318 \cs_new:Npn \BNVS_new_conditional_vvc:cnnn #1 {
319   \BNVS_use:nc {
320     \BNVS_new_conditional_vvc:cNnnn { #1 }
321   } { #1:nncTF }
322 }

323 \cs_new:Npn \BNVS_new_conditional_vc:cNn #1 #2 #3 {
324   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #3 } {
325     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } {
326       \prg_return_true:
327     } {
328       \prg_return_false:
329     }
330   }
331 }

332 \cs_new:Npn \BNVS_new_conditional_vc:cn #1 {
333   \BNVS_use:nc {
334     \BNVS_new_conditional_vc:cNn { #1 }
335   } { #1:ncTF }
336 }

337 \cs_new:Npn \BNVS_new_conditional_vvc:cNn #1 #2 #3 {
338   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #3 } {
339     \BNVS_tl_use:nv {
340       \BNVS_tl_use:Nv #2 { ##1 }
341     } { ##2 } { ##3 } {
342       \prg_return_true:
343     } {
344       \prg_return_false:
345     }
346   }
347 }

348 \cs_new:Npn \BNVS_new_conditional_vvc:cn #1 {
349   \BNVS_use:nc {
350     \BNVS_new_conditional_vvc:cNn { #1 }
351   } { #1:nncTF }
352 }

```

1.3.5 Regex

```

353 \cs_new:Npn \BNVS_regex_use:Nc #1 #2 {
354   \BNVS_use_Raw:Nc #1 { c \BNVS:c { #2 } _regex }
355 }

```

1.3.6 Token lists

<u>__bnvs_tl_clear:c</u>	<u>__bnvs_tl_clear:c {<core key tl>}</u>
<u>__bnvs_tl_clear_new:c</u>	<u>__bnvs_tl_use:c {<core>}</u>
<u>__bnvs_tl_new</u>	<u>__bnvs_tl_count:c {<core>}</u>
<u>__bnvs_tl_set_eq:cc</u>	<u>__bnvs_tl_set_eq:cc {<lhs core name>} {<rhs core name>}</u>
<u>__bnvs_tl_set:cn</u>	<u>__bnvs_tl_set:cn {<core>} {<tl>}</u>
<u>__bnvs_tl_set:(cv cx ce co)</u>	<u>__bnvs_tl_set:cv {<core>} {<value core name>}</u>
<u>__bnvs_tl_put_left:cn</u>	<u>__bnvs_tl_put_left:cn {<core>} {<tl>}</u>
<u>__bnvs_tl_put_right:cn</u>	<u>__bnvs_tl_put_right:cn {<core>} {<tl>}</u>
<u>__bnvs_tl_put_right:(cx ce cv co)</u>	<u>__bnvs_tl_put_right:cv {<core>} {<value core name>}</u>
	<u>__bnvs_tl_put_right:co {<core>} {<tl>}</u>

These are shortcuts to

- \tl_clear:c {l__bnvs_<core>_tl}
- \tl_clear_new:c {l__bnvs_<core>_tl}
- \tl_use:c {l__bnvs_<core>_tl}
- \tl_set_eq:cc {l__bnvs_<lhs core>_tl} {l__bnvs_<rhs core>_tl}
- \tl_set:cv {l__bnvs_<core>_tl} {l__bnvs_<value core>_tl}
- \tl_set:cx {l__bnvs_<core>_tl} {<tl>}
- \tl_set:co {l__bnvs_<core>_tl} {<tl>}
- \tl_put_left:cn {l__bnvs_<core>_tl} {<tl>}
- \tl_put_right:cn {l__bnvs_<core>_tl} {<tl>}
- \tl_put_right:cv {l__bnvs_<core>_tl} {l__bnvs_<value core>_tl}
- \tl_put_right:co {l__bnvs_<core>_tl} {<tl>}

<u>__bnvs_tl_clear_all:n</u>	<u>__bnvs_tl_clear_all:n {<cores>}</u>
-------------------------------	---

Run __bnvs_tl_clear:c for each individual item of <cores>.

```

356 \BNVS_new:cpn { tl_clear_all:n } #1 {
357   \tl_map_inline:nn { #1 } {
358     \__bnvs_tl_clear:c { ##1 }
359   }
360 }

```

<u>\BNVS_new_conditional_vnc:cn</u>	<u>\BNVS_new_conditional_vnc:cn {<core>} {<conditions>}</u>
-------------------------------------	---

Creates <core>:vncTF from <core>:nncTF.

```

361 \cs_new:Npn \BNVS_new_conditional_vnc:cNn #1 #2 #3 {
362   \BNVS_new_conditional:cpnn { #1:vnc } ##1 ##2 ##3 { #3 } {
363     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } { ##3 } {
364       \prg_return_true:
365     } {
366       \prg_return_false:
367     }
368   }
369 }

370 \cs_new:Npn \BNVS_new_conditional_vnc:cn #1 {
371   \BNVS_use:nc {
372     \BNVS_new_conditional_vnc:cNn { #1 }
373   } { #1:nnctf }
374 }

```

\BNVS_new_conditional_vvnc:cn **\BNVS_new_conditional_vvnc:cn** {<core>} {<conditions>}

Creates <core>:vvncTF from <core>:nnctf.

```

375 \cs_new:Npn \BNVS_new_conditional_vvnc:cNn #1 #2 #3 {
376   \BNVS_new_conditional:cpnn { #1:vvnc } ##1 ##2 ##3 ##4 { #3 } {
377     \BNVS_tl_use:nv {
378       \BNVS_tl_use:Nv #2 { ##1 }
379     } { ##2 } { ##3 } { ##4 } {
380       \prg_return_true:
381     } {
382       \prg_return_false:
383     }
384   }
385 }

386 \cs_new:Npn \BNVS_new_conditional_vvnc:cn #1 {
387   \BNVS_use:nc {
388     \BNVS_new_conditional_vvnc:cNn { #1 }
389   } { #1:nnctf }
390 }

391 \cs_new:Npn \BNVS_new_conditional_vvvc:cNn #1 #2 #3 {
392   \BNVS_new_conditional:cpnn { #1:vvvc } ##1 ##2 ##3 ##4 { #3 } {
393     \BNVS_tl_use:nvv {
394       \BNVS_tl_use:Nv #2 { ##1 }
395     } { ##2 } { ##3 } { ##4 } {
396       \prg_return_true:
397     } {
398       \prg_return_false:
399     }
400   }
401 }

```

\BNVS_new_conditional_vvvc:cn **\BNVS_new_conditional_vvvc:cn** {<core>} {<conditions>}

Creates <core>:vvvcTF from <core>:nnctf.

```

402 \cs_new:Npn \BNVS_new_conditional_vvvc:cn #1 {

```

```

403 \BNVS_use:nc {
404     \BNVS_new_conditional_vvvc:cNn { #1 }
405 } { #1:nnncTF }
406 }

407 \cs_new:Npn \BNVS_new_tl_c:c {
408     \BNVS_new_c:nc { tl }
409 }
410 \BNVS_new_tl_c:c { clear }
411 \BNVS_new_tl_c:c { clear_new }
412 \BNVS_new_tl_c:c { use }
413 \BNVS_new_tl_c:c { count }

414 \BNVS_new:cpn { tl_set:eq:cc } #1 #2 {
415     \BNVS_use:ncncn { \tl_set:eq:NN } { #1 } { tl } { #2 } { tl }
416 }

417 \cs_new:Npn \BNVS_new_tl_cn:c {
418     \BNVS_new_cn:nc { tl }
419 }

420 \cs_new:Npn \BNVS_new_tl_cv:c #1 {
421     \BNVS_new_cv:ncn { tl } { #1 } { tl }
422 }
423 \BNVS_new_tl_cn:c { set }
424 \BNVS_new_tl_cv:c { set }

425 \BNVS_new:cpn { tl_set:cx } {
426     \exp_args:Nnx \__bnvs_tl_set:cn
427 }

428 \BNVS_new:cpn { tl_set:ce } {
429     \exp_args:Nne \__bnvs_tl_set:cn
430 }

431 \BNVS_new:cpn { tl_set:co } {
432     \exp_args:Nno \__bnvs_tl_set:cn
433 }
434 \BNVS_new_tl_cn:c { put_right }
435 \BNVS_new_tl_cv:c { put_right }

436 \BNVS_new:cpn { tl_put_right:cx } {
437     \exp_args:Nnnx \BNVS_use:c { tl_put_right:cn }
438 }

439 \BNVS_new:cpn { tl_put_right:ce } {
440     \exp_args:Nnne \BNVS_use:c { tl_put_right:cn }
441 }
442 \BNVS_new:cpn { tl_put_right:co } {
443     \exp_args:Nnno \BNVS_use:c { tl_put_right:cn }
444 }
445 \BNVS_new_tl_cn:c { put_left }
446 \BNVS_new_tl_cv:c { put_left }

447 \BNVS_new:cpn { tl_put_left:cx } {
448     \exp_args:Nnnx \BNVS_use:c { tl_put_left:cn }
449 }

```

```

\__bnvs_tl_if_exist:cTF \__bnvs_tl_if_empty:cTF  {\<core>} {\<yes:>} {\<no:>}}
\__bnvs_tl_if_empty:cTF \__bnvs_tl_if_exist:cTF  {\<core>} {\<yes:>} {\<no:>}}
\__bnvs_tl_if_blank:vTF \__bnvs_tl_if_blank:vTF  {\<core>} {\<yes:>} {\<no:>}}
\__bnvs_tl_if_eq:cnTF \__bnvs_tl_if_eq:cnTF  {\<core>} {\<tl>} {\<yes:>} {\<no:>}}

```

These are shortcuts to

- \tl_if_exist:cTF {l__bnvs_<core>_tl} {\<yes:>} {\<no:>}}
- \tl_if_empty:cTF {l__bnvs_<core>_tl} {\<yes:>} {\<no:>}}
- \tl_if_eq:cnTF {l__bnvs_<core>_tl} {\<tl>} {\<yes:>} {\<no:>}}

```

450 \cs_new:Npn \BNVS_new_conditional_c:ncNn #1 #2 #3 #4 {
451   \BNVS_new_conditional:cpnn { #2 } ##1 { #4 } {
452     \BNVS_use:Ncn #3 { ##1 } { #1 } {
453       \prg_return_true:
454     } {
455       \prg_return_false:
456     }
457   }
458 }

459 \cs_new:Npn \BNVS_new_conditional_c:ncn #1 #2 {
460   \BNVS_use_Raw:nc {
461     \BNVS_new_conditional_c:ncNn { #1 } { #1_#2:c }
462   } { #1_#2:NTF }
463 }
464 \BNVS_new_conditional_c:ncn { tl } { if_empty } { p, T, F, TF }
465 \BNVS_new_conditional_c:ncn { tl } { if_exist } { p, T, F, TF }

466 \BNVS_new_conditional:cpnn { tl_if_blank:v } #1 { T, F, TF } {
467   \BNVS_tl_use:Nv \tl_if_blank:nTF { #1 } {
468     \prg_return_true:
469   } {
470     \prg_return_false:
471   }
472 }

473 \cs_new:Npn \BNVS_new_conditional_cn:ncNn #1 #2 #3 #4 {
474   \BNVS_new_conditional:cpnn { #2:cn } ##1 ##2 { #4 } {
475     \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } {
476       \prg_return_true:
477     } {
478       \prg_return_false:
479     }
480   }
481 }

482 \cs_new:Npn \BNVS_new_conditional_cn:ncn #1 #2 {
483   \BNVS_use_Raw:nc {
484     \BNVS_new_conditional_cn:ncNn { #1 } { #1_#2 }
485   } { #1_#2:NnTF }
486 }
487 \BNVS_new_conditional_cn:ncn { tl } { if_eq } { T, F, TF }

```

```

488 \cs_new:Npn \BNVS_new_conditional_cv:ncNn #1 #2 #3 #4 {
489   \BNVS_new_conditional_cpnn { #2:cv } ##1 ##2 { #4 } {
490     \BNVS_use:nvn {
491       \BNVS_use:Ncn #3 { ##1 } { #1 }
492       } { ##2 } { #1 } {
493       \prg_return_true:
494     } {
495       \prg_return_false:
496     }
497   }
498 }

499 \cs_new:Npn \BNVS_new_conditional_cv:ncn #1 #2 {
500   \BNVS_use_Raw:nc {
501     \BNVS_new_conditional_cv:ncNn { #1 } { #1_#2 }
502   } { #1_#2:NnTF }
503 }
504 \BNVS_new_conditional_cv:ncn { t1 } { if_eq } { T, F, TF }

505 \BNVS_new:cpn { gput_right:ccn } #1 #2 #3 {
506   \tl_gput_right:cn { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
507 }

```

1.3.7 Strings

`\__bnvs_str_if_eq:vvTF` `\__bnvs_str_if_eq:vvTF {<core>} {<tl>} {<yes:>} {<no:>}`

These are shortcuts to

- `\str_if_eq:ccTF l__bnvs_<core>_tl l__bnvs_<core>_tl {<yes:>} {<no:>}`

```

508 \cs_new:Npn \BNVS_new_conditional_vv:cNn #1 #2 #3 {
509   \BNVS_new_conditional_cpnn { #1:vv } ##1 ##2 { #3 } {
510     \BNVS_tl_use:nv {
511       \BNVS_tl_use:Nv #2 { ##1 }
512     } { ##2 } {
513       \prg_return_true:
514     } {
515       \prg_return_false:
516     }
517   }
518 }

519 \cs_new:Npn \BNVS_new_conditional_vv:cn #1 {
520   \BNVS_use:nc {
521     \BNVS_new_conditional_vvnc:cNn { #1 }
522   } { #1:nnTF }
523 }

524 \cs_new:Npn \BNVS_new_conditional_vn:ncNn #1 #2 #3 #4 {
525   \BNVS_new_conditional_cpnn { #2:vn } ##1 ##2 { #4 } {
526     \BNVS_use:Nvn #3 { ##1 } { #1 } { ##2 } {
527       \prg_return_true:
528     } {

```



```

529     \prg_return_false:
530   }
531 }
532 }

533 \cs_new:Npn \BNVS_new_conditional_vn:ncn #1 #2 {
534   \BNVS_use_Raw:nc {
535     \BNVS_new_conditional_vn:ncNn { #1 } { #1_#2 }
536   } { #1_#2:nnTF }
537 }
538 \BNVS_new_conditional_vn:ncn { str } { if_eq } { T, F, TF }

539 \cs_new:Npn \BNVS_new_conditional_vv:ncNn #1 #2 #3 #4 {
540   \BNVS_new_conditional:cpnn { #2:vv } ##1 ##2 { #4 } {
541     \BNVS_use:nvn {
542       \BNVS_use:Nvn #3 { ##1 } { #1 }
543     } { ##2 } { #1 } {
544       \prg_return_true:
545     } {
546       \prg_return_false:
547     }
548   }
549 }

550 \cs_new:Npn \BNVS_new_conditional_vv:ncn #1 #2 {
551   \BNVS_use_Raw:nc {
552     \BNVS_new_conditional_vv:ncNn { #1 } { #1_#2 }
553   } { #1_#2:nnTF }
554 }
555 \BNVS_new_conditional_vv:ncn { str } { if_eq } { T, F, TF }

```

1.3.8 Sequences

<code>__bnvs_seq_count:c</code>	<code>__bnvs_seq_new:c {<core>}</code>
<code>__bnvs_seq_clear:c</code>	<code>__bnvs_seq_count:c {<core>}</code>
<code>__bnvs_seq_set_eq:cc</code>	<code>__bnvs_seq_clear:c {<core>}</code>
<code>__bnvs_seq_gset_eq:cc</code>	<code>__bnvs_seq_set_eq:cc {<core₁>} {<core₂>}</code>
<code>__bnvs_seq_use:cn</code>	<code>__bnvs_seq_use:cn {<core>} {<separator>}</code>
<code>__bnvs_seq_item:cn</code>	<code>__bnvs_seq_item:cn {<core>} {<integer expression>}</code>
<code>__bnvs_seq_remove_all:cn</code>	<code>__bnvs_seq_remove_all:cn {<core>} {<t1>}</code>
<code>__bnvs_seq_put_left:cn</code>	<code>__bnvs_seq_put_right:cn {<seq core>} {<t1>}</code>
<code>__bnvs_seq_put_left:cv</code>	<code>__bnvs_seq_put_right:cv {<seq core>} {<t1 core>}</code>
<code>__bnvs_seq_put_right:cn</code>	<code>__bnvs_seq_set_split:cnn {<seq core>} {<t1>} {<separator>}</code>
<code>__bnvs_seq_put_right:cv</code>	<code>__bnvs_seq_pop_left:cc {<core₁>} {<core₂>}</code>
<code>__bnvs_seq_set_split:cnn</code>	
<code>__bnvs_seq_set_split:(cnv cnx)</code>	
<code>__bnvs_seq_pop_left:cc</code>	

These are shortcuts to

- `\seq_set_eq:cc {l__bnvs_<core1>_seq} {l__bnvs_<core2>_seq}`
- `\seq_count:c {l__bnvs_<core>_seq}`
- `\seq_use:cn {l__bnvs_<core>_seq} {<separator>}`
- `\seq_item:cn {l__bnvs_<core>_seq} {<integer expression>}`
- `\seq_remove_all:cn {l__bnvs_<core>_seq} {<t1>}`
- `\seq_clear:c {l__bnvs_<core>_seq}`
- `\seq_put_right:cv {l__bnvs_<core>_seq} {l__bnvs_<core>_t1}`
- `\seq_set_split:cnn {l__bnvs_<core>_seq} {l__bnvs_<core>_t1}{<t1>}`

```

556 \BNVS_new_c:nc { seq } { count }
557 \BNVS_new_c:nc { seq } { clear }
558 \BNVS_new_cn:nc { seq } { use }
559 \BNVS_new_cn:nc { seq } { item }
560 \BNVS_new_cn:nc { seq } { remove_all }
561 \BNVS_new_cn:nc { seq } { map_inline }
562 \BNVS_new_cc:nc { seq } { set_eq }
563 \BNVS_new_cc:nc { seq } { gset_eq }
564 \BNVS_new_cn:ncn { seq } { put_left } { t1 }
565 \BNVS_new_cv:ncn { seq } { put_left } { t1 }
566 \BNVS_new_cn:ncn { seq } { put_right } { t1 }
567 \BNVS_new_cv:ncn { seq } { put_right } { t1 }
568 \BNVS_new_cnn:nc { seq } { set_split }
569 \BNVS_new_cnv:nc { seq } { set_split }
570 \BNVS_new_cnx:nc { seq } { set_split }
571 \BNVS_new_cc:ncn { seq } { pop_left } { t1 }
572 \BNVS_new_cc:ncn { seq } { pop_right } { t1 }

```

```

__bnvs_seq_if_empty:cTF  __bnvs_seq_if_empty:cTF {<seq core>} {<yes:>} {<no:>}
__bnvs_seq_get_right:ccTF __bnvs_seq_get_right:ccTF {<seq core>} {<tl core>} {<yes:>} {<no:>}
__bnvs_seq_pop_left:ccTF
__bnvs_seq_pop_right:ccTF

```

```

573 \cs_new:Npn \BNVS_new_conditional_cc:ncnn #1 #2 #3 #4 {
574   \BNVS_new_conditional:cpnn { #1_#2:cc } ##1 ##2 { #4 } {
575     \BNVS_use:ncncn {
576       \BNVS_use_Raw:c { #1_#2:NNTF }
577     } { ##1 } { #1 } { ##2 } { #3 } {
578       \prg_return_true:
579     } {
580       \prg_return_false:
581     }
582   }
583 }
584 \BNVS_new_conditional_c:ncn { seq } { if_empty } { T, F, TF }
585 \BNVS_new_conditional_cc:ncnn
586 { seq } { get_right } { tl } { T, F, TF }
587 \BNVS_new_conditional_cc:ncnn
588 { seq } { pop_left } { tl } { T, F, TF }
589 \BNVS_new_conditional_cc:ncnn
590 { seq } { pop_right } { tl } { T, F, TF }

```

1.3.9 Integers

```

__bnvs_int_new:c  __bnvs_int_new:c {<core>}
__bnvs_int_use:c  __bnvs_int_use:c {<core>}
__bnvs_int_zero:c __bnvs_int_incr:c {<core>}
__bnvs_int_inc:c  __bnvs_int_decr:c {<core>}
__bnvs_int_decr:c __bnvs_int_set:cn {<core>} {<value>}
__bnvs_int_set:cn
__bnvs_int_set:cv

```

These are shortcuts to

- `\int_new:c {l__bnvs_<core>_int}`
- `\int_use:c {l__bnvs_<core>_int}`
- `\int_incr:c {l__bnvs_<core>_int}`
- `\int_decr:c {l__bnvs_<core>_int}`
- `\int_set:cn {l__bnvs_<core>_int} {<value>}`

```

591 \BNVS_new_c:nc { int } { new }
592 \BNVS_new_c:nc { int } { use }
593 \BNVS_new_c:nc { int } { zero }
594 \BNVS_new_c:nc { int } { incr }
595 \BNVS_new_c:nc { int } { decr }
596 \BNVS_new_cn:nc { int } { set }
597 \BNVS_new_cv:ncn { int } { set } { int }

```

1.4 Debug facilities

Typesetting file `beanoves.dtx` creates both `beanoves` and `beanoves-debug` style files. The former is intended for everyday use whereas the latter contains supplemental debugging and testing facilities which are intentionally left undocumented. For example, we have aliases for `\group_begin:` and `\group_end:` to allow the display of supplemental informations while debugging. We also have some techniques for coverage as well as inline testing.

1.5 Local variables

We make heavy use of local variables and function scopes. Many functions are executed within a `TeX` group, which ensures no name collision with the caller stack. The number of variables used has not been optimized, nor the `TeX` groups used. Optimization often goes against readability. Next local variables are used by different functions. These are one word letter variables.

`\l__bnvs_A_tl:` ,

`\l__bnvs_Z_tl:` ,

`\l__bnvs_L_tl:` Start, end and lentgh of a range

```
598      \tl_new:N \l__bnvs_A_tl
599      \tl_new:N \l__bnvs_Z_tl
600      \tl_new:N \l__bnvs_L_tl
```

`\l__bnvs_C_tl:` A function name in a call, as regex caught group

`\l__bnvs_I_tl:` A frame identifier

`\l__bnvs_J_tl:` The current frame identifier

`\l__bnvs_S_tl:` A short name, a regex caught group.

`\l__bnvs_R_tl:` A relative, a regex caught group.

`\l__bnvs_P_tl:` A path, a regex caught group.

`\l__bnvs_M_tl:` An assignment modifier, a regex caught group.

```
601      \tl_new:N \l__bnvs_I_tl
602      \tl_new:N \l__bnvs_J_tl
603      \tl_new:N \l__bnvs_K_tl
604      \tl_new:N \l__bnvs_S_tl
605      \tl_new:N \l__bnvs_R_tl
606      \tl_new:N \l__bnvs_P_tl
607      \tl_new:N \l__bnvs_M_tl
```

`\l__bnvs_scanned_tl:` To store the scanned material.

```
608      \tl_new:N \l__bnvs_xprt_tl
609      \tl_new:N \l__bnvs_xprted_Raw_tl
610      \tl_new:N \l__bnvs_xprted_tl
```

\l__bnvs_N_tl: The representation of an unsigned decimal integer.

```
611      \tl_new:N \l__bnvs_N_tl
```

\l__bnvs_V_tl: A number value as opposit of a range value.

```
612      \tl_new:N \l__bnvs_V_tl
```

\l__bnvs_H_tl: The left hand side of an assignment.

```
613      \tl_new:N \l__bnvs_H_tl
614 \tl_new:N \l__bnvs_B_tl
615 \tl_new:N \l__bnvs_T_tl
616 \tl_new:N \l__bnvs_U_tl
617 \tl_new:N \l__bnvs_F_tl
618 \tl_new:N \l__bnvs_Q_tl
619 \tl_new:N \l__bnvs_a_tl
620 \tl_new:N \l__bnvs_z_tl
621 \tl_new:N \l__bnvs_l_tl
622 \tl_new:N \l__bnvs_r_tl
623 \tl_new:N \l__bnvs_n_tl
624 \tl_new:N \l__bnvs_ref_tl
625 \tl_new:N \l__bnvs_v_tl
626 \tl_new:N \l__bnvs_b_tl
627 \tl_new:N \l__bnvs_c_tl
628 \tl_new:N \l__bnvs_ans_tl
629 \tl_new:N \l__bnvs_base_tl
630 \tl_new:N \l__bnvs_group_tl
631 \tl_new:N \l__bnvs_scaN_tl
632 \tl_new:N \l__bnvs_query_tl
633 \tl_new:N \l__bnvs_n_incr_tl
634 \tl_new:N \l__bnvs_incr_tl
635 \tl_new:N \l__bnvs_plus_tl
636 \tl_new:N \l__bnvs_rhs_tl
637 \tl_new:N \l__bnvs_post_tl
638 \tl_new:N \l__bnvs_suffix_tl
639 \tl_new:N \l__bnvs_index_tl
640 \int_new:N \g__bnvs_call_int
641 \int_new:N \l__bnvs_i_int
642 \seq_new:N \g__bnvs_def_seq
643 \seq_new:N \l__bnvs_a_seq
644 \seq_new:N \l__bnvs_b_seq
645 \seq_new:N \l__bnvs_ans_seq
646 \seq_new:N \l__bnvs_Q_seq
647 \seq_new:N \l__bnvs_R_seq
648 \seq_new:N \l__bnvs_match_seq
649 \seq_new:N \l__bnvs_split_seq
650 \seq_new:N \l__bnvs_P_seq
651 \bool_new:N \l__bnvs_parse_bool
652 \bool_set_false:N \l__bnvs_parse_bool
653 \bool_new:N \l__bnvs_deep_bool
654 \bool_set_false:N \l__bnvs_deep_bool
655 \bool_new:N \l__bnvs_no_range_bool
656 \bool_set_false:N \l__bnvs_no_range_bool
```

```

657 \BNVS_new:cpn { tl_clear_ans: } {
658   \__bnvs_tl_clear:c { ans }
659 }

660 \cs_new:Npn \BNVS_error_ans:x {
661   \__bnvs_tl_put_right:cn { ans } 0
662   \BNVS_error:x
663 }

```

In order to implement the provide feature, we add getters and setters

```

664 \BNVS_new:cpn { set_true:c } #1 {
665   \exp_args:Nc \bool_set_true:N { \BNVS_l:cn { #1 } { bool } }
666 }

667 \BNVS_new:cpn { set_false:c } #1 {
668   \exp_args:Nc \bool_set_false:N { \BNVS_l:cn { #1 } { bool } }
669 }

```

1.6 Infinite loop management

Unending recursivity is managed here.

`\g__bnvs_call_int` Some functions calls, as well as some loop bodies, decrement this counter. When this counter reaches 0, an error is raised or a computation is aborted.

(End of definition for `\g__bnvs_call_int`.)

```

670 \int_const:Nn \c__bnvs_max_call_int { 8192 }

```

`\__bnvs_greset_call:` `\__bnvs_greset_call:`

Reset globally the call stack counter to its maximum value.

```

671 \BNVS_new:cpn { greset_call: } {
672   \int_gset:Nn \g__bnvs_call_int { \c__bnvs_max_call_int }
673 }

```

`\__bnvs_if_call:TF` `\__bnvs_call_do:TF {<yes:>} {<no:>}`

Decrement the `\g__bnvs_call_int` counter globally and execute `<yes:>` if we have not reached 0, `<no:>` otherwise.

```

674 \BNVS_new_conditional:cpnn { if_call: } { T, F, TF } {
675   \int_gdecr:N \g__bnvs_call_int
676   \int_compare:nNnTF \g__bnvs_call_int > 0 {
677     \prg_return_true:
678   } {
679     \prg_return_false:
680   }
681 }

```

1.7 Overlay specification

1.7.1 Registration

We keep track of the $\langle id \rangle ! \langle path \rangle$ combinations and provide looping mechanisms. Hereafter, $\langle id \rangle$, $\langle path \rangle$ and $\langle key \rangle$ allways represent pure strings.

```

__bnvs_g_IPK:w  __bnvs_g_IPK:w {<id>} {<path>} {<key>}
__bnvs_g_IP:w   __bnvs_g_IP:w  {<id>} {<path>}
__bnvs_g_I:w    __bnvs_g_I:w   {<id>}

```

Create a unique global name from the arguments.

```

682 \BNVS_new:cpn { g_IPK:w } #1 #2 #3 { g__bnvs@#1!#2/#3_tl }
683 \BNVS_new:cpn { g_IP:w } #1 #2 { g__bnvs@#1!#2_keys_seq }
684 \BNVS_new:cpn { g_I:w } #1 { g__bnvs@#1_paths_seq }

```

\g__bnvs_I_seq Global list of globally registered identifiers.

(End of definition for \g__bnvs_I_seq.)

```

685 \seq_new:N \g__bnvs_I_seq

```

1.7.2 Data model

The whole data model is somehow similar to a hierarchical file system: the $\langle id \rangle$ corresponds to the drive, the $\langle path \rangle$ points to a directory, each directory may contain individual files. File named “=” contains initial data whereas file named “@” contains the static resolved data. The latter is initially undefined and becomes empty as soon as the initial data is defined or set. In order to allow index override, files respectively named “ $\langle N \rangle$ ” and “ $@ \langle N \rangle$ ” are used similarly, $\langle N \rangle$ denoting a normalized integer (no leading 0, no leading + sign). The dot (.) is used to separate components of $\langle path \rangle$.

1.7.3 Low level IPK model functions

These are `gset` and `gunset` functions. Each `gset` function must be balanced by a `gunset` function to prevent “memory leaks”.

```

__bnvs_gset:nnnn  __bnvs_gset:nnnn {<id>} {<path>} {<key>} {<def>}
__bnvs_gset:(nnnv|nnne|vnnn|nvnn|nvvn|vvnn|vvvnv)

```

Manage the storage. Keep track of all the $\langle path \rangle$ ’s for a given $\langle id \rangle$ and all the $\langle key \rangle$ ’s for a given $\langle id \rangle ! \langle path \rangle$ combination.

Implementation details, next macros are defined lazily at the global level:

- `__bnvs_gset@<id>!<path>:nn {<key>} {<def>}`,
- `__bnvs_gset@<id>:nnn {<path>} {<key>} {<def>}`,
- `\g__bnvs@<id>!_paths_tl`, records all $\langle path \rangle$ for $\langle id \rangle$,
- `\g__bnvs@<id>!<path>_keys_tl`, records all $\langle key \rangle$ for $\langle id \rangle ! \langle path \rangle$,
- `\g__bnvs@<id>!<path>_keys_tl`,

- `\g__bnvs@⟨id⟩!⟨path⟩/⟨key⟩_tl`, record the `⟨def⟩` for the `⟨id⟩!⟨path⟩/⟨key⟩` combination.

```

686 \BNVS_new:cpn { gset:Nn } #1 {
687   \tl_gclear_new:N #1
688   \tl_gset:Nn #1
689 }
690 \BNVS_new:cpn { gset_IPKV:NNw } #1 #2 #3 #4 {
691   \seq_gclear_new:N #1
692   \cs_new:Npn #2 ##1 ##2 {
693     \seq_if_in:NnF #1 { ##1 } {
694       \seq_gput_right:Nn #1 { ##1 }
695     }
696     \exp_args:Nc \__bnvs_gset:Nn {
697       \__bnvs_g_IPK:w { #3 } { #4 } { ##1 }
698     } { ##2 }
699   }
700   #2
701 }
702 \BNVS_new:cpn { gset:NNnnnn } #1 #2 #3 #4 {
703   \cs_if_exist_use:NF #1 {
704     \seq_gput_right:Nn \g__bnvs_I_seq { #3 }
705     \seq_gclear_new:N #2
706     \cs_new:Npn #1 ##1 {
707       \seq_if_in:NnF #2 { ##1 } {
708         \seq_gput_right:Nn #2 { ##1 }
709       }
710       \exp_args:Ncc \__bnvs_gset_IPKV:NNw
711       { \__bnvs_g_IP:w { #3 } { ##1 } }
712       { \__bnvs_gset@#3!##1:nn } { #3 } { ##1 }
713     }
714     #1
715   }
716   { #4 }
717 }
718 \BNVS_new:cpn { gset:nnnn } #1 #2 #3 #4 {
719   \cs_if_exist_use:cF { \__bnvs_gset@#1!#2:nn } {
720     \exp_args:Ncc \__bnvs_gset:NNnnnn
721     { \__bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
722   } { #3 } { #4 }
723 }
724 \BNVS_new:cpn { gset:vnnn } {
725   \BNVS_tl_use:cv { gset:nnnn }
726 }
727 \BNVS_new:cpn { gset:nvnn } #1 {
728   \BNVS_tl_use:nv { \__bnvs_gset:nnnn { #1 } }
729 }
730 \BNVS_new:cpn { gset:vvnn } {
731   \BNVS_tl_use:Nvv \__bnvs_gset:nnnn
732 }

```



```

733 \BNVS_new:cpn { gset:nnnv } #1 #2 #3 {
734   \BNVS_tl_use:nv {
735     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
736   }
737 }

738 \BNVS_new:cpn { gset:vvnv } #1 #2 #3 {
739   \BNVS_tl_use:nv {
740     \__bnvs_gset:vvnn { #1 } { #2 } { #3 }
741   }
742 }

743 \BNVS_new:cpn { gset:nvnv } {
744   \BNVS_tl_use:Nv \__bnvs_gset:nnnv
745 }

746 \cs_generate_variant:Nn \__bnvs_gset:nnnn { nnne }

```

__bnvs_gset:nnv __bnvs_gset:nnv {<id>} {<path>} {<key>}

Syntactic sugar for __bnvs_gset:nnnv {<id>} {<path>} {<key>} {<key>}.

```

747 \BNVS_new:cpn { gset:nnv } #1 #2 #3 {
748   \__bnvs_gset:nnnv { #1 } { #2 } { #3 } { #3 }
749 }

```

__bnvs_gunset:nnn __bnvs_gunset:nnv {<id>} {<path>} {<key>}
__bnvs_gunset:(nvn|vvnn)

Removes the specifications for the <id>!<path>/<key> combination.

```

750 \seq_new:N \l__bnvs_I_seq
751 \BNVS_new:cpn { seq_gremove_once:Nn } #1 #2 {
752   \seq_clear:N \l__bnvs_I_seq
753   \cs_set:Npn \BNVS_seq_gremove_Nn:n ##1 {
754     \tl_if_eq:nnTF { ##1 } { #2 } {
755       \cs_set:Npn \BNVS_seq_gremove_Nn:n ####1 {
756         \seq_put_right:Nn \l__bnvs_I_seq { ####1 }
757       }
758     } {
759       \seq_put_right:Nn \l__bnvs_I_seq { ##1 }
760     }
761   }
762   \seq_map_function:NN #1 \BNVS_seq_gremove_Nn:n
763   \seq_gset_eq:NN #1 \l__bnvs_I_seq
764 }

765 \BNVS_new:cpn { gunset_IP:Nw } #1 #2 #3 {
766   \__bnvs_seq_gremove_once:Nn #1 { #3 }
767   \seq_if_empty:NT #1 {
768     \__bnvs_seq_gremove_once:Nn \g__bnvs_I_seq { #2 }
769     \cs_undefine:c { __bnvs_gset@#2:nnn}
770   }
771 }

```

```

772 \BNVS_new:cpn { gunshot:NNnnn } #1 #2 #3 #4 #5 {
773   \tl_if_exist:NT #1 {
774     \cs_undefine:N #1
775     \__bnvs_seq_gremove_once:Nn #2 { #5 }
776     \seq_if_empty:NT #2 {
777       \exp_args:Nc \__bnvs_gunset_IP:Nw {
778         \__bnvs_g_I:w { #3 }
779       } { #3 } { #4 }
780       \cs_undefine:c { __bnvs_gset@#3!#4:nn}
781     }
782   }
783 }

784 \BNVS_new:cpn { gunshot:nnn } #1 #2 #3 {
785   \exp_args:Ncc
786   \__bnvs_gunset:NNnnn
787   { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
788   { \__bnvs_g_IP:w { #1 } { #2 } }
789   { #1 } { #2 } { #3 }
790 }

791 \BNVS_new:cpn { gunshot:nvn } #1 {
792   \BNVS_tl_use:nv { \__bnvs_gunset:nnn { #1 } }
793 }

794 \BNVS_new:cpn { gunshot:vvv } {
795   \BNVS_tl_use:Nvv \__bnvs_gunset:nnn
796 }

```

```

\__bnvs_is_gset:nnnTF      \__bnvs_is_gset:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}
\__bnvs_is_gset:(nvn|vvv|vxn)TF

```

Test for the existence of some data for that $\langle id \rangle ! \langle path \rangle / \langle key \rangle$ combination.

```

797 \BNVS_new_conditional:cpnn { is_gset:nnn } #1 #2 #3 { T, F, TF } {
798   \tl_if_exist:cTF { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
799   \prg_return_true: \prg_return_false:
800 }

801 \BNVS_new_conditional:cpnn { is_gset:vxn } #1 #2 #3 { T, F, TF } {
802   \exp_args:NNnx \BNVS_tl_use:Nv
803   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
804   \prg_return_true: \prg_return_false:
805 }

806 \BNVS_new_conditional:cpnn { is_gset:nvn } #1 #2 #3 { T, F, TF } {
807   \BNVS_tl_use:nv { \__bnvs_is_gset:nnnTF { #1 } } { #2 } { #3 }
808   \prg_return_true: \prg_return_false:
809 }

810 \BNVS_new_conditional:cpnn { is_gset:vvv } #1 #2 #3 { T, F, TF } {
811   \BNVS_tl_use:Nvv \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
812   \prg_return_true: \prg_return_false:
813 }

```

1.7.4 Loops

<code>_bnvs_foreach_I:N</code>	<code>_bnvs_foreach_I:N <function:n></code>
<code>_bnvs_foreach_I:n</code>	<code>_bnvs_foreach_I:n {<n code>}</code>
<code>_bnvs_foreach_I_break:</code>	<code>_bnvs_foreach_I_break:</code>
<code>_bnvs_foreach_I_break:n</code>	<code>_bnvs_foreach_I_break:n {<code>}</code>

Execute the `<function:n>` or the `<n code>` for each declared identifier. The break commands work like `\seq_map_break:` and

```

814 \BNVS_new:cpn { foreach_I:N } {
815   \seq_map_function:NN \g__bnvs_I_seq
816 }

817 \BNVS_new:cpn { foreach_I:n } {
818   \seq_map_inline:Nn \g__bnvs_I_seq
819 }

820 \cs_set_eq:NN \_bnvs_foreach_I_break: \seq_map_break:
821 \cs_set_eq:NN \_bnvs_foreach_I_break:n \seq_map_break:n

```

<code>_bnvs_foreach_P:nNTF</code>	<code>_bnvs_foreach_P:nNTF {<id>} <function:nn> {<yes:>} {<no:>}</code>
<code>_bnvs_foreach_P:nnTF</code>	<code>_bnvs_foreach_P:nnTF {<id>} {<code:nn>} {<yes:>} {<no:>}</code>

If `<id>` is a declared identifier, execute `<function:nn>` or `<code:nn>` for each combination of `<id>` and its associate `<path>`s. Both are the arguments passed to `<function:nn>` or `<nn code>` body (through `##1` and `##2`).

```

822 \BNVS_new_conditional:cpnn { foreach_P:nN } #1 #2 { T, F, TF } {
823   \seq_if_exist:CTF { \_bnvs_g_I:w { #1 } } {
824     \seq_map_inline:cn {
825       \_bnvs_g_I:w { #1 }
826     } {
827       #2 { #1 } { ##1 }
828     }
829   \prg_return_true:
830 } {
831   \prg_return_false:
832 }
833 }

834 \BNVS_new_conditional:cpnn { foreach_P:nn } #1 #2 { T, F, TF } {
835   \seq_if_exist:CTF { \_bnvs_g_I:w { #1 } } {
836     \cs_set:Npn \BNVS_foreach_P_nn:nn ##1 ##2 { #2 }
837     \seq_map_inline:cn { \_bnvs_g_I:w { #1 } } {
838       \BNVS_foreach_P_nn:nn { #1 } { ##1 }
839     }
840   \prg_return_true:
841 } {
842   \prg_return_false:
843 }
844 }

```

__bnvs_foreach_K:nnN	__bnvs_foreach_K:nnN {<id> } {<path> } <function:nnn>
__bnvs_foreach_K:nnn	__bnvs_foreach_K:nnn {<id> } {<path> } {<code:nnn> }

Execute the <function:nnn> or the <code:nnn> body for each of <id>!<tag>/<key> combination. These are the arguments to the function or code body.

```

845 \BNVS_new:cpn { foreach_K:NnnN } #1 #2 #3 #4 {
846   \seq_if_exist:NT #1 {
847     \seq_map_inline:Nn #1 {
848       #4 { #2 } { #3 } { ##1 }
849     }
850   }
851 }

852 \BNVS_new:cpn { foreach_K:nnN } #1 #2 #3 {
853   \exp_args:Nc \__bnvs_foreach_K:NnnN
854   { \__bnvs_g_IP:w { #1 } { #2 } }
855   { #1 } { #2 } #3
856 }

857 \BNVS_new:cpn { foreach_K:nnn } #1 #2 #3 {
858   \cs_set:Npn \BNVS_foreach_K_nnn:w ##1 ##2 ##3 { #3 }
859   \__bnvs_foreach_K:nnN { #1 } { #2 } \BNVS_foreach_K_nnn:w
860 }

```

__bnvs_foreach_R:nnNTF	__bnvs_foreach_R:nnNTF {<id> } {<tag> } <function:nn> {<yes:> } {<no:> }
__bnvs_foreach_R:nnnTF	__bnvs_foreach_R:nnnTF {<id> } {<tag> } {<code:nn> } {<yes:> } {<no:> }

If the <id> is known, execute the <function:nn> or the <code:nn> body for each of <id>!<tag>.<subtag> combination. The <id> and <tag>.<subtag> are the arguments of the function and the function body. After that the <yes:> code is executed if some combination is found, otherwise it executes <no:> code.

```

861 \BNVS_new_conditional:cpnn { foreach_R:nnn } #1 #2 #3 { T, F, TF } {
862   \cs_set:Npn \BNVS_foreach_R_nnnTF:nn ##1 ##2 {
863     #3
864   }
865   \__bnvs_foreach_R:nnNTF { #1 } { #2 } \BNVS_foreach_R_nnnTF:nn
866   \prg_return_true: \prg_return_false:
867 }

868 \BNVS_new_conditional:cpnn { foreach_R:nnN } #1 #2 #3 { T, F, TF } {
869   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {

```

This is not reentrant.

```

870   \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
871     \cs_set:Npn \BNVS_foreach_R_nnnTF_return: { \prg_return_true: }
872     \str_if_in:nnTF { ~ ##1 . } { ~ #2 . } {
873       #3 { #1 } { ##1 }
874     } {
875     }
876   }
877   \BNVS_foreach_R_nnnTF_return:
878 } {
879   \prg_return_false:
880 }
881 }

```

```

__bnvs_foreach_IP:N  __bnvs_foreach_IP:N <function:nn>
__bnvs_foreach_IP:n  __bnvs_foreach_IP:n {<code:nn>}

```

Execute the <function:nn> or the <code:nn> for each <id>!<path> combination.

```

882 \BNVS_new:cpn { foreach_IP:N } #1 {
883   __bnvs_foreach_I:n {
884     __bnvs_foreach_P:nNT { ##1 } #1 { }
885   }
886 }

887 \BNVS_new:cpn { foreach_IP:NT } #1 #2 {
888   __bnvs_foreach_I:n {
889     __bnvs_foreach_P:nNT { ##1 } #1 { #2 }
890   }
891 }

892 \BNVS_new:cpn { foreach_IP:n } #1 {
893   \cs_set:Npn \BNVS_foreach_IP_n:nn ##1 ##2 { #1 }
894   __bnvs_foreach_I:n {
895     __bnvs_foreach_P:nNT { ##1 } \BNVS_foreach_IP_n:nn { }
896   }
897 }

```

1.7.5 Path function

```

__bnvs_walk_path:nnnn  __bnvs_walk_path:nnnn {<path>} {<relative>} {<first:n>} {<other:n>}

```

UNUSED <path> is a dot separated list of components.

```

898 \BNVS_seq_new:c { walk }
899 \BNVS_set:cpn { walk_path:nnnn } #1 #2 #3 #4 {
900   \BNVS_set:cpn { walk_path_ii:n } ##1 { #2 }
901   \BNVS_set:cpn { walk_path_iii:n } ##1 { #3 }
902   \BNVS_set:cpn { walk_path_iiii:n } ##1 { #4 }
903   \BNVS_seq_use:Nc \seq_set_split:Nnn { walk } . { #1 }
904   \BNVS_set:cpn { walk_path_next: } { }
905   \cs_set:Npn \KWN:n ##1 {
906     \BNVS_set:cpn { walk_path_next: } {
907       __bnvs_walk_path_ii:n { ##1 }
908     }
909     \cs_set:Npn \KWN:n #####1 {
910       __bnvs_walk_path_iii:n { ##1 }
911       \BNVS_set:cpn { walk_path_next: } { }
912       \cs_set:Npn \KWN:n #####1 {
913         __bnvs_walk_path_iiii:n { #####1 }
914       }
915       __bnvs_walk_path_iiii:n { #####1 }
916     }
917   }
918   \BNVS_seq_use:Nc \seq_map_function:NN { walk } \KWN:n
919   __bnvs_walk_path_next:
920 }

```

```

__bnvs_foreach_relative:nn  __bnvs_foreach_relative:nn {<relative>} {<code:n>}

```

`<relative>` is a dot separated list of components starting with a dot.

```

921 \BNVS_set:cpn { foreach_relative:nn } #1 #2 {
922   \BNVS_set:cpn { foreach_relative:n } ##1 { #2 }
923   \BNVS_seq_use:Nc \seq_set_split:Nnn { walk } . { #1 }
924   \cs_set:Npn \KWN:n ##1 {
925     \cs_set:Npn \KWN:n { __bnvs_foreach_relative:n }
926   }
927   \BNVS_seq_use:Nc \seq_map_function:NN { walk } \KWN:n
928 }

```

1.7.6 Low level IP model functions

1.7.7 Low level I model functions

1.7.8 Getting

```

__bnvs_if_get:nnnTF  __bnvs_if_get:nnnTF {<id>} {<path>} {<key>} {<yes:n>} {<no:>}
__bnvs_if_spec:nnnTF __bnvs_if_spec:nnnTF {<id>} {<path>} {<key>} {<yes:n>} {<no:>}

```

The `__bnvs_if_get:nnnTF` variant calls `<yes:n>` with what was previously stored with key `<id>!``<path>/<key>`. Otherwise executes `<no:>` without changing the contents of the `<ans>` `tl` variable.

The `_spec:` variant is similar except that it uses `<id>!``<path>/<key>` combinations when available or `!``<path>/<key>` otherwise.

```

929 \BNVS_new:cpn { if_get:nnnTF } #1 #2 #3 #4 #5 {
930   __bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
931     \cs_set:Npn \KWN:n ##1 { #4 }
932     \exp_args:Nv \KWN:n { __bnvs_g_IPK:w { #1 } { #2 } { #3 } }
933   } {
934     #5
935   }
936 }

937 \BNVS_new:cpn { if_spec:nnnTF } #1 #2 #3 #4 #5 {
938   __bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
939     \cs_set:Npn \KWN:n ##1 { #4 }
940     \exp_args:Nv \KWN:n { __bnvs_g_IPK:w { #1 } { #2 } { #3 } }
941   } {
942     \tl_if_empty:nTF { #1 } {
943       #5
944     } {
945       __bnvs_is_gset:nnnTF {} { #2 } { #3 } {
946         \cs_set:Npn \KWN:n ##1 { #4 }
947         \exp_args:Nv \KWN:n { __bnvs_g_IPK:w {} { #2 } { #3 } }
948       } {
949         #5
950       }
951     }
952   }
953 }

```

<code>_bnvs_if_get:nnncTF</code> <code>_bnvs_if_spec:nnncTF</code> <code>_bnvs_if_get:nncTF</code> <code>_bnvs_if_get:vvcTF</code>	<code>_bnvs_if_get:nnncTF {<id> } {<path> } {<key> } {<ans> } {<yes:> } {<no:> }</code> <code>_bnvs_if_spec:nnncTF {<id> } {<path> } {<key> } {<ans> } {<yes:> } {<no:> }</code> <code>_bnvs_if_get:nncTF {<id> } {<path> } {<ans> } {<yes:> } {<no:> }</code>
---	--

The `\\_bnvs_if_get:nnnc...` variant puts what was stored for `<id> ! <path> / <key>` into the `<ans>` variable, if any, then executes `<yes:>`. Otherwise executes `<no:>` without changing the contents of the `<ans>` t1 variable.

`\\_bnvs_if_get:nncTF` is a convenient shortcut for `\\_bnvs_if_get:nnncTF` when `<key>` and `<ans>` happen to be the same.

The `_spec:` variant is similar except that it uses `<id> ! <path> / <key>` combinations when available or `! <path> / <key>` otherwise.

```

954 \BNVS_new_conditional:cpnn { if_get:nnnc } #1 #2 #3 #4 { T, F, TF } {
955   \\_bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
956     \BNVS_t1_use:Nc \cs_set_eq:Nc
957     { #4 } { \\_bnvs_g_IPK:w { #1 } { #2 } { #3 } }
958     \prg_return_true:
959   } {
960     \prg_return_false:
961   }
962 }
963 \BNVS_undefine:c { if_get:nnnc }

964 \BNVS_new_conditional:cpnn { if_spec:nnnc } #1 #2 #3 #4 { T, F, TF } {
965   \\_bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
966     \t1_set_eq:cc { \BNVS_l:cn { #4 } { t1 } } {
967       \\_bnvs_g_IPK:w { #1 } { #2 } { #3 }
968     }
969     \prg_return_true:
970   } {
971     \t1_if_empty:nTF { #1 } {
972       \prg_return_false:
973     } {
974       \\_bnvs_if_spec:nnncTF {} { #2 } { #3 } { #4 }
975       \prg_return_true: \prg_return_false:
976     }
977   }
978 }
979 \BNVS_undefine:c { if_spec:nnnc }

980 \BNVS_new_conditional:cpnn { if_get:nnc } #1 #2 #3 { T, F, TF } {
981   \\_bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #3 }
982   \prg_return_true: \prg_return_false:
983 }

984 \BNVS_new_conditional:cpnn { if_get:vvc } #1 #2 #3 { T, F, TF } {
985   \BNVS_t1_use:Nvv \\_bnvs_if_get:nncTF { #1 } { #2 } { #3 }
986   \prg_return_true: \prg_return_false:
987 }

```

<code>_bnvs_if_resolved:nncTF</code>	<code>_bnvs_if_resolved:nnncTF {<id> } {<path> } {<ans> } {<yes:> } {<no:> }</code>
--	---

Execute `<yes:>` if resolution succeeded for either `<id> ! <path> / r` or the fallback `! <path> / r`, otherwise execute `<no:>`.

```

988 \BNVS_new_conditional:cpnn { if_resolved:nnc } #1 #2 #3 { T, F, TF } {

```

```

989 \__bnvs_if_get:nnncTF { #1 } { #2 } r { #3 } {
990   \prg_return_true:
991   \prg_return_false:
992 } {
993   \tl_if_empty:nTF { #1 } {
994     \prg_return_false:
995   } {
996     \__bnvs_if_resolved:nnncTF { } { #2 } r { #3 }
997     \prg_return_true:
998     \prg_return_false:
999   }
1000 }
1001 }
1002 \BNVS_undefine:c { if_resolved:nnc }

```

__bnvs_if_resolved:nnncTF __bnvs_if_resolved:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}

Execute <yes:> if resolution succeeded for either <id>!<path>/<key> or the fallback !<path>/<key>, otherwise execute <no:>.

```

1003 \BNVS_new_conditional:cpnn { if_resolved:nnnc } #1 #2 #3 #4 { T, F, TF } {
1004   \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #4 } {
1005     \__bnvs_is_will_gset:cTF { #3 }
1006     \prg_return_true: \prg_return_false:
1007   } {
1008     \tl_if_empty:nTF { #1 } {
1009       \prg_return_false:
1010     } {
1011       \__bnvs_if_resolved:nnncTF { #1 } { #2 } { #3 } { #4 }
1012       \prg_return_true: \prg_return_false:
1013     }
1014   }
1015 }

```

__bnvs_if_resolved_IP:wTF __bnvs_if_resolved_IP:wTF {<id>} {<path>} {<yes:n>} {<no:>}

If <id>!<path>/r has been set, execute <yes:n> code with that value as argument, otherwise execute <no:> code.

```

1016 \BNVS_tl_new:c { if_resolved_IP:wTF }
1017 \BNVS_new:cpn { if_resolved_IP:wTF } #1 #2 {
1018   \__bnvs_if_get:nnncTF { #1 } { #2 } r { if_resolved_IP:wTF } {
1019     \BNVS_tl_use:Nv \BNVS_apply_i:nnn { if_resolved_IP:wTF }
1020   } {
1021     \use_ii:nn
1022   }
1023 }

```

__bnvs_require:nnnc __bnvs_require:nnnc {<id>} {<path>} {<key>} {<ans>}
__bnvs_require:nnc __bnvs_require:nnc {<id>} {<path>} {<ans>}
__bnvs_require:vvc

Calls __bnvs_if_get:nnncTF, does nothing on success and, on failure, does nothing or raise when in debug mode. __bnvs_require:nnc is a shortcut for the more qualified function __bnvs_require:nnnc when <key> and <ans> happen to be identical.


```

1024 \BNVS_new:cpn { require:nnnc } #1 #2 #3 #4 {
1025   \__bnvs_if_get:nnncF { #1 } { #2 } { #3 } { #4 } {
1026     \BNVS_fatal_unreachable:
1027   }
1028 }

```

<code>__bnvs_gunset:nn</code> <code>__bnvs_gunset:(nv vn vv)</code>	<code>__bnvs_gunset:nn {<id>} {<path>}</code>
--	--

Removes the specifications for the $\langle id \rangle ! \langle path \rangle$ and all possible keys combination.

```

1029 \BNVS_new:cpn { gunset:NNNnn } #1 #2 #3 #4 #5 {
1030   \seq_map_inline:Nn #1 {
1031     \__bnvs_gunset:nnn { #4 } { #5 } { ##1 }
1032   }
1033   \cs_undefine:N #1
1034   \__bnvs_seq_gremove_once:Nn #2 { #5 }
1035   \seq_if_empty:NT #2 {
1036     \cs_undefine:N #2
1037     \__bnvs_seq_gremove_once:Nn #3 { #4 }
1038     \cs_undefine:c { __bnvs_gset@ #4 :nnn }
1039   }
1040 }
1041 \BNVS_new:cpn { gunset:Nnn } #1 #2 #3 {
1042   \seq_map_inline:Nn #1 {
1043     \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
1044       \exp_args:Nc \__bnvs_gunset:NNNnn
1045         { \__bnvs_g_IP:w { #2 } { ##1 } }
1046         #1 \g__bnvs_I_seq { #2 } { ##1 }
1047       \cs_undefine:c { __bnvs_gset@{ #2 } { ##1 }:nn }
1048     }
1049   }
1050 }
1051 \BNVS_new:cpn { gunset:nn } #1 #2 {
1052   \exp_args:Nc
1053   \__bnvs_gunset:Nnn { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
1054 }
1055 \BNVS_new:cpn { gunset:nv } #1 {
1056   \BNVS_tl_use:nv { \__bnvs_gunset:nn { #1 } }
1057 }
1058 \BNVS_new:cpn { gunset:vn } {
1059   \BNVS_tl_use:cv { gunset:nn }
1060 }
1061 \BNVS_new:cpn { gunset:vv } {
1062   \BNVS_tl_use:Nvv \__bnvs_gunset:nn
1063 }

```

<code>__bnvs_gunset_deep:nn</code> <code>__bnvs_gunset_deep:(nv vv)</code>	<code>__bnvs_gunset_deep:nn {<id>} {<path>}</code>
---	---

Removes the specifications for each existing $\langle id \rangle ! \langle path \rangle . \langle subpath \rangle$ combination.

```

1064 \BNVS_new:cpn { gunset_deep:Nnn } #1 #2 #3 {
1065   \seq_if_exist:NT #1 {
1066     \seq_map_inline:Nn #1 {

```

#3 is exactly ##1 or start with ##1., which is equivalent to #3. starts with ##1.:

```

1067     \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
1068         \__bnvs_gunset:nn { #2 } { ##1 }
1069     }
1070 }
1071 }
1072 }

1073 \BNVS_new:cpn { gunset_deep:nn } #1 #2 {
1074     \exp_args:Nc \__bnvs_gunset_deep:Nnn
1075     { \__bnvs_g_I:w { #1 } }
1076     { #1 } { #2 }
1077 }

1078 \BNVS_new:cpn { gunset_deep:nv } #1 {
1079     \BNVS_tl_use:nv { \__bnvs_gunset_deep:nn { #1 } }
1080 }

1081 \BNVS_new:cpn { gunset_deep:vv } {
1082     \BNVS_tl_use:Nvv \__bnvs_gunset_deep:nn
1083 }

```

`\__bnvs_gunset:n` `\__bnvs_gunset:n {⟨id⟩}`
`\__bnvs_gunset:` `\__bnvs_gunset:`

Removes the specifications for the `⟨id⟩` and all possible tag combination. The second does the same for all possible `⟨id⟩`.

```

1084 \BNVS_new:cpn { gunset:NNNn } #1 #2 #3 #4 {
1085     \cs_if_exist:NT #1 {
1086         \cs_undefine:N #1
1087         \seq_map_inline:Nn #2 {
1088             \__bnvs_gunset:nn { #4 } { ##1 }
1089         }
1090         \cs_undefine:N #2
1091         \__bnvs_seq_gremove_once:Nn #3 { #4 }
1092     }
1093 }
1094 \BNVS_new:cpn { gunset:n } #1 {
1095     \exp_args:Ncc \__bnvs_gunset:NNNn
1096     { \__bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } }
1097     \g__bnvs_I_seq { #1 }
1098 }
1099 \BNVS_new:cpn { gunset: } {
1100     \seq_map_inline:Nn \g__bnvs_I_seq {
1101         \__bnvs_gunset:n { ##1 }
1102     }
1103 }

```

`\__bnvs_VS_gunset:` `\__bnvs_VS_gunset:`

Removes any value for the “V” and “S” keys and any `⟨id⟩ ! ⟨path⟩` combination.

```

1104 \BNVS_new:cpn { VS_gunset: } {
1105     \__bnvs_foreach_IP:n {

```

```

1106     \_bnvs_gunset:nnn { ##1 } { ##2 } V
1107     \_bnvs_gunset:nnn { ##1 } { ##2 } S
1108   }
1109 }

```

1.7.9 Circular dependencies

```

\_bnvs_will_gset:nnn \_bnvs_will_gset:nnn {<id>} {<path>} {<key>}

```

To manage circular dependencies. After this command, `\_bnvs_is_will_gset:nnnTF` is true for the same arguments.

```

1110 \BNVS_new:cpn { will_gset:nnn } #1 #2 #3 {
1111   \_bnvs_gset:nnnn { #1 } { #2 } { #3 } { \q_no_value }
1112 }

```

```

\_bnvs_is_will_gset:cTF \_bnvs_is_will_gset:cTF {<in>} {<yes:>} {<no:>}

```

To manage circular dependencies. See `\_bnvs_will_gset:nnn` above.

```

1 \_bnvs_will_gset:nnn {<id>} {<path>} {<key>}
2 ...
3 \_bnvs_if_get:nnncT {<id>} {<path>} {<key>} {<in>} {
4   ...
5   \_bnvs_is_will_gset:cTF
6     {<in>}
7     {<yes:>}
8     {<no:>}
9   ...
10 }

```

```

1113 \BNVS_new_conditional:cpnn { is_will_gset:c } #1 { T, F, TF } {
1114   \BNVS_tl_use:Nv \quark_if_no_value:nTF { #1 } {
1115     \prg_return_true:
1116   } {
1117     \prg_return_false:
1118   }
1119 }

```

1.7.10 Spring cleaning

```

\_bnvs_gclear:nn \_bnvs_gclear:nn {<id>} {<path>}
\_bnvs_gclear:nv \_bnvs_gclear:n {<id>}
\_bnvs_gclear: \_bnvs_gclear:
\_bnvs_greset:nn \_bnvs_greset:nn {<id>} {<path>}
\_bnvs_greset:nv \_bnvs_greset:n {<id>}
\_bnvs_greset: \_bnvs_greset:

```

The first ones clear out every data stored for `<id> ! <path>`, including deeper and registration data. When `<path>` is omitted, any known path is considered. When `<id>` is omitted, any known id is considered. The second one only clears out the data created on the fly after the initialization step. The initial data (for key =) is left untouched.

```

1120 \BNVS_new:cpn { greset:nn } #1 #2 {
1121   \__bnvs_foreach_P:nnT { #1 } {
1122     \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1123       \__bnvs_foreach_K:nnn { #1 } { ##2 } {
1124         \tl_if_in:nnF { \q_bnvs ####3 } { \q_bnvs = } {
1125           \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1126         }
1127       }
1128     }
1129   } { }
1130 }

1131 \BNVS_new:cpn { greset:nv } #1 {
1132   \BNVS_tl_use:nv { \__bnvs_greset:nn { #1 } }
1133 }

```

Clears out every data associated to $\langle id \rangle ! \langle path \rangle$, and any $\langle id \rangle ! \langle path \rangle . \langle subpath \rangle$.

```

1134 \BNVS_new:cpn { gclear:nn } #1 #2 {
1135   \__bnvs_foreach_P:nnT { #1 } {
1136     \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1137       \__bnvs_foreach_K:nnn { #1 } { ##2 } {
1138         \tl_if_in:nnTF { \q_bnvs ####3 } { \q_bnvs = } {
1139           \__bnvs_gset:nnnn { #1 } { ##2 } { ####3 } { 0 }
1140         } {
1141           \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1142         }
1143       }
1144     }
1145   } { }
1146 }

1147 \BNVS_new:cpn { gclear:n } #1 {
1148   \__bnvs_foreach_P:nNT { #1 } \__bnvs_gclear:nn {}
1149 }

1150 \BNVS_new:cpn { gclear: } {
1151   \__bnvs_foreach_IP:N \__bnvs_gclear:nn
1152 }

1153 \BNVS_new:cpn { greset:n } #1 {
1154   \__bnvs_foreach_P:nNT { #1 } \__bnvs_greset:nn {}
1155 }

1156 \BNVS_new:cpn { greset: } {
1157   \__bnvs_foreach_IP:N \__bnvs_greset:nn
1158 }

```

```

\__bnvs_gclear_resolved:nn \__bnvs_gclear_resolved:nn {<id>} {<path>}

```

```

\__bnvs_gclear_resolved:nv

```

Clear the resolved data, aka any key that does not start with “=”.

```

1159 \quark_new:N \q__bnvs_X

```

```

1160 \BNVS_new:cpn { gclear_resolved:nn } #1 #2 {
1161   \__bnvs_foreach_IP:nnn { #1 } { #2 } {
1162     \tl_if_in:nnF { \q__bnvs_X ##1 } { \q__bnvs_X = } {
1163       \__bnvs_gunset:nnn { #1 } { #2 } { ##1 }
1164     }
1165   }
1166 }

1167 \BNVS_new:cpn { gclear_resolved:nv } #1 {
1168   \BNVS_tl_use:nv {
1169     \__bnvs_gclear_resolved:nn { #1 }
1170   }
1171 }

```

1.7.11 The provide mode

\l__bnvs_provide_bool Manage the provide mode.

```

1172 \bool_new:N \l__bnvs_provide_bool

```

(End of definition for \l__bnvs_provide_bool.)

```

1173 \BNVS_new:cpn { provide_ON: } {
1174   \__bnvs_set_true:c { provide }
1175 }

1176 \BNVS_new:cpn { provide_OFF: } {
1177   \__bnvs_set_false:c { provide }
1178 }
1179 \__bnvs_provide_OFF:

```

__bnvs_is_gset_and_provide:nnnTF	__bnvs_is_gset_and_provide:nnnTF {<id>} {<path>} {<key>} {<yes:>}
__bnvs_is_gset_and_provide:(nvn vvn)TF	{<no:>}
__bnvs_is_gset_and_provide:nnTF	__bnvs_is_gset_and_provide:nnTF {<id>} {<path>} {<yes:>} {<no:>}

Execute <yes:> when in provide mode and <id>!<path>/<key> is known, <no:> otherwise. When not specified, <key> is “=”.

```

1180 \BNVS_new_conditional:cpnn { is_gset_and_provide:nnn }
1181   #1 #2 #3 { T, F, TF } {
1182     \__bnvs_if:cTF { provide } {
1183       \if_cs_exist:w \__bnvs_g_IPK:w { #1 } { #2 } { #3 } \cs_end:
1184       \prg_return_true:
1185     }
1186     \else:
1187       \prg_return_false:
1188     \fi:
1189   } {
1190     \prg_return_false:
1191   }

```

```

1192 \BNVS_new_conditional:cpnn { is_gset_and_provide:nvn }
1193         #1 #2 #3 { T, F, TF } {
1194     \BNVS_tl_use:nv {
1195         \__bnvs_is_gset_and_provide:nnnTF { #1 }
1196     } { #2 } { #3 }
1197     \prg_return_true: \prg_return_false:
1198 }

1199 \BNVS_new_conditional:cpnn { is_gset_and_provide:vvv }
1200         #1 #2 #3 { T, F, TF } {
1201     \BNVS_tl_use:Nvv \__bnvs_is_gset_and_provide:nnnTF { #1 } { #2 } { #3 }
1202     \prg_return_true: \prg_return_false:
1203 }

```

```

\__bnvs_gprovide:TnnnnF \__bnvs_gprovide:TnnnnF {<pre code>} {<id>} {<path>} {<key>} {<value>} {<no:>}}

```

```

\__bnvs_gprovide:TvnnnF Execute <no:> exclusively when not in provide mode. Does nothing when something was
\__bnvs_gprovide:TvvnnF set for the <id>!<path>/<key> combination. Execute <pre code> before setting.

```

```

1204 \BNVS_new:cpn { gprovide:TnnnnF } #1 #2 #3 #4 #5 {
1205     \__bnvs_if:cTF { provide } {
1206         \__bnvs_is_gset:nnnF { #2 } { #3 } { #4 } {
1207             #1
1208             \__bnvs_gset:nnnn { #2 } { #3 } { #4 } { #5 }
1209         }
1210     }
1211 }

1212 \BNVS_new:cpn { gprovide:TvnnnF } #1 {
1213     \BNVS_tl_use:nv { \__bnvs_gprovide:TnnnnF { #1 } }
1214 }
1215 \BNVS_new:cpn { gprovide:TvvnnF } #1 {
1216     \BNVS_tl_use:nvv { \__bnvs_gprovide:TnnnnF { #1 } }
1217 }

```

```

\__bnvs_is_provided:nnnTF \__bnvs_is_provided:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}}

```

If <id>!<path>/<key> has been successfully resolved and not unset since then, execute <yes:>. Otherwise execute <no:>.

```

1218 \tl_new:N \l__bnvs_is_provided_tl
1219 \BNVS_new_conditional:cpnn { is_provided:nnn } #1 #2 #3 { T, F, TF } {
1220     \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { is_provided } {
1221         \__bnvs_is_will_gset:cTF { is_provided } {
1222             \prg_return_false:
1223         } {
1224             \prg_return_true:
1225         }
1226     } {
1227         \prg_return_false:
1228     }
1229 }

```

1.7.12 Initialize

`\__bnvs_ginit:nnnn` `\__bnvs_ginit:nnnn {⟨id⟩} {⟨short⟩} {⟨relative⟩} {⟨definition⟩}`

Called by `\__bnvs_gdef:nnnn`. This function supports provide mode by calling `\__bnvs_is_gset_and_provide:nnnTF`. Somehow a shortcut to `\__bnvs_gset:nnnn` with key “=”. `⟨relative⟩` is a list of components, if it is not empty, each intermediate path is modified or defined appropriately by calling `\__bnvs_greset:nn` among other things.

```

1230 \BNVS_new:cpn { ginit:nnnn } #1 #2 #3 #4 {
1231   \tl_if_empty:nTF { #3 } {
1232     \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
1233       \__bnvs_gunset_deep:nn { #1 } { #2 }
1234       \__bnvs_gset:nnnn { #1 } { #2 } = { #4 }
1235     }
1236   } {
1237     \BNVS_begin:
1238     \__bnvs_greset:nn { #1 } { #2 }
1239     \__bnvs_is_gset:nnnF { #1 } { #2 } = {
1240       \__bnvs_gset:nnnn { #1 } { #2 } = {}
1241     }
1242     \__bnvs_tl_set:cn P { #2 }
1243     \__bnvs_foreach_relative:nn { #3 } {
1244       \__bnvs_tl_put_right:cn P { . ##1 }
1245       \__bnvs_greset:nv { #1 } P
1246       \__bnvs_is_gset:nvnF { #1 } P = {

```

Each intermediate step is just defined empty when not already defined. The last step setting is overridden just after the loop.

```

1247       \__bnvs_gset:nvnn { #1 } P = {}
1248     }
1249   }
1250   \__bnvs_is_gset_and_provide:nvnF { #1 } P = {
1251     \__bnvs_gunset_deep:nv { #1 } P
1252     \__bnvs_gset:nvnn { #1 } P = { #4 }
1253   }
1254   \BNVS_end:
1255 }
1256 }
```

`\__bnvs_ginit:nnn` `\__bnvs_ginit:nnn {⟨id⟩} {⟨path⟩} {⟨simple definition⟩}`
`\__bnvs_ginit:(nnv|vvn)`

Called by `\__bnvs_gdef:nnn`. Somehow a shortcut to `\__bnvs_gset:nnnn` with key “=”. If `⟨path⟩` has more than one component, each intermediate path is modified or defined appropriately. This function supports provide mode by calling the branch function `\__bnvs_is_gset_and_provide:nnnTF`.

```

1257 \BNVS_new:cpn { ginit:nnn } #1 #2 {
1258   \__bnvs_ginit:nnnn { #1 } { #2 } {}
1259 }

1260 \BNVS_new:cpn { ginit:nnv } #1 #2 {
1261   \BNVS_tl_use:nv { \__bnvs_ginit:nnn { #1 } { #2 } }
1262 }
```

```

1263 \BNVS_new:cpn { ginit:vvv } {
1264   \BNVS_tl_use:Nvv \_bnvs_ginit:nnn
1265 }

```

_bnvs_if_should_ginit:nnTF _bnvs_if_should_ginit:nnTF {<id>} {<path>} {<yes:>} {<no:>}

Execute <yes:> when in provide mode and <id>!<tag> combination is not initialized by _bnvs_ginit:nnn. Execute <no:> otherwise.

```

1266 \BNVS_new_conditional:cpnn { if_should_ginit:nn } #1 #2 { T, F, TF } {
1267   \_bnvs_if:cTF { provide } {
1268     \_bnvs_is_ginit:nnTF { #1 } { #2 } {
1269       \prg_return_false:
1270     } {
1271       \prg_return_true:
1272     }
1273   } {
1274     \prg_return_true:
1275   }
1276 }
1277 \BNVS_undefine:c { if_should_ginit:nn }

```

_bnvs_is_ginit:nnTF _bnvs_is_ginit:nnTF {<id>} {<path>} {<yes:>} {<no:>}

Test for the existence of some data for that <id>!<path> combination.

```

1278 \BNVS_new_conditional:cpnn { is_ginit:nn } #1 #2 { T, F, TF } {
1279   \_bnvs_is_gset:nnnTF { #1 } { #2 } = \prg_return_true: \prg_return_false:
1280 }
1281 \BNVS_undefine:c { is_ginit:nn }

```

_bnvs_if_ginit:nncTF _bnvs_if_ginit:nncTF {<id>} {<tag>} {<var>} {<yes:>} {<no:>}

Test for the existence of some initialization data for that <id>!<path> combination. On success, <var> is set to this data.

```

1282 \BNVS_new_conditional:cpnn { if_ginit:nnc } #1 #2 #3 { T, F, TF } {
1283   \_bnvs_if_get:nnncTF { #1 } { #2 } = { #3 }
1284   \prg_return_true: \prg_return_false:
1285 }
1286 \BNVS_undefine:c { if_ginit:nnc }

```

1.8 Regular expressions

\c__bnvs_A_reserved_Z_regex Reserved words.

(End of definition for \c__bnvs_A_reserved_Z_regex.)

```

1287 \regex_const:Nn \c__bnvs_A_reserved_Z_regex {
1288   \A (? : _+ \d | _* [a-z] ) [_a-z0-9]* \Z
1289 }

```

\c__bnvs_A_VAZLLZZL_Z_regex Used to parse named overlay specifications. V, A:Z, A::L on one side, :Z, :Z::L and ::L:Z on the other sides. Next are the capture groups. The first one is for the whole match.

(End of definition for \c__bnvs_A_VAZLLZZL_Z_regex.)

```

1290 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
1291   \A \s* (?:

      • 2 → V

1292   ( [^:]+? )

      • 3, 4, 5 → A : Z? or A :: L?

1293   | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )

      • 6, 7 → ::(L:Z)?

1294   | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )

      • 8, 9 → :(Z::L)?

1295   | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
1296   )
1297   \s* \Z
1298 }

```

1.9 End group setters

```

1299 \cs_new:Npn \BNVS_did_end_tl_put_right:cv #1 {
1300   \BNVS_did_end:n {
1301     \__bnvs_tl_put_right:cn { #1 }
1302   }
1303   \BNVS_tl_use:Nv \BNVS_did_end_braced:n
1304 }

1305 \cs_new:Npn \BNVS_end_tl_put_right:cv #1 #2 {
1306   \BNVS_did_end_tl_put_right:cv { #1 } { #2 }
1307   \BNVS_end:
1308 }

1309 \cs_new:Npn \BNVS_did_end_tl_gset:nnnv #1 #2 #3 {
1310   \BNVS_did_end:n {
1311     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
1312   }
1313   \BNVS_tl_use:Nv \BNVS_did_end_braced:n
1314 }

1315 \cs_new:Npn \BNVS_end_tl_gset:nnnv #1 #2 #3 #4 {
1316   \BNVS_did_end_tl_gset:nnnv { #1 } { #2 } { #3 } { #4 }
1317   \BNVS_end:
1318 }

```

```

1319 \cs_new:Npn \BNVS_did_end_tl_set:cv #1 {
1320   \BNVS_did_end:n {
1321     \__bnvs_tl_set:cn { #1 }
1322   }
1323   \BNVS_tl_use:Nv \BNVS_did_end_braced:n
1324 }

1325 \cs_new:Npn \BNVS_end_tl_set:cv {
1326   \BNVS_tl_set:ncv { \BNVS_end: }
1327 }

1328 \cs_new:Npn \BNVS_tl_set:ncv #1 #2 {
1329   \BNVS_tl_use:nv {
1330     #1
1331     \__bnvs_tl_set:cn { #2 }
1332   }
1333 }

1334 \cs_new:Npn \BNVS_tl_set:ncvcv #1 #2 #3 #4 {
1335   \BNVS_tl_set:ncv {
1336     \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1337   } { #4 }
1338 }

1339 \cs_new:Npn \BNVS_tl_set:ncvcvcv #1 #2 #3 #4 #5 #6 {
1340   \BNVS_tl_set:ncv {
1341     \BNVS_tl_set:ncv {
1342       \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1343     } { #4 } { #5 }
1344   } { #6 }
1345 }

1346 \cs_new:Npn \BNVS_did_end_int_set:cv #1 {
1347   \BNVS_did_end:n {
1348     \__bnvs_int_set:cn { #1 }
1349   }
1350   \BNVS_tl_use:Nv \BNVS_did_end_braced:n
1351 }

1352 \cs_new:Npn \BNVS_end_int_set:cv #1 #2 {
1353   \BNVS_did_end_int_set:cv { #1 } { #2 }
1354   \BNVS_end:
1355 }

```

1.10 beamer.cls interface

Work in progress.

```

1356 \RequirePackage{keyval}

1357 \define@key{beamerframe}{beanoves-id}[]{}
1358   \tl_set:Nx \l__bnvs_J_tl { #1 }
1359 }

```

```

1360 \bool_new:N \l__bnvs_in_frame_bool
1361 \bool_set_false:N \l__bnvs_in_frame_bool
1362 \AddToHook{env/beamer@frameslide/before}{
1363   \__bnvs_greset_call:
1364   \__bnvs_VS_gunset:
1365   \__bnvs_set_true:c { in_frame }
1366   \__bnvs_tl_clear:c I
1367   \__bnvs_tl_clear:c { S() }
1368 }

1369 \AddToHook{env/beamer@frameslide/after}{
1370   \__bnvs_set_false:c { in_frame }
1371 }

```

1.11 Utilities

1.11.1 Split utilities

```

\__bnvs_if_regex_split:cncTF \__bnvs_if_regex_split:cncTF {<regex core>} {<expression>} <seq core> {<yes:>}
\__bnvs_if_regex_split:cnTF {<no:>}
\__bnvs_if_regex_split:cnTF \__bnvs_if_regex_split:cnTF {<regex core>} {<expression>} {<yes:>} {<no:>}

```

These are shortcuts to `\regex_split:NnNTF` with the `split` sequence as last N argument.

```

1372 \BNVS_new_conditional:cpnn { if_regex_split:cnc } #1 #2 #3 { T, F, TF } {
1373   \BNVS_seq_use:nc {
1374     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1375   } { #3 } \prg_return_true: \prg_return_false:
1376 }

1377 \BNVS_new_conditional:cpnn { if_regex_split:cn } #1 #2 { T, F, TF } {
1378   \BNVS_seq_use:nc {
1379     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1380   } { split } \prg_return_true: \prg_return_false:
1381 }

```

```

\__bnvs_split_pop: \__bnvs_split_pop:
\__bnvs_split_pop:c \__bnvs_split_pop:c {<var1>}
\__bnvs_split_pop:cc \__bnvs_split_pop:cc {<var1>} {<var2>}
\__bnvs_split_pop:ccc \__bnvs_split_pop:ccc {<var1>} {<var2>} {<var3>}
\__bnvs_split_pop:cccc \__bnvs_split_pop:cccc {<var1>} {<var2>} {<var3>} {<var4>}
\__bnvs_split_pop:ccccc \__bnvs_split_pop:ccccc {<var1>} {<var2>} {<var3>} {<var4>} {<var5>}

```

```

1382 \tl_new:N \l__bnvs_split_pop_tl
1383 \BNVS_new:cpn { split_pop: } {
1384   \seq_pop_left:NN \l__bnvs_split_seq \l__bnvs_split_pop_tl
1385 }

1386 \BNVS_new:cpn { split_pop:c } #1 {
1387   \BNVS_tl_use:nc {
1388     \seq_pop_left:NN \l__bnvs_split_seq
1389   } { #1 }
1390 }

```

```

1391 \BNVS_new:cpn { split_pop:cc } #1 {
1392   \__bnvs_split_pop:c { #1 }
1393   \__bnvs_split_pop:c
1394 }

1395 \BNVS_new:cpn { split_pop:ccc } #1 {
1396   \__bnvs_split_pop:c { #1 }
1397   \__bnvs_split_pop:cc
1398 }

1399 \BNVS_new:cpn { split_pop:cccc } #1 {
1400   \__bnvs_split_pop:c { #1 }
1401   \__bnvs_split_pop:ccc
1402 }

1403 \BNVS_new:cpn { split_pop:ccccc } #1 {
1404   \__bnvs_split_pop:c { #1 }
1405   \__bnvs_split_pop:cccc
1406 }

1407 \BNVS_new_conditional:cpnn { split_if_pop:c } #1 { T, F, TF } {
1408   \__bnvs_seq_pop_left:ccTF { split } { #1 } {
1409     \prg_return_true:
1410   } {
1411     \prg_return_false:
1412   }
1413 }

```

1.11.2 Match utilities

```

\__bnvs_match_if_once:NnTF \__bnvs_match_if_once:NnTF <regex variable> {<expression>} {<yes:>} {<no:>}
\__bnvs_match_if_once:NvTF \__bnvs_match_if_once:nnTF {<regex>} {<expression>} {<yes:>} {<no:>}
\__bnvs_match_if_once:nnTF

```

These are shortcuts to

- \regex_match_if_once:NnNTF with the match sequence as N argument
- \regex_match_if_once:nnNTF with the match sequence as N argument
- \regex_split:NnNTF with the split sequence as last N argument

```

1414 \BNVS_new_conditional:cpnn { if_extract_once:Ncn } #1 #2 #3 { T, F, TF } {
1415   \BNVS_use:ncn {
1416     \regex_extract_once:NnNTF #1 { #3 }
1417   } { #2 } { seq } \prg_return_true: \prg_return_false:
1418 }

```

\l__bnvs_match_seq Local storage for the match result.

(End of definition for \l__bnvs_match_seq.)

```

1419 \BNVS_new_conditional:cpnn { match_if_once:Nn } #1 #2 { T, F, TF } {
1420   \BNVS_use:ncn {
1421     \regex_extract_once:NnNTF #1 { #2 }
1422   } { match } { seq } \prg_return_true: \prg_return_false:
1423 }

```

```

1424 \BNVS_new_conditional:cpnn { if_extract_once:Ncv } #1 #2 #3 { T, F, TF } {
1425   \BNVS_seq_use:nc {
1426     \BNVS_tl_use:nv {
1427       \regex_extract_once:NnNTF #1
1428     } { #3 }
1429   } { #2 } \prg_return_true: \prg_return_false:
1430 }

1431 \BNVS_new_conditional:cpnn { match_if_once:Nv } #1 #2 { T, F, TF } {
1432   \BNVS_seq_use:nc {
1433     \BNVS_tl_use:nv {
1434       \regex_extract_once:NnNTF #1
1435     } { #2 }
1436   } { match } \prg_return_true: \prg_return_false:
1437 }

1438 \BNVS_new_conditional:cpnn { match_if_once:nn } #1 #2 { T, F, TF } {
1439   \BNVS_seq_use:nc {
1440     \regex_extract_once:nnNTF { #1 } { #2 }
1441   } { match } \prg_return_true: \prg_return_false:
1442 }

1443 \BNVS_new_conditional:cpnn { match_if_pop_left:c } #1 { T, F, TF } {
1444   \BNVS_tl_use:nc {
1445     \BNVS_seq_use:Nc \seq_pop_left:NNTF { match }
1446   } { #1 } {
1447     \prg_return_true:
1448   } {
1449     \prg_return_false:
1450   }
1451 }

```

__bnvs_match_pop:	__bnvs_match_pop:
__bnvs_match_pop:c	__bnvs_match_pop:c {<var ₁ >}
__bnvs_match_pop:cc	__bnvs_match_pop:cc {<var ₁ >} {<var ₂ >}
__bnvs_match_pop:ccc	__bnvs_match_pop:ccc {<var ₁ >} {<var ₂ >} {<var ₃ >}
__bnvs_match_pop:cccc	__bnvs_match_pop:cccc {<var ₁ >} {<var ₂ >} {<var ₃ >} {<var ₄ >}
__bnvs_match_pop:cccc	__bnvs_match_pop:cccc {<var ₁ >} {<var ₂ >} {<var ₃ >} {<var ₄ >} {<var ₅ >}

```

1452 \tl_new:N \l__bnvs_match_pop_tl
1453 \BNVS_new:cpn { match_pop: } {
1454   \seq_pop_left:NN \l__bnvs_match_seq \l__bnvs_match_pop_tl
1455 }

1456 \BNVS_new:cpn { match_pop:c } #1 {
1457   \BNVS_tl_use:nc {
1458     \seq_pop_left:NN \l__bnvs_match_seq
1459   } { #1 }
1460 }

1461 \BNVS_new:cpn { match_pop:cc } #1 {
1462   \__bnvs_match_pop:c { #1 }
1463   \__bnvs_match_pop:c
1464 }

```

```

1465 \BNVS_new:cpn { match_pop:ccc } #1 {
1466   \_bnvs_match_pop:c { #1 }
1467   \_bnvs_match_pop:cc
1468 }

1469 \BNVS_new:cpn { match_pop:cccc } #1 {
1470   \_bnvs_match_pop:c { #1 }
1471   \_bnvs_match_pop:ccc
1472 }

1473 \BNVS_new:cpn { match_pop:ccccc } #1 {
1474   \_bnvs_match_pop:c { #1 }
1475   \_bnvs_match_pop:cccc
1476 }

```

1.12 Scanner

There are different policies to store the intermediate results of a scanning operation. One possibility is to feed a global variable as scanning occurs. As you cannot use unbalanced braces in function arguments, we can use other characters than usual “\”, “{” and “}” before we rescan the final result with an appropriate catcode regime. Another possibility is to heavily use $\TeX$ groups to store intermediate results in local variables and only consider properly balanced material. At the end of the group, the scanned material must be forwarded to the group owner. The latter is used here.

The scanning process reads $\langle content \rangle$ from the $\langle input\ stream \rangle$ and store a corresponding *abstract syntax tree* (AST) denoted by $\langle content \rangle^*$. The AST will be executed either to define named overlay sets or to resolve named overlay queries. The workflow consists of

1. begin with `\BNVS_begin_scaN:`
2. define the AST exportation policy
3. define the scanning policy
4. define the branch functions named `\BNVS_scaNned_⟨branch⟩`, where $\langle branch \rangle$ is one of `Block:w`, `One_colon:w`, `Colons:w`, `comMa:w`, `cLose:w`, `Stop:`,
5. call `\BNVS_scaN_BOCMLS:w` or some `\BNVS_scaN_...:...w` function to peek characters from the $\langle input\ stream \rangle$ and store corresponding AST
6. end with `\BNVS_end_scaN:`, which exports the AST created according to the exportation policy.

We use the same scanner to parse definitions and queries. We do not use any key–value facility built into $\LaTeX$. The scanned material consists of characters (category code 12) and spaces (category code 10).

```

\BNVS_tl_to_str_unbraced:Nn \BNVS_tl_to_str_unbraced:Nn ⟨function⟩ ⟨content⟩
\BNVS_tl_to_str_unbraced:nn \BNVS_tl_to_str_unbraced:nn {⟨instructions⟩} ⟨content⟩

```

Based on `\tl_to_str:n`.

```

1477 \cs_new:Npn \BNVS_tl_to_str_unbraced:Nn #1 #2 {
1478   \exp_last_unbraced:No #1 { \tl_to_str:n { #2 } }
1479 }

```

```

1480 \cs_new:Npn \BNVS_tl_to_str_unbraced:nn #1 #2 {
1481   \exp_last_unbraced:Nno \use:n { #1 } { \tl_to_str:n { #2 } }
1482 }

```

1.12.1 Base grammar

Hereafter is a pragmatic grammar used for parsing, not all the expressions covered do have a real meaning. The characters all have category other. We start with basic entities.

```

1  <spc>          ::= Horizontal white space characters
2  <lower>        ::= a lowercase letter
3  <upper>        ::= an uppercase letter
4  <digit>        ::= a decimal digit
5  <sign>         ::= '-' | '+'
6  <comma>        ::= <spc> ',' <spc>
7  <alpha>        ::= <lower> | <upper> | '_'
8  <alnum>        ::= <alpha> | <digit>
9  <int>          ::= <digit>+
10 <index>        ::= <sign>* <int>+
11 <number>       ::= <sign>? <significand> <exponent>?
12 <significand> ::= ( '.'? <digit>+ ) | ( <digit>+ '.' <digit>* )
13 <exponent>     ::= ( 'e' | 'E' ) <sign>? <digit>+
14 <other>        ::= <number> | '+' | '-' | '*' | '/'

```

\c__bnvs_N_regex Signed integer.

(End of definition for \c__bnvs_N_regex.)

```

1483 \regex_const:Nn \c__bnvs_N_regex {
1484   [-+]* \d+
1485 }

```

\c__bnvs_dot_N_regex Signed integer with a dot . just before.

(End of definition for \c__bnvs_dot_N_regex.)

```

1486 \regex_const:Nn \c__bnvs_dot_N_regex {
1487   \. \ur{c__bnvs_N_regex}
1488 }

```

\c__bnvs_A_dot_N_Z_regex Full signed integer with a dot . jus before.

(End of definition for \c__bnvs_A_dot_N_Z_regex.)

```

1489 \regex_const:Nn \c__bnvs_A_dot_N_Z_regex {
1490   \A \ur { c__bnvs_dot_N_regex } \Z
1491 }

```

\c__bnvs_A_i_Z_regex Signed integer with no capture groups.

(End of definition for \c__bnvs_A_i_Z_regex.)

```

1492 \regex_const:Nn \c__bnvs_A_i_Z_regex { \A \ur{c__bnvs_N_regex} \Z }

```

`\c__bnvs_dot_R_regex` A possibly empty list of . *⟨short name⟩* items representing a path. Integers are allowed as well.

```

1493 \regex_const:Nn \c__bnvs_R_regex {
1494   \ur{c__bnvs_N_regex} | [[:alnum:]]_+
1495 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_dot_R_regex` A possibly empty list of . *⟨short name⟩* items representing a path. Integers are allowed as well.

```

1496 \regex_const:Nn \c__bnvs_dot_R_regex {
1497   \. \ur{c__bnvs_R_regex}
1498 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_S_regex` This regular expression is used for short names. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit.

```

1499 \regex_const:Nn \c__bnvs_S_regex {
1500   [[:alpha:]]_+[[:alnum:]]_*
1501 }

```

(End of definition for \c__bnvs_S_regex.)

`\c__bnvs_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

1502 \regex_const:Nn \c__bnvs_P_regex {
1503   \d*[[:alpha:]]_+[[:alnum:]]_*
1504 }

```

(End of definition for \c__bnvs_P_regex.)

`\c__bnvs_dot_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

1505 \regex_const:Nn \c__bnvs_dot_P_regex {
1506   \. \ur{c__bnvs_P_regex}
1507 }

```

(End of definition for \c__bnvs_dot_P_regex.)

1.12.2 Storage

<code>\BNVS_xprt_put:n</code> <code>\BNVS_xprt_clear:</code>	<code>\BNVS_xprt_put:n {⟨content⟩★}</code> <code>\BNVS_xprt_clear:</code>
---	--

`\BNVS_xprt_put:n` is a utility function to store its argument in the `xprted` variable. `\BNVS_xprt_clear:` resets the scan storage, called when starting a new T_EX group dedicated to scanning.

```

1508 \cs_new:Npn \BNVS_xprt_clear: {
1509   \__bnvs_tl_clear:c { xprted }
1510 }

1511 \cs_new:Npn \BNVS_xprt_put:n #1 {
1512   \BNVS_xprt_ast_Raw:
1513   \__bnvs_tl_put_right:cn { xprted } { #1 }
1514 }
```

There is a special situation for raw material that is stored separately and exported at once before other material is exported or when a scan group ends.

<code>\BNVS_xprt_Raw:n</code> <code>\BNVS_xprt_ast_Raw:</code>	<code>\BNVS_xprt_Raw:n {⟨something⟩}</code> <code>\BNVS_xprt_ast_Raw:</code>
---	---

`\BNVS_xprt_Raw:n` synchronously cumulates its argument in the `xprted_Raw` variable. The function `\BNVS_xprt_ast_Raw:` is executed both at the beginning of the function `\BNVS_xprt_put:n` and when a scan group ends. It exports the contents of the stored `⟨material⟩`, when non empty, to the `xprted` variable as `\BNVS:n{⟨material⟩}` instruction and cleans the `xprted_Raw` variable.

```

1515 \cs_new:Npn \BNVS_xprt_Raw:n #1 {
1516   \tl_if_empty:nF { #1 } {
1517     \__bnvs_tl_put_right:cn { xprted_Raw } { #1 }
1518   }
1519 }

1520 \cs_new:Npn \BNVS_xprt_ast_Raw: {
1521   \__bnvs_tl_if_empty:cF { xprted_Raw } {
1522     \BNVS_tl_use:nv {
1523       \__bnvs_tl_clear:c { xprted_Raw }
1524       \BNVS_xprt_put:n { \BNVS:n }
1525       \BNVS_xprt_braced:n
1526     } { xprted_Raw }
1527   }
1528 }
```

<code>\BNVS_xprt_braced:n</code>	<code>\BNVS_xprt_braced:n {⟨scanned⟩}</code>
----------------------------------	--

Feeds the scan storage with `⟨scanned⟩`, enclosed between one pair of curly braces.

```

1529 \cs_new:Npn \BNVS_xprt_braced:n #1 {
1530   \BNVS_xprt_put:n { { #1 } }
1531 }
```

`\BNVS_xprted_brace:` `\BNVS_xprted_brace:`

Wraps what was exported so far between a pair of braces.

```

1532 \cs_new:Npn \BNVS_xprted_brace: {
1533   \exp_args:NNx \BNVS_xprt_clear: \BNVS_xprt_put:n {
1534     { \BNVS_xprted_use:n { \exp_not:n } }
1535   }
1536 }
```

`\BNVS_xprt_space:` `\BNVS_xprt_space:`
`\BNVS_xprt_space_set:n` `\BNVS_xprt_space_set:n {<code>}`

`\BNVS_xprt_space:` exports the space character. `\BNVS_xprt_space_set:n` sets the meaning of `\BNVS_xprt_space:` to `<code>`.

```

1537 \cs_new:Npn \BNVS_xprt_space_set:n #1 {
1538   \cs_set:Npn \BNVS_xprt_space: {
1539     #1
1540   }
1541 }

1542 \BNVS_xprt_space_set:n {\BNVS_xprt_Raw:n { ~ } }
```

`\BNVS_xprted_use:n` `\BNVS_xprted_use:n {<code:n>}`
`\BNVS_xprted_converter_set:n` `\BNVS_xprted_converter_set:n {<converter:n>}`

The `\BNVS_xprted_use:n` exports the raw material, executes `<converter:n>`, previously stored by `\BNVS_xprted_convert_set:n`, then puts back to the input stream the instructions `<code:n>` followed by the exported material so far, enclosed between one pair of braces. The `<converter:n>` body instructions take as unique argument the exported material so far at the current level. It should export some more material because the previous one has been cleared out. It is used only once.

```

1543 \cs_new:Npn \BNVS_xprted_use:n #1 {
1544   \BNVS_xprt_ast_Raw:
1545   \BNVS_xprted_convert:
1546   \BNVS_tl_use:nv { #1 } { xprted }
1547 }

1548 \cs_new:Npn \BNVS_xprted_convert: {}
```

❗ Logically unreachable.

```

1549 \cs_new:Npn \BNVS_xprted_convert:n {
1550   \BNVS_ast_put:n
1551 }

1552 \cs_new:Npn \BNVS_xprted_converter_set:n #1 {
1553   \cs_set:Npn \BNVS_xprted_convert:n ##1 {
1554     #1
1555   }
1556   \cs_set:Npn \BNVS_xprted_convert: {
1557     \cs_set:Npn \BNVS_xprted_convert: { }
1558     \BNVS_tl_use:nv { \BNVS_xprt_clear: \BNVS_xprted_convert:n } { xprted }
1559   }
1560 }
```

\BNVS_if_xprted_empty:TF	\BNVS_if_xprted_empty:TF {<yes:n>} {<no:n>}
--------------------------	---

Execute <yes:n> instructions if something has been exported, <no:n> instructions otherwise.

```

1561 \cs_new:Npn \BNVS_if_xprted_empty:TF {
1562   \BNVS_xprted_use:n { \tl_if_empty:nTF }
1563 }

```

1.12.3 AST exportation policy

This is used by \BNVS_end_scaN: to wrap the AST created during the scanning process before it is exported. The case of an empty created AST is sometimes special.

\BNVS_ast_pfX_set:n	\BNVS_ast_pfX_set:n {<pfX:w>}
\BNVS_ast_pfX_will_set:n	\BNVS_ast_pfX_will_set:n {<pfX:w>}

Set the function \BNVS_ast_pfX:w to expand to \exp_not:n {<pfX:w>}. This is used to define \BNVS_ast_node_make:n, when the arguments must be scanned prior to knowing which function <pfX:w> fits. The “will” variant defers the execution just before \BNVS_end:.

```

1564 \cs_new:Npn \BNVS_ast_pfX:w {}

1565 \cs_new:Npn \BNVS_ast_pfX_set:n #1 {
1566   \cs_set:Npn \BNVS_ast_pfX:w { \exp_not:n { #1 } }
1567 }

1568 \cs_new:Npn \BNVS_ast_pfX_will_set:n #1 {
1569   \BNVS_will_end:n {
1570     \BNVS_ast_pfX_set:n { #1 }
1571   }
1572 }

```

\BNVS_ast_plc_init:n	\BNVS_ast_plc_init:n {<template:n>}
\BNVS_ast_plc_init:X	\BNVS_ast_plc_init:X {<pfX>}
\BNVS_ast_plc_init::X::n	\BNVS_ast_plc_init::X::n {<pfX>} {<template:n>}
\BNVS_ast_plc_init_pfx:n	\BNVS_ast_plc_init_pfx:n {<template:n>}
\BNVS_ast_plc_init_mpt:n	\BNVS_ast_plc_init_mpt:n {<template:n>}
\BNVS_ast_plc_init_mpt:X	\BNVS_ast_plc_init_mpt:X {<pfX>}
\BNVS_ast_plc_init_mpt::X::n	\BNVS_ast_plc_init_mpt::X::n {<pfX>} {<template:n>}
\BNVS_ast_plc_init_grp:n	\BNVS_ast_plc_init_grp:n {<template:n>}
\BNVS_ast_plc_init:XXn	\BNVS_ast_plc_Raw:
\BNVS_ast_plc_brc:	\BNVS_ast_plc_none:
\BNVS_ast_plc_Raw:	
\BNVS_ast_plc_none:	

Initialize the AST exportation policy through the meaning of the auxiliary function \BNVS_ast_node_make:n. The <template:n> is by default ##1. With the first variant, nothing is exported if the input AST is blank, this is the default exportation policy. With the pfx variant, only the <pfX:w> is exported when the input AST is blank. The mpt and grp variants never export an empty AST. The first one exports the input AST as is whereas the second one exports the input AST between a pair of braces. \BNVS_ast_plc_Raw: exports as is, with no brace nor prefix. \BNVS_ast_plc_ignore: exports nothing at all. The ...:X variant is equivalent to the ...:X::n variant with {##1} as last argument.

```

1573 \cs_new:Npn \BNVS_ast_plc_init:n #1 {
1574   \cs_set:Npn \BNVS_ast_node_make:n ##1 {
1575     \tl_if_blank:nF { ##1 } {
1576       \BNVS_ast_pfx:w
❧ and \exp_not:n{⟨xprt:n⟩}.
1577       \exp_not:n { #1 }
1578     }
1579   }
1580 }
1581 \BNVS_ast_plc_init:n { #1 }

1582 \cs_new:Npn \BNVS_ast_plc_init:X #1 {
1583   \BNVS_ast_pfx_set:n { #1 }
1584   \BNVS_ast_plc_init:n { { ##1 } }
1585 }

1586 \cs_new:Npn \BNVS_ast_plc_init::X::n #1 #2 {
1587   \BNVS_ast_pfx_set:n { #1 }
1588   \BNVS_ast_plc_init:n { #2 }
1589 }

1590 \cs_new:Npn \BNVS_ast_plc_init_pfx:n #1 {
1591   \cs_set:Npn \BNVS_ast_node_make:n ##1 {
1592     \BNVS_ast_pfx:w
1593     \tl_if_blank:nF { ##1 } {
❧ and \exp_not:n{⟨xprt:n⟩}.
1594     \exp_not:n { #1 }
1595   }
1596 }
1597 }

1598 \cs_new:Npn \BNVS_ast_plc_init_mpt:n #1 {
1599   s
1600   \cs_set:Npn \BNVS_ast_node_make:n ##1 {
1601     \BNVS_ast_pfx:w
❧ and \exp_not:n{⟨xprt:n⟩}.
1602     \exp_not:n { #1 }
1603   }
1604 }

1605 \cs_new:Npn \BNVS_ast_plc_init_mpt:X #1 {
1606   \BNVS_ast_pfx_set:n { #1 }
1607   \BNVS_ast_plc_init_mpt:n { { ##1 } }
1608 }

1609 \cs_new:Npn \BNVS_ast_plc_init_mpt::X::n #1 #2 {
1610   \BNVS_ast_pfx_set:n { #1 }
1611   \BNVS_ast_plc_init_mpt:n { #2 }
1612 }

1613 \cs_new:Npn \BNVS_ast_plc_init_grp:n #1 {
1614   s
1615   \cs_set:Npn \BNVS_ast_node_make:n ##1 {
1616     \BNVS_ast_pfx:w

```

```

❧ and \exp_not:n{⟨xp̄rt:n⟩}.
1617     \exp_not:n { { #1 } }
1618   }
1619 }

1620 \cs_new:Npn \BNVS_ast_plc_init::XX::n #1 #2 #3 {
1621   \cs_set:Npn \BNVS_ast_node_make:n ##1 {
1622     \tl_if_blank:nTF { ##1 } {
1623       \exp_not:n { #1 }
1624     } {
1625       \exp_not:n { #2 }
❧ and \exp_not:n{⟨xp̄rt:n⟩}.
1626     \exp_not:n { #3 }
1627   }
1628 }
1629 }

1630 \cs_new:Npn \BNVS_ast_plc_brc: {
1631   \cs_set:Npn \BNVS_ast_node_make:n ##1 {
1632     \exp_not:n { { ##1 } }
1633   }
1634 }

1635 \cs_new:Npn \BNVS_ast_plc_Raw: {
1636   s
1637   \cs_set:Npn \BNVS_ast_node_make:n ##1 { \exp_not:n { ##1 } }
1638 }

1639 \cs_new:Npn \BNVS_ast_plc_none: {
1640   s
1641   \cs_set:Npn \BNVS_ast_node_make:n ##1 { }
1642 }

```

1.12.4 Branch functions

The scanning process only applies to characters of category codes space and other. It definitely stops at the first token that is not of that category. The expressions scanned may contain properly balanced delimiters (...), [...] or {...}. The single characters ‘,’ and ‘:’ as well as a ‘::’ combination are the global separators. The scanning process takes one of 6 branches depending on what is scanned:

- $\langle B:w \rangle$ or $\backslash\text{BNVS_scaNned_B:w}$, for unit Blocks like names,
- $\langle O:w \rangle$ or $\backslash\text{BNVS_scaNned_O:w}$, after One single colon,
- $\langle C:w \rangle$ or $\backslash\text{BNVS_scaNned_C:w}$, after multiple Colons,
- $\langle M:w \rangle$ or $\backslash\text{BNVS_scaNned_M:w}$, after a coMma,
- $\langle L:w \rangle$ or $\backslash\text{BNVS_scaNned_L:w}$, after a cLosing delimiter,
- $\langle S: \rangle$ or $\backslash\text{BNVS_scaNned_S:}$, in Stop situations, when scanning is done.

The terminating w in the function signature means that the $\langle \text{input stream} \rangle$ will be scanned.

Block branch The Block branch is used when the last character scanned from the *<input stream>* fits the current requirement as defined by the last scan function called, but the next character does not.

<code>\BNVS_scaN_B:Fw</code>	<code>\BNVS_scaN_B:Fw {<F:w>} <input stream></code>
<code>\BNVS_scaNned_B:w</code>	<code>\BNVS_scaNned_B:w <input stream></code>
<code>\BNVS_scaNned_set::B</code>	<code>\BNVS_scaNned_set::B {<B:w>}</code>
<code>\BNVS_scaN::B::Fw</code>	<code>\BNVS_scaN::B::Fw {<scaN::B:w>} {<F:w>} <input stream></code>
<code>\BNVS_scaN_def::B:F</code>	<code>\BNVS_scaN_def::B:F {<block::B::Fw>}</code>

The `\BNVS_scaNned_B:w` branch function is called by `\BNVS_scaN_B:Fw` once a *<block>* has been scanned from the head of the *<input stream>* (greedy match). The function `\BNVS_scaNned_set::B` sets the meaning of `\BNVS_scaNned_B:w` to *<B:w>*. The exact definition of a block is driven by `\BNVS_scaN::B::Fw` which meaning is set by `\BNVS_scaN_def::B:F`.

```

1643 \cs_new:Npn \BNVS_scaN_B:Fw {
1644     \BNVS_scaN::B::Fw {
1645         \BNVS_scaNned_B:w
1646     }

```

⊘ There must be absolutely no code here. No hooking allowed either.

```

1647 }

1648 \cs_new:Npn \BNVS_scaN_def::B:F #1 {
1649     \cs_set:Npn \BNVS_scaN::B::Fw ##1 ##2 {
1650         #1
1651     }
1652 }

1653 \BNVS_scaN_def::B:F {

```

¶ At the top level, the *<F:w>* instructions are always executed.

```

1654     #2
1655 }

```

Colon branch

`\c__bnvs_Colons_regex` For ranges defined by a colon syntax. One catching group for more than one colon.

```

1656 \regex_const:Nn \c__bnvs_Colons_regex { \s* :(:*) \s* }

```

(End of definition for `\c__bnvs_Colons_regex`.)

<code>\BNVS_scaN_OC:Fw</code>	<code>\BNVS_scaN_OC:Fw {<F:w>} <input stream></code>
<code>\BNVS_scaNned_O:w</code>	<code>\BNVS_scaNned_O:w <input stream></code>
<code>\BNVS_scaNned_set::O</code>	<code>\BNVS_scaNned_set::O {<O:w>}</code>
<code>\BNVS_scaNned_C:w</code>	<code>\BNVS_scaNned_C:w <input stream></code>
<code>\BNVS_scaNned_set::C</code>	<code>\BNVS_scaNned_set::C {<C:w>}</code>

On scanning a standalone colon from the *<input stream>*, calls the branch function `\BNVS_scaNned_O:w`. On scanning multiple colons from the *<input stream>*, calls the `\BNVS_scaNned_C:w` branch function. In other cases, execute the *<F:w>* instructions.

¶ One shot utility function only used by `\BNVS_scaN_OC:Fw`

```

1657 \cs_new:Npn \BNVS_scaNned_C_regex:n #1 {
1658     \tl_if_empty:nTF { #1 } {

```

```

1659     \BNVS_scaNned_0:w
1660   } {
1661     \BNVS_scaNned_C:w
1662   }
1663 }

1664 \cs_new:Npn \BNVS_scaN_OC:Fw #1 {
1665   \peek_regex_replace_once:NnTF \c__bnvs_Colons_regex { \cB\{ \1 \cE\} } {
1666     \BNVS_scaNned_C_regex:n
1667   } {
1668     #1
1669   }
1670 }

```

Comma branch

\c__bnvs_M_regex Comma as a separator.

```

1671 \regex_const:Nn \c__bnvs_M_regex { \s* , \s* }

```

(End of definition for \c__bnvs_M_regex.)

🔗 \peek_regex_remove_once:NnTF is buggy in L3 programming layer <2025-01-18>.

```

1672 \cs_new:Npn \BNVS_peek_regex_remove_once:NnTF #1 {
1673   \peek_regex_replace_once:NnTF #1 { }
1674 }

```

\BNVS_scaN_M:Fw	\BNVS_scaN_M:Fw {⟨F:w⟩} ⟨input stream⟩
\BNVS_scaNned_M:w	\BNVS_scaN_M:w ⟨input stream⟩
\BNVS_scaNned_set::M	\BNVS_scaN_M_set:n {⟨M:w⟩}

Once a comma has just been scanned from the head of the ⟨input stream⟩, calls \BNVS_scaNned_M:w branch function, otherwise execute ⟨F:w⟩ instructions. Function \BNVS_scaN_M_set:n sets the meaning of \BNVS_scaN_M:w to ⟨comma:w⟩.

```

1675 \cs_new:Npn \BNVS_scaN_M:Fw #1 {
🚫 \peek_regex_remove:NnTF is buggy in L3 programming layer <2025-01-18>.
1676   \BNVS_peek_regex_remove_once:NnTF \c__bnvs_M_regex {
1677     \BNVS_scaNned_M:w
1678   } {
1679     #1
1680   }

```

🚫 There must be absolutely no code here. No hooking allowed either.

```

1681 }

```

Close branch

\c__bnvs_cLose_regex Closing delimiters.

```

1682 \regex_const:Nn \c__bnvs_cLose_regex { \s* (%([{
1683   \}|\\]|\\)
1684   ) \s*
1685 }
```

(End of definition for \c__bnvs_cLose_regex.)

\BNVS_scaN_L:Fw	\BNVS_scaN_L:Fw {<F:w>} <input stream>
\BNVS_scaNned_L:nw	\BNVS_scaNned_L:nw <clositer> <input stream>
\BNVS_scaNned_L:w	\BNVS_scaNned_L:w <input stream>
\BNVS_scaNned_L_set:	\BNVS_scaNned_set::L <L:w>

When the \BNVS_scaN_L:Fw function scans one of ‘)’, ‘]’ or ‘}’ closing delimiter, it calls the branch function \BNVS_scaNned_L:nw with that delimiter as argument, which in turn calls \BNVS_scaNned_L:w, after it manages eventual errors in balancing delimiters. Otherwise execute <F:w> code.

```

1686 \cs_new:Npn \BNVS_scaN_L:Fw #1 {
1687   \peek_regex_replace_once:NnTF \c__bnvs_cLose_regex { \cB\{ \1 \cE\} } {
1688     \BNVS_scaNned_L:nw
1689   } {
1690     #1
1691   }
1692 }

1693 \cs_new:Npn \BNVS_scaNned_L:nw #1 {
1694   \BNVS_error:n { Unexpected-delimiter~`#1' }
1695   \BNVS_scaNned_L:w
1696 }

1697 \tl_map_inline:nn {{Block}{One_colon}{Colons}{comMa}{cLose}} {
1698   \cs_new:cpn { BNVS_scaNned_#1_set:n } ##1 {
1699     \cs_set:cpn { BNVS_scaNned_#1:w } {
1700       ##1
1701     }
1702 }
```

❗ Do nothing operation at the top level.

```

1703 \cs_new:cpn { BNVS_scaNned_#1:w } {
1704 }
1705 }

1706 \tl_map_inline:nn {BOCML} {
1707   \cs_new:cpn { BNVS_scaNned_set::#1 } ##1 {
1708     \cs_set:cpn { BNVS_scaNned_#1:w } {
1709       ##1
1710     }
1711 }
```

❗ Do nothing operation at the top level.

```

1712 \cs_new:cpn { BNVS_scaNned_#1:w } {
1713 }
1714 }
```


Stop branch The Stop branch is executed in the Stop situation, *id est* when the head of the `<input stream>` cannot be scanned. In practice, when the head of the `<input stream>` is not a character of category code space or other.

<code>\BNVS_scaN_S:F</code> <code>\BNVS_scaNned_S:</code> <code>\BNVS_scaNned_set::S</code>	<code>\BNVS_scaN_S:F {<F:w>} <input stream></code> <code>\BNVS_scaNned_S:</code> <code>\BNVS_scaNned_set::S {<S:>}</code>
---	---

When the head of the `<input stream>` is not a space nor a character of category code other, `\BNVS_scaN_S:F` calls the `\BNVS_scaNned_S:` branch function, otherwise execute `<F:w>` instructions. The function `\BNVS_scaNned_set::S` sets the meaning of `\BNVS_scaNned_S:` to `<S:>`.

```

1715 \cs_new:Npn \BNVS_scaNned_set::S #1 {
1716   \cs_set:Npn \BNVS_scaNned_S: {
1717     #1
1718   }
1719 }
```

¶ Do nothing operation at the top level.

```

1720 \cs_new:Npn \BNVS_scaNned_S: {
1721 }
```

```

1722 \cs_new:Npn \BNVS_scaN_S:F #1 {
1723   \peek_catcode:NTF \c_catcode_other_token { #1 } {
1724     \peek_catcode_remove:NTF \c_space_token {
```

¶ Export spaces, depending on the current policy.

```

1725     \BNVS_xprt_space:
1726     #1
1727   } {
1728     \BNVS_scaNned_S:
1729   }
1730 }
```

⊘ There must be absolutely no code here. No hooking allowed either.

```

1731 }
```

Branch setter

<code>\BNVS_scaNned_set::BOCMLS</code> <code>\BNVS_scaNned_set::OCMLS</code> <code>\BNVS_scaNned_set::OC</code> <code>\BNVS_scaNned_set::LS</code> <code>\BNVS_scaNned_set::MLS</code>	<code>\BNVS_scaNned_set::BOCMLS {<B:w>} {<O:w>} {<C:w>} {<M:w>} {<L:w>} {<S:>}</code> <code>\BNVS_scaNned_set::OCMLS {<O:w>} {<C:w>} {<M:w>} {<L:w>} {<S:>}</code> <code>\BNVS_scaNned_set::OC {<O:w>} {<C:w>}</code> <code>\BNVS_scaNned_set::LS {<L:w>} {<S:>}</code> <code>\BNVS_scaNned_set::MLS {<M:w>} {<L:w>} {<S:>}</code>
--	--

Combination of branch setters. Better for reading experience.

```

1732 \cs_new:Npn \BNVS_scaNned_set::BOCMLS #1 {
1733   \BNVS_scaNned_set::B { #1 }
1734   \BNVS_scaNned_set::OCMLS
1735 }
```

```

1736 \cs_new:Npn \BNVS_scaNned_set::OC #1 #2 {
1737   \BNVS_scaNned_set::O { #1 }
1738   \BNVS_scaNned_set::C { #2 }
1739 }

1740 \cs_new:Npn \BNVS_scaNned_set::OCMLS #1 #2 {
1741   \BNVS_scaNned_set::OC { #1 } { #2 }
1742   \BNVS_scaNned_set::MLS
1743 }

1744 \cs_new:Npn \BNVS_scaNned_set::LS #1 #2 {
1745   \BNVS_scaNned_set::L { #1 }
1746   \BNVS_scaNned_set::S { #2 }
1747 }

1748 \cs_new:Npn \BNVS_scaNned_set::MLS #1 {
1749   \BNVS_scaNned_set::M { #1 }
1750   \BNVS_scaNned_set::LS
1751 }

```

1.12.5 Begin and end of scanning processes

\BNVS_begin_scaN: [\BNVS_begin_scaN:](#)

Start a scanning group. *Implementation detail:* the function is split into 3 parts for debugging and testing purposes.

```

1752 \cs_new:Npn \BNVS_begin_scaN: {
1753   \BNVS_scaN_will_begin:
1754   \BNVS_begin:
1755   \BNVS_scaN_did_begin:
1756 }

1757 \cs_new:Npn \BNVS_scaN_will_begin: {
1758   \BNVS_xprt_ast_Raw:
1759 }

1760 \cs_new:Npn \BNVS_scaN_did_begin: {
1761   \BNVS_xprt_clear:
1762   \BNVS_ast_plc_init:n { ##1 }
1763   \BNVS_ast_pfX_set:n {}
1764   \cs_set:Npn \BNVS_xprted_convert: {}
1765 }

```

\BNVS_end_scaN:n [\BNVS_end_scaN:n](#) $\{\langle content \rangle^*\}$
\BNVS_end_scaN: [\BNVS_end_scaN:](#)

Exactly one of these functions must be called to close the T_EX group opened while beginning the scanning process. [\BNVS_end_scaN:](#) call [\BNVS_end_scaN:n](#) on what was exported before closing the group (at this level), once prepared by [\BNVS_ast_node_make:n...](#) [\BNVS_end_scaN_no_I_change:](#) does not export I and S($\langle I \rangle$) variables.

```

1766 \cs_new:Npn \BNVS_end_scaN:n #1 {
1767   \BNVS_will_end:n {
1768     \__bnvs_tl_if_empty:cF { I } {

```

```

1769     \BNVS_tl_use:nv { \_bnvs_tl_set:cn I } I
1770     \BNVS_tl_use:nv { \_bnvs_tl_set:cn { S ( \l__bnvs_I_tl ) } } {
1771         S ( \l__bnvs_I_tl )
1772     }
1773 }
1774 }

```

❗ `\BNVS_ast_node_make:n` must be called as lately as possible before ending the group.

```

1775 \BNVS_will_end:n {
1776     \BNVS_did_end:n { \BNVS_xprt_put:n }
1777     \exp_args:Nx \BNVS_did_end_braced:n {
1778         \BNVS_ast_node_make:n { #1 }
1779     }
1780 }
1781 \BNVS_end:
1782 }

1783 \cs_new:Npn \BNVS_end_scaN: {
1784     \BNVS_xprted_use:n {
1785         \BNVS_end_scaN:n
1786     }
1787 }

```

<code>\BNVS_scaNned_B_wrap:n:B</code> <code>\BNVS_scaNned_B_wrap::B</code>	<code>\BNVS_scaNned_B_wrap:n:B {⟨before⟩} {⟨scaN::B:w⟩}</code> <code>\BNVS_scaNned_B_wrap::B {⟨scaN::B:w⟩}</code>
---	--

Build the `\BNVS_scaNned_B:w` branch function. `⟨before⟩` instructions are executed before ending the current scan group whereas `⟨scaN::B:w⟩` instructions are executed after. They must terminate by a branch function. `\BNVS_scaNned_B_wrap:n:B` calls `\BNVS_will_end:n1` and `\BNVS_scaNned_B_wrap::B`. with an empty `⟨before⟩`.

```

1788 \cs_new:Npn \BNVS_scaNned_B_wrap:n:B #1 {
1789     \BNVS_will_end:n { #1 }
1790     \BNVS_scaNned_B_wrap::B
1791 }

1792 \cs_new:Npn \BNVS_scaNned_B_wrap::B #1 {
1793     \BNVS_scaNned_set::B {
1794         \BNVS_end_scaN: #1
1795     }
1796 }

```

<code>\BNVS_no_I_change:</code> <code>\BNVS_end_scaN_no_I_change:</code>	<code>\BNVS_no_I_change:</code> <code>\BNVS_end_scaN_no_I_change:</code>
---	---

Used as `⟨before⟩` instruction in order not to export the frame identifier..

```

1797 \cs_new:Npn \BNVS_no_I_change: {
1798     \BNVS_will_end:n { \_bnvs_tl_clear:c I }
1799 }

1800 \cs_new:Npn \BNVS_end_scaN_no_I_change: {
1801     \BNVS_no_I_change:
1802     \BNVS_end_scaN:
1803 }

```

¹Not yet documented

1.12.6 Main scanners

<code>\BNVS_scaN_no_B:Fw</code>	<code>\BNVS_scaN_no_B:Fw {⟨F:w⟩}</code> <code>⟨input stream⟩</code>
---------------------------------	--

In progress.

```

1804 \cs_new:Npn \BNVS_scaN_no_B:Fw #1 {
1805   \BNVS_scaN_OC:Fw {
1806     \BNVS_scaN_M:Fw {
1807       \BNVS_scaN_L:Fw {
1808         \BNVS_scaN_S:F { #1 }
1809       }
1810     }
1811   }
1812 }
```

<code>\BNVS_scaN_no_B:w</code>	<code>\BNVS_scaN_no_B:w ⟨input stream⟩</code>
--------------------------------	---

For the scanning of `⟨balanced⟩` material in a `[⟨key⟩=]⟨balanced⟩` instruction, after delimited material or in comma separated lists. The scanning ends with blocks, or one of comma, colon, `\q_stop` or unbalanced closing delimiter.

```

1813 \cs_new:Npn \BNVS_scaN_no_B:w {
1814   \BNVS_scaN_Rnd_no_B:Fw {
1815     \BNVS_scaN_Srq_no_B:Fw {
1816       \BNVS_scaN_Crl_no_B:Fw {
1817         \BNVS_scaN_Run_no_B:Fw {
1818           \BNVS_scaN_Raw::B::Fw { \BNVS_scaN_no_B:w } {
1819             \BNVS_scaN::B::Fw { \BNVS_scaN_no_B:w } {
1820               \BNVS_scaN_no_B:Fw {
1821                 \BNVS_scaN_Raw::N::nw \BNVS_scaN_no_B:w
1822               }
1823             }
1824           }
1825         }
1826       }
1827     }
1828   }
1829 }
```

<code>\BNVS_scaN_BOCMLS:w</code>	<code>\BNVS_scaN_BOCMLS:w ⟨input stream⟩</code>
<code>\BNVS_scaN_Raw::N::nw</code>	<code>\BNVS_scaN_Raw::N::nw ⟨scaN:w⟩ {⟨raw⟩} ⟨input stream⟩</code>

Medley of the above `\BNVS_scaN_...:Fw` scan functions. Mutability is allowed by a prior call to `\BNVS_scaN_def::B:F`.

```

1830 \cs_new:Npn \BNVS_scaN_BOCMLS:w {
1831   \BNVS_scaN_B:Fw {
1832     \BNVS_scaN_no_B:Fw {
1833       \BNVS_scaN_Raw::N::nw \BNVS_scaN_BOCMLS:w
1834     }
1835   }
1836 }

1837 \cs_new:Npn \BNVS_scaN_Raw::N::nw #1 #2 {
```

```

1838 \BNVS_xprt_Raw:n { #2 }
1839 #1
1840 }

```

```

\__bnvs_scaN_other:Nw \__bnvs_scaN_other:Nw <character> <input stream>

```

The *<character>* is stored by `\BNVS_xprt_put:n` then `\BNVS_scaN_no_B:w` is executed.

```

\BNVS_scaN_Raw_OCMLS:w \BNVS_scaN_Raw_OCMLS:w <input stream>

```

In progress.

```

1841 \cs_new:Npn \BNVS_scaN_Raw_OCMLS:w {
1842   \BNVS_scaN_no_B:Fw {
1843     \BNVS_scaN::B::Fw {
1844       \BNVS_scaN_Raw_OCMLS:w
1845     } {
1846       \BNVS_scaN_Raw::N::nw \BNVS_scaN_Raw_OCMLS:w
1847     }
1848   }
1849 }

```

1.12.7 Scan sessions

```

\BNVS_scaN::RBOCMLSTN::w \BNVS_scaN::RBOCMLSTN::w {<R:>}
{<B:w>} {<D:w>} {<C:w>} {<M:w>} {<L:w>} {<S:>}
{<T:>} {<scaN:w>} <input stream>

```

Start a scanning session by executing the *<R:>* instructions. The six next arguments are function bodies used to define the corresponding branch functions. The *<T:>* and *<scaN:w>* instructions are finally executed. They are expected to scan the input stream and terminate by calling one of the branch functions just defined. They consist of some `\BNVS_scaN_...:w` function defined below.

The scanning functions have instructions as argument. Instead of simply using *n* in the signature of such functions, we use letters *RBOCMLSTN* as defined here, with a supplemental colon separator. Using *b* in the signature means that the instructions will be followed by a call to the `\BNVS_scaNned_B:w` function.

```

1850 \cs_new:Npn \BNVS_scaN::RBOCMLSTN::w #1 #2 #3 #4 #5 #6 #7 #8 #9 {
1851   #1
1852   \BNVS_scaNned_set::BOCMLS { #2 } { #3 } { #4 } { #5 } { #6 } { #7 }
1853   #8
1854   #9
1855 }

```

```

\BNVS_begin_scaN:nn \BNVS_begin_scaN:nn {<xprt:n>} {<next:>}

```

 **OBSOLETE?**. Call `\BNVS_begin_scaN:` with default arguments that end the scan group and call the eponym branch function.

```

1856 \cs_new:Npn \BNVS_begin_scaN:nn #1 #2 {
1857   \BNVS_begin_scaN:
1858   \BNVS_ast_plc_init:n { #1 }
1859 }

```

\BNVS_subscan::RTN::w	\BNVS_subscan::RTN::w {<R:>} {<T:>} {<scaN_no_B:w>} <input stream>
\BNVS_subscan::N::w	\BNVS_subscan::N::w {<scaN_BOCMLS:w>} <input stream>
\BNVS_subscan::MLSN::w	\BNVS_subscan::OCMLSN::w {<O:w>} {<C:w>} {<M:w>} {<L:w>} {<S:>} {<scaN_no_B:w>}
\BNVS_subscan::OCMLSN::w	<input stream>
\BNVS_subscan::BSN::w	\BNVS_subscan::BSN::w {<B:w>} {<S:>} {<scaN:w>} <input stream>

Start a scanning branch section by executing the <R:> instructions and setting the branch functions to default values. The <T:> instructions are then executed to eventually override these defaults. The process is executed within a group. Finally, the <scaN_BOCMLS:w> instructions are executed. They must terminate with a call to one of the branch functions.

```

1860 \cs_new:Npn \BNVS_subscan::RTN::w #1 #2 #3 {
1861   \BNVS_scaN::RBOCMLSTN::w {
1862     \BNVS_begin_scaN:
1863     #1
1864   } {
1865     \BNVS_end_scaN: \BNVS_scaNned_B:w
1866   } {
1867     \BNVS_end_scaN: \BNVS_scaNned_O:w
1868   } {
1869     \BNVS_end_scaN: \BNVS_scaNned_C:w
1870   } {
1871     \BNVS_end_scaN: \BNVS_scaNned_M:w
1872   } {
1873     \BNVS_end_scaN: \BNVS_scaNned_L:w
1874   } {
1875     \BNVS_end_scaN: \BNVS_scaNned_S:
1876   } {
1877     #2
1878   } {
1879     #3
1880   }
1881 }

1882 \cs_new:Npn \BNVS_subscan::N::w {
1883   \BNVS_subscan::RTN::w { } { }
1884 }

1885 \cs_new:Npn \BNVS_subscan::OCMLSN::w #1 #2 #3 #4 #5 {
1886   \BNVS_subscan::RTN::w { } {
1887     \BNVS_scaNned_set::OCMLS { #1 } { #2 } { #3 } { #4 } { #5 }
1888   }

```

⊘ There must be absolutely no code here. No hook is allowed either.

```

1889 }

1890 \cs_new:Npn \BNVS_subscan::MLSN::w #1 #2 #3 {
1891   \BNVS_subscan::RTN::w { } {
1892     \BNVS_scaNned_set::MLS { #1 } { #2 } { #3 }
1893   }

```

⊘ There must be absolutely no code here. No hook is allowed either.

```

1894 }

1895 \cs_new:Npn \BNVS_subscan::BSN::w #1 #2 #3 {
1896   \BNVS_scaN::RBOCMLSTN::w {

```

```

1897     \BNVS_begin_scaN:
1898   } {
1899     \BNVS_end_scaN: #1
1900   } {
1901     \BNVS_end_scaN: \BNVS_scaNned_0:w
1902   } {
1903     \BNVS_end_scaN: \BNVS_scaNned_C:w
1904   } {
1905     \BNVS_end_scaN: \BNVS_scaNned_M:w
1906   } {
1907     \BNVS_end_scaN: \BNVS_scaNned_L:w
1908   } {
1909     \BNVS_end_scaN: #2
1910   } {
1911   } {
1912     #3
1913   }
1914 }

```

1.12.8 Delimited units

We assume that delimiters are paired and properly balanced. Nevertheless, some errors are caught and managed appropriately. We start by a shared function.

```

\BNVS_scaN:N::RBSTN::w \BNVS_scaN:N::RBSTN:w <cLose>
{<R:>} {<B:w>} {<S:>} {<T:>} {<scaN:w>} <input stream>

```

An opening delimiter has been scanned, this function scans the *<input stream>* up to the balancing closing delimiter *<cLose>* using the *<scaN:w>* instructions. Implementation detail: we begin a scanning group that will be closed on scanning the *<cLose>* delimiter, a different closing delimiter, always at the same level, or in a Stop situation, in which case the closing delimiter is missing. The *<T:>* allows some fine tuning. The *<scaN:w>* function scans the *<input stream>*, and is expected to end on a closing delimiter at the current level by sending *\BNVS_scaNned_L:w*. The *<scaN:w>* will call itself after a block, a comma or a colon, if the corresponding branch function is called. This is the normal instruction flow. Only the *<B:w>* and *<S:>* branch functions are called *in fine*.

```

1915 \cs_new:Npn \BNVS_scaN:N::RBSTN::w #1 #2 #3 #4 #5 #6 {
1916   \BNVS_scaN::RBOCMLSTN::w {

```

¶ One group level for the contents.

```

1917     \BNVS_begin_scaN:
1918     \cs_set:Npn \BNVS_scaNned_L:nw ##1 {

```

¶ Anyways, branch here.

```

1919       \BNVS_scaNned_L:w
1920     }
1921     #2
1922   } {

```

¶ Any branch function, but the two last ones, executes the *<scaN:w>* instructions.

```

1923     #6
1924   } {
1925     #6

```

```

1926 } {
1927     #6
1928 } {
1929     #6
1930 } {

```

- ❗ Close branch. End the group, execute $\langle \text{block} \rangle$ instructions and calls the function $\backslash \text{BNVS_scaNned_B:w}$. Intermediate groups should be closed appropriately with the right definition of branch functions. The value of I is not exported to the containing group.

```

1931     \BNVS_end_scaN_no_I_change:
1932     #3
1933 } {

```

- ❗ Stop branch. Execute the $\langle \text{stop} \rangle$ instructions after the group ends.

```

1934     \BNVS_error:n { Missing~`#1'. }
1935     \BNVS_end_scaN_no_I_change:
1936     #4
1937 } {

```

- ❗ Execute the $\langle T \rangle$ instructions.

```

1938     #5
1939 } {

```

- ❗ Execute the $\langle \text{scaN:w} \rangle$ instructions.

```

1940     #6
1941 }
1942 }

```

$\langle \text{Rnd} \rangle$ **units.** Round delimiters are used to gather material, for function calls or like in mathematical expressions.

```

15  $\langle \text{Rnd}(\langle \text{blnc} \rangle) \rangle ::= \langle \text{RndOpen} \rangle \langle \text{blnc} \rangle \langle \text{RndClose} \rangle$ 
16  $\langle \text{RndOpen} \rangle ::= \langle \text{spc} \rangle ' ( ' \langle \text{spc} \rangle$ 
17  $\langle \text{RndClose} \rangle ::= \langle \text{spc} \rangle ' ) ' \langle \text{spc} \rangle$ 
18  $\langle \text{ISRC} \rangle ::= \langle \text{ISR} \rangle \langle C \rangle$ 
19  $\langle C \rangle ::= \langle \text{Rnd} \rangle$ 
20  $\langle \text{call} \rangle ::= \langle \text{RndOpen} \rangle \langle \text{blnc} \rangle \langle \text{RndClose} \rangle$ 

```

The corresponding AST is

```

 $\langle \text{ISRC} \rangle^* ::= \backslash \text{BNVS}::\text{ISRC} \{ \langle I \rangle \} \{ \langle S \rangle \} \{ \langle R \rangle^* \} \{ \langle \text{RndLst} \rangle^* \}$ 

```

$\backslash \text{c_bnvs_Rnd_open_regex}$ Round opening delimiter.

```

1943 \regex_const:Nn \c_bnvs_Rnd_open_regex { \s* \( \%
1944     \s* }

```


(End of definition for `\c__bnvs_Rnd_open_regex`.)

```

\BNVS_scaN_Rnd::RBSTN::Fw \BNVS_scaN_Rnd::RBSTN::Fw {⟨R:⟩} {⟨B:w⟩} {⟨S:⟩} {⟨T:⟩} {⟨scaN:w⟩} {⟨F:w⟩}
\BNVS_scaN_Rnd::BN::Fw    ⟨input stream⟩
\BNVS_scaN_Rnd_no_B:Fw    \BNVS_scaN_Rnd_no_B:Fw ⟨input stream⟩
\BNVS_scaN_Rnd::BN::Fw    \BNVS_scaN_Rnd::BN::Fw {⟨B:w⟩} {⟨scaN:w⟩} {⟨F:w⟩} ⟨input stream⟩

```

Scan a `⟨Rnd⟩` unit if any and execute one of `⟨B:w⟩` or `⟨S:⟩` instructions, otherwise execute `⟨F:w⟩` instructions.

```

1945 \cs_new:Npn \BNVS_scaN_Rnd::BN::Fw #1 {
1946   \BNVS_scaN_Rnd::RBSTN::Fw { } { #1 } { \BNVS_scaNned_S: } { }

```

⊘ There must be absolutely no code here. No hook is allowed either.

```

1947 }

1948 \cs_new:Npn \BNVS_scaN_Rnd_no_B:Fw {
1949   \BNVS_scaN_Rnd::BN::Fw { \BNVS_scaN_no_B:w } { \BNVS_scaN_VAZLLst_LS:w }

```

⊘ There must be absolutely no code here. No hook is allowed either.

```

1950 }

1951 \cs_new:Npn \BNVS_scaN_Rnd::RBSTN::Fw #1 #2 #3 #4 #5 #6 {
1952   \BNVS_peek_regex_remove_once:NTF \c__bnvs_Rnd_open_regex {
1953     \BNVS_scaN:N::RBSTN::w %(
1954   ) {
1955     \BNVS_ast_plc_init_grp:n { ##1 }
1956     #1
1957   } { #2 } { #3 } { #4 } { #5 }
1958 } {
1959   #6
1960 }
1961 }

```

⟨Sqr⟩ units. Square delimiters are used for key–value lists where the key is optional.

```

2  ⟨Sqr⟩      ::= ⟨SqrOpen⟩ ⟨SqrLst⟩ ⟨SqrClose⟩
3  ⟨SqrOpen⟩ ::= ⟨spc⟩ '[' ⟨spc⟩
4  ⟨SqrClose⟩ ::= ⟨spc⟩ ']' ⟨spc⟩
5  ⟨SqrLst⟩   ::= ⟨SqrItem⟩? ( ⟨comma⟩ ⟨SqrItem⟩ ) *
6  ⟨SqrItem⟩  ::= ⟨NMOR⟩ | ⟨Rhs⟩
7  ⟨NMOR⟩     ::= ⟨N⟩ ⟨MOR⟩

```

The corresponding AST is

```

⟨Sqr⟩*      ::= \BNVSsqr:n { (⟨SqrItem⟩*) * }
⟨SqrItem⟩* ::= \BNVS::NMOR { ⟨N⟩ } { ⟨Mod⟩ } { ⟨Op⟩ } { ⟨Rhs⟩* } | \BNVS::R {
  ↪  ⟨NMOR⟩* }

```

`\c__bnvs_Sqr_open_regex` Square opening delimiter.

```

1962 \regex_const:Nn \c__bnvs_Sqr_open_regex { \s* \[ %]
1963   \s* }

```

(End of definition for `\c__bnvs_Sqr_open_regex`.)

```

\BNVS_scaN_Sqr::RBSTN::Fw \BNVS_scaN_Sqr::RBSTN::Fw {⟨R:⟩} {⟨B:w⟩} {⟨S:⟩} {⟨T:⟩} {⟨scaN:w⟩} {⟨F:w⟩}
\BNVS_scaN_Sqr::BN::Fw    ⟨input stream⟩
\BNVS_scaN_Srq_no_B:Fw    \BNVS_scaN_Sqr::BN::Fw {⟨B:w⟩} {⟨scaN:w⟩} {⟨F:w⟩} ⟨input stream⟩
\BNVS_scaN_Srq_BS:Fw      \BNVS_scaN_Srq_BS:Fw {⟨F:w⟩} ⟨input stream⟩

```

Scan `⟨Sqr⟩` unit with the help of `⟨scaN_LS:w⟩`, then execute one of `⟨B:w⟩` or `⟨S:⟩` instructions. `⟨scaN_LS:w⟩` must end with one of the branch functions `\BNVS_scaNned_L:w` or `\BNVS_scaNned_S:`. Otherwise execute `⟨F:w⟩` instructions.

```

1964 \cs_new:Npn \BNVS_scaN_Srq_no_B:Fw {
1965   \BNVS_scaN_Sqr::BN::Fw { \BNVS_scaN_no_B:w } { \BNVS_scaN_no_B:w }

```

⊘ There must be absolutely no code here. No hook is allowed either.

```

1966 }

1967 \cs_new:Npn \BNVS_scaN_Sqr::BN::Fw #1 {
1968   \BNVS_scaN_Sqr::RBSTN::Fw { } { #1 } { \BNVS_scaNned_S: } { }

```

⊘ There must be absolutely no code here. No hook is allowed either.

```

1969 }

1970 \cs_new:Npn \BNVS_scaN_Sqr::RBSTN::Fw #1 #2 #3 #4 #5 #6 {
1971   \BNVS_peek_regex_remove_once:NTF \c__bnvs_Sqr_open_regex {
1972     \BNVS_scaN:N::RBSTN::w %[
1973     ] {
1974       \BNVS_ast_plc_init_grp:n { ##1 }
1975       #1
1976     } { #2 } { #3 } { #4 } { #5 }
1977   } {
1978     #6
1979   }
1980 }

```

⟨Cr1⟩ units. Curly delimiters are used for key-value lists where the value is optional.

```

3  ⟨Cr1⟩ ::= ⟨Cr1Open⟩ ⟨Cr1Lst⟩ ⟨Cr1Close⟩
4  ⟨Cr1Open⟩= ⟨spc⟩ '{' ⟨spc⟩
5  ⟨Cr1Close⟩= ⟨spc⟩ '}' ⟨spc⟩
6  ⟨Cr1Lst⟩:= ⟨Cr1Item⟩? ( ⟨comma⟩ ⟨Cr1Item⟩ )*
7  ⟨Cr1Item⟩= ⟨ISR⟩ | ⟨ISRMOR⟩

```

The corresponding AST is

```

⟨Cr1⟩* ::= \BNVSCr1:n { ( { ⟨Cr1Item⟩* } )* }

```

`\c__bnvs_Cr1_open_regex` Curly opening delimiter.

(End of definition for `\c__bnvs_Cr1_open_regex`.)

```

1981 \regex_const:Nn \c__bnvs_Cr1_open_regex { \s* \c0 \{ %}
1982   \s* }

```

```

\BNVS_scaN_Cr1::RBSTN::Fw \BNVS_scaN_Cr1::RBSTN::Fw {⟨R:⟩} {⟨B:w⟩} {⟨S:⟩} {⟨T:⟩} {⟨scaN:w⟩} {⟨F:w⟩}
\BNVS_scaN_Cr1::BN::Fw    ⟨input stream⟩
\BNVS_scaN_Cr1_no_B:Fw    \BNVS_scaN_Cr1::BN::Fw {⟨B:w⟩} {⟨scaN:w⟩} {⟨F:w⟩} ⟨input stream⟩
\BNVS_scaN_Cr1_no_B:Fw    \BNVS_scaN_Cr1_no_B:Fw {⟨F:w⟩} ⟨input stream⟩

```

Scan $\langle Cr1 \rangle$ unit with the help of $\langle scaN:w \rangle$, then execute one of $\langle B:w \rangle$ or $\langle S: \rangle$ instructions. The $\langle scaN:w \rangle$ instructions must end with one of the branch functions $\backslash BNVS_scaNned_L:w$ or $\backslash BNVS_scaNned_S:.$ Otherwise execute $\langle F:w \rangle$ instructions.

```

1983 \cs_new:Npn \BNVS_scaN_Cr1::BN::Fw #1 {
1984   \BNVS_scaN_Cr1::RBSTN::Fw { } { #1 } { \BNVS_scaNned_S: } { }

```

⊗ There must be absolutely no code here. No hook is allowed either.

TEST/top/S:A

```

1985 }

1986 \cs_new:Npn \BNVS_scaN_Cr1_no_B:Fw {
1987   \BNVS_scaN_Cr1::BN::Fw { \BNVS_scaN_no_B:w } { \BNVS_scaN_no_B:w }

```

⊗ There must be absolutely no code here. No hook is allowed either.

TEST/top/S:A

```

1988 }

1989 \group_begin:
1990 \char_set_catcode_group_begin:N (
1991 \char_set_catcode_group_end:N )
1992 \char_set_catcode_other:N \{
1993 \char_set_catcode_other:N \}
1994 \cs_new:Npn \BNVS_scaN_Cr1::RBSTN::Fw #1 #2 #3 #4 #5 #6 (
1995   \BNVS_peek_regex_remove_once:NTF \c__bnvs_Cr1_open_regex (
1996     \BNVS_scaN:N::RBSTN::w %{
1997       } (
1998         \BNVS_ast_plc_init_grp:n ( ##1 )
1999         #1
2000       ) ( #2 ) ( #3 ) ( #4 ) ( #5 )
2001     ) (
2002       #6
2003     )
2004   )
2005 \group_end:

```

1.12.9 Raw units

Here is a partial grammar for raw material

```

2  ⟨Raw⟩      ::= ⟨RawOpen⟩ ⟨call⟩ ⟨RndClose⟩
3  ⟨RawOpen⟩ ::= ⟨spc⟩ ' '(' ⟨spc⟩
4  ⟨RawLst⟩   ::= ( ⟨RawRnd⟩ | ⟨RawSqr⟩ | ⟨RawCr1⟩ | ⟨RawOtr⟩ ) *
5  ⟨RawRnd⟩   ::= ⟨RndOpen⟩ ⟨RawLst⟩ ⟨RndClose⟩
6  ⟨RawSqr⟩   ::= ⟨SqrOpen⟩ ⟨RawLst⟩ ⟨SqrClose⟩
7  ⟨RawCr1⟩   ::= ⟨Cr1Open⟩ ⟨RawLst⟩ ⟨Cr1Close⟩
8  ⟨RawOtr⟩   ::= ⟨characters of category other except standard delimiters⟩

```

The corresponding AST is

```

<raw>*      ::= \BNVS:n {<other>*}
<other>* ::= <characters of category code other>

```

\c__bnvs_open_Raw_regex Raw opening delimiter.

```

2006 \regex_const:Nn \c__bnvs_open_Raw_regex { \s* "\ ( %)
2007      \s* }

```

(End of definition for \c__bnvs_open_Raw_regex.)

```

\BNVS_scaN_Raw_LS:w \BNVS_scaN_Raw_LS:w <input stream>
\BNVS_scaN_Raw:nw \BNVS_scaN_Raw:nw {<raw>} <input stream>

```

Scan balanced material taking only delimiters into account. It stops on an unbalanced closing delimiter or in a stop situation. Everything in between is exported as is.

```

2008 \cs_new:Npn \BNVS_scaN_Raw:nw #1 {
2009     \BNVS_xprt_put:n { #1 }
2010     \BNVS_scaN_Raw_LS:w
2011 }

```

```

2012 \group_begin:
2013 \char_set_catcode_group_begin:N <
2014 \char_set_catcode_group_end:N >
2015 \char_set_catcode_other:N \{
2016 \char_set_catcode_other:N \}
2017 \cs_new:Npn \BNVS_scaN_Raw_LS:w <

```

¶ If we find an opening delimiter.

```

2018 \BNVS_scaN_Rnd::RBSTN::Fw <
2019     \BNVS_xprt_space_set:n < \BNVS_xprt_put:n < ~ > >% (
2020     \BNVS_ast_plc_init_mpt:n < ##1 ) >
2021 > <
2022     \BNVS_scaNned_set::L <
2023     \BNVS_end_scaN_no_I_change:
2024     \BNVS_scaN_Raw_LS:w
2025 >
2026     \BNVS_scaNned_B:w
2027 > <% (
2028     \BNVS_error:n < Missing~`)' > % (
2029     \BNVS_xprt_put:n < ) >
2030     \BNVS_scaNned_S:
2031 > <
2032     \BNVS_xprt_put:n < ( > % )
2033 > <
2034     \BNVS_scaN_Raw_LS:w
2035 > <
2036     \BNVS_scaN_Sqr::RBSTN::Fw <
2037     \BNVS_xprt_space_set:n < \BNVS_xprt_put:n < ~ > >% [
2038     \BNVS_ast_plc_init_mpt:n < ##1 ] >
2039 > <
2040     \BNVS_scaNned_set::L <
2041     \BNVS_end_scaN_no_I_change:
2042     \BNVS_scaN_Raw_LS:w
2043 >

```

```

2044 \BNVS_scaNned_B:w
2045 > <%[
2046 \BNVS_error:n < Missing~`]' > %[
2047 \BNVS_xprt_put:n < ] >
2048 \BNVS_scaNned_S:
2049 > <
2050 \BNVS_xprt_put:n < [ > %[
2051 > <
2052 \BNVS_scaN_Raw_LS:w
2053 > <
2054 \BNVS_scaN_Cr1::RBSTN::Fw <
2055 \BNVS_xprt_space_set:n < \BNVS_xprt_put:n < ~ > >{%{
2056 \BNVS_ast_plc_init_mpt:n < ##1 } >
2057 > <
2058 \BNVS_scaNned_set::L <
2059 \BNVS_end_scaN_no_I_change:
2060 \BNVS_scaN_Raw_LS:w
2061 >
2062 \BNVS_scaNned_B:w
2063 > <{%{
2064 \BNVS_error:n < Missing~`]' > {%{
2065 \BNVS_xprt_put:n < } >
2066 \BNVS_scaNned_S:
2067 > <
2068 \BNVS_xprt_put:n < { > }%{
2069 > <
2070 \BNVS_scaN_Raw_LS:w
2071 > <

```

❗ The head of the *<input stream>* is not an opening delimiter, we look for a closing delimiter, or a stop situation or finally export one character.

```

2072 \BNVS_scaN_L:Fw < \BNVS_scaN_S:F < \BNVS_scaN_Raw:nw > >
2073 > > > >
2074 \group_end:

```

```

\BNVS_scaN_Raw::B::Fw \BNVS_scaN_Raw::B::Fw {<B:w>} {<F:w>} <input stream>

```

```

\BNVS_scaN_Raw_no_B::Fw Scan balanced material taking only delimiters into account. Starts on a "(.

```

```

2075 \cs_new:Npn \BNVS_scaN_Raw_no_B:Fw {
2076 \BNVS_scaN_Raw::B::Fw { \BNVS_scaN_no_B:w }

```

🚫 There must be absolutely no code here. No hooking allowed either.

```

2077 % \begin{KWN.test}{BNVS:Nn=\BNVS_scaN_Raw_no_B:Fw{/Block:w/empty}, eringo}
2078 % \BNVS_scaNned_set::B {
2079 % \BNVS_coverage:n { TEST / top / B:w }
2080 % }
2081 % \BNVS_scaN_Raw_no_B:Fw {
2082 % \test_fail:n { TEST / :Fw }
2083 % } "( )
2084 % \assert_coverage:nn { TEST / top / B:w } A
2085 % \assert_xprted:nn { } Z
2086 % \end{KWN.test}
2087 % \begin{KWN.test}{BNVS:Nn=\BNVS_scaN_Raw_no_B:Fw{/L:w/1}, eringo}
2088 % \BNVS_scaNned_set::B {

```

```

2089 % \BNVS_coverage:n { TEST / top / B:w }
2090 % }
2091 % \BNVS_scaN_Raw_no_B:Fw {
2092 % \test_fail:n { TEST / :Fw }
2093 % } "( 1,~2,~3 )
2094 % \assert_coverage:nn { TEST / top / B:w } A
2095 % \assert_xprted:nn { \BNVS:n { 1,~2,~3 } } Z
2096 % \end{KWN.test}
2097 % \begin{KWN.test}{BNVS:Nn=\BNVS_scaN_Raw::B::Fw{/Stop:/empty}, catch_error, eringo}
2098 % \BNVS_scaNned_set::S {
2099 % \BNVS_coverage:n { TEST / top / S: }
2100 % }
2101 % \BNVS_scaN_Raw_no_B:Fw {
2102 % \test_fail:n { TEST / :Fw }
2103 % } "(^A)
2104 % \assert_in_error:nn { Missing } 0
2105 % \assert_coverage:nn { TEST / top / S: } A
2106 % \assert_xprted:nn { } Z
2107 % \end{KWN.test}
2108 % \begin{KWN.test}{BNVS:Nn=\BNVS_scaN_Raw_no_B:Fw{/Stop:/1}, catch_error, eringo}
2109 % \BNVS_scaNned_set::S {
2110 % \BNVS_coverage:n { TEST / top / S: }
2111 % }
2112 % \BNVS_scaN_Raw_no_B:Fw {
2113 % \test_fail:n { TEST / ...F }
2114 % } "( 1,~2,~3^A)
2115 % \assert_in_error:nn { Missing } 0
2116 % \assert_coverage:nn { TEST / top / S: } A
2117 % \assert_xprted:nn { \BNVS:n { 1,~2,~3 } } Z
2118 % \end{KWN.test}
2119 % \begin{macrocode}
2120 }

2121 \cs_new:Npn \BNVS_scaN_Raw::B::Fw #1 #2 {
2122 \BNVS_peek_regex_remove_once:NTF \c__bnvs_open_Raw_regex {
2123 \BNVS_scaN:N::RBSTN::w %(
2124 ) {
2125 \BNVS_ast_plc_init_pfx:n { \BNVS:n { ##1 } }
2126 \BNVS_xprt_space_set:n { \BNVS_xprt_put:n { ~ } }
2127 } { #1 } { \BNVS_scaNned_S: } { } {
2128 \BNVS_scaN_Raw_LS:w
2129 }
2130 } {
2131 #2
2132 }
2133 }

```

1.12.10 $\langle csl(\langle item \rangle) \rangle$ units

Here is a partial grammar for a comma separated list template.

$$\langle csl(\langle item \rangle) \rangle ::= \langle item \rangle ? (\langle comma \rangle \langle item \rangle ?)^*$$

The corresponding AST template is below, where $\langle item \rangle^*$ is the AST for $\langle item \rangle$:

$$\langle csl(\langle item \rangle) \rangle^* ::= \{ (\{ \langle item \rangle^* \})^* \}$$

```
\BNVS_scaN_csl::RlsTN::w \BNVS_scaN_csl::RlsTN::w { \R: } { \l: } { \s: } { \T: } { \scaN_LS:w } \input stream
```

This function scans the $\langle input\ stream \rangle$ for a comma separated list of items, regardless enclosing delimiters, according to the $\langle scaN_LS:w \rangle$ instructions. These instruction must end with a branch function, of which only $\backslash BNVS_scaNned_L:w$ and $\backslash BNVS_scaNned_S:$ are caught. The scanned material is then exported according to the instructions given in the $\langle R: \rangle$. On scanning a comma, a new group is created. The exported material is $\{ \}$ for an empty list, for input a it is $\{ \{ a^* \} \}$, for input a,b it is $\{ \{ a^* \} \{ b^* \} \}$, and so on.

```
2134 \cs_new:Npn \BNVS_scaN_csl::RlsTN::w #1 #2 #3 #4 #5 {
2135   \BNVS_scaN::RBOCMLSTN::w {
```

❗ One group level for each item of the list.

```
2136   \BNVS_begin_scaN:
```

❗ Execute the $\langle R: \rangle$ instructions.

```
2137   \BNVS_ast_plc_init:n { { ##1 } }
2138   #1
2139 }
```

❗ For the first branch functions (BOC), execute the $\langle scaN:w \rangle$ instructions.

```
2140   { #5 }
2141   { #5 }
2142   { #5 }
2143   {
2144     \BNVS_if_xprted_empty:TF {
```

❗ Comma branch, the first item is empty, skip it.

```
2145     #5
2146   } {
```

❗ Comma branch, export the first item when not empty. Forwards I, to the outer group.

```
2147     \BNVS_end_scaN:
2148     \BNVS_scaN_csl::RlsTN::w { #1 } { } { } { #4 } { #5 }
2149   }
2150 }
```

❗ Close branch.

```
2151   { \BNVS_end_scaN: #2 \BNVS_scaNned_L:w }
2152   { \BNVS_end_scaN: #3 \BNVS_scaNned_S: }
```

❗ By default, the list contains only one item.

```
2153   { #4 }
2154   { #5 }
2155 }
```

```
\BNVS_scaN_csl::RT::w \BNVS_scaN_csl::RT::w {<R:>} {<T:>} <input stream>
```

These functions scan the comma separated list of items, regardless the enclosing delimiters, based on `\BNVS_scaN_BOCLMS:w`.

```
2156 \cs_new:Npn \BNVS_scaN_csl::RT::w #1 #2 {
2157   \BNVS_scaN_csl::RlsTN::w
2158   { #1 }
2159   { }
2160   { }
2161   { #2 }
2162   { \BNVS_scaN_BOCLMS:w }
2163 }
```

1.12.11 VAZL units

Here is a preliminary grammar for ranges or values:

```
2 <V>      ::= <blnc>
3 <A>      ::= <blnc>
4 <Z>      ::= <blnc>
5 <L>      ::= <blnc>
6 <VAZL>   ::= <V> | <AZL> | <ALZ>
7 <AZL>    ::= <A>? ':' <Z> ( ':' <L>? )?
8 <ALZ>    ::= <A>? ':' <L> ( ':' <Z>? )?
9 <blnc>   ::= <See below>
```

and the associate AST :

```
<V>*      ::= \BNVS::V {<blnc>*}
<A>*      ::= \BNVS::A {<blnc>*}
<Z>*      ::= \BNVS::Z {<blnc>*}
<L>*      ::= \BNVS::L {<blnc>*}
<AZ>*     ::= \BNVS::AZ {<blnc>*} {<blnc>*}
<AL>*     ::= \BNVS::AL {<blnc>*} {<blnc>*}
<ZL>*     ::= \BNVS::ZL {<blnc>*} {<blnc>*}
<LZ>*     ::= \BNVS::LZ {<blnc>*} {<blnc>*}
<AZL>*    ::= \BNVS::AZL {<blnc>*} {<blnc>*} {<blnc>*}
<ALZ>*    ::= \BNVS::ALZ {<blnc>*} {<blnc>*} {<blnc>*}
<star>*::= \BNVSStar:
<VAZL>*::= <V>* | <A>* | <Z>* | <L>* | <AZ>* | <AL>* | <ZL>* | <LZ>* | <AZL>* | <ALZ>*
↪ | \BNVSStar:
<blnc>*::= <See below>
```

```
\BNVS_skip_no_B::oc::w \BNVS_skip_no_B::oc::w {<O error>} {<C error>}
\BNVS_skip_no_B:w \BNVS_skip_OCML:w
```

Utilities. Skip range components, to recover from errors.

```
2164 \cs_set:Npn \BNVS_skip_no_B::oc::w #1 #2 {
2165   \BNVS_subscan::RTN::w { } {
2166     \BNVS_scaNned_set::OCMLS {
2167       #1
```



```

2168     \BNVS_ast_plc_none: \BNVS_end_scaN: \BNVS_skip_no_B::oc::w { } { }
2169 } {
2170     #2
2171     \BNVS_ast_plc_none: \BNVS_end_scaN: \BNVS_skip_no_B::oc::w { } { }
2172 } {
2173     \BNVS_ast_plc_none: \BNVS_end_scaN: \BNVS_scaNned_M:w
2174 } {
2175     \BNVS_ast_plc_none: \BNVS_end_scaN: \BNVS_scaNned_L:w
2176 } {
2177     \BNVS_ast_plc_none: \BNVS_end_scaN: \BNVS_scaNned_S:
2178 }
2179 } {
2180     \BNVS_scaN_Raw_OCMLS:w
2181 }
2182 }

2183 \cs_set:Npn \BNVS_skip_no_B:w {
2184     \BNVS_skip_no_B::oc::w { } { }
2185 }

```

\BNVS_scaN_VAZL_MLS:w \BNVS_scaN_VAZL_MLS:w *<input stream>*

Scan the input stream for a *<VAZL>* unit. The scanning process will end on the first comma, the first unbalanced closing delimiter (at the same level) or in stop situation, calling one of *\BNVS_scaNned_L:w*, *\BNVS_scaNned_M:w* or *\BNVS_scaNned_S:*. Calls *\BNVS_scaN_Raw_OCMLS:w* to scan from the input stream material delimited by comma, colons, standard open and close delimiters.

```

2186 \cs_new:Npn \BNVS_scaN_VAZL_MLS:w {

```

- ❗ Create a subscan session to manage the ast policy. The *<pfX>* will depend on the material scanned. The first component is scanned in this session, the other ones are scanned in sub sessions.

```

2187     \BNVS_subscan::OCMLSN::w {

```

- ❗ A single colon has been scanned, next operations depend on the emptiness of *<A>*.

```

2188     \BNVS_if_xprted_empty:TF {

```

- ❗ Scan a *:<Z>* or a *:<Z>::<L>* range. Keep the same scanning session for *<Z>*.

```

2189     \BNVS_ast_plc_init::XX::n { \BNVSStar: } { \BNVS::Z } { { #####1 } }
2190     \BNVS_scaNned_set::OC {
2191         \BNVS_error:n { Unexpected~`:',~replaced~by~`::'. }
2192         \BNVS_scaNned_C:w
2193     } {
2194         \BNVS_if_xprted_empty:TF {

```

- ❗ Bad *: ::...<L>* range.

```

2195         \BNVS_error:n { Unexpected~`::...',~ignored. }
2196         \BNVS_ast_plc_init_mpt::X::n { \BNVSStar: } { }
2197         \BNVS_skip_no_B::oc::w { } { }
2198     } {

```

❗ : $\langle Z \rangle$ range.

```

2199         \BNVS_ast_plc_init_mpt::X::n { } { #####1 }
2200         \BNVS_xprted_brace:
2201         \BNVS_scaNning_ZL:w
2202     }
2203 }
2204 \BNVS_scaN_no_B:w
2205 } {

```

❗ What has been scanned so far is $\langle A \rangle$, put it into braces and go for a $\langle A \rangle : \langle Z \rangle$ range.

```

2206         \BNVS_xprted_brace:
2207         \BNVS_scaNning_AZ:w
2208     }
2209 } {

```

❗ A double colon has been scanned, next operations depend on the emptiness of $\langle A \rangle$.

```

2210         \BNVS_if_xprted_empty:TF {

```

❗ Scan a $:: \langle L \rangle$ or a $:: \langle L \rangle : \langle Z \rangle$ range. Keep the same scanning session for $\langle L \rangle$.

```

2211         \BNVS_ast_plc_init::XX::n { \BNVSStar: } { \BNVS::L } { { #####1 } }
2212         \BNVS_scaNned_set::OC {
2213         \BNVS_if_xprted_empty:TF {

```

❗ Bad $:: \dots \langle L \rangle$ range.

```

2214         \BNVS_error:n { Unexpected~`::...',~ignored. }
2215         \BNVS_ast_plc_init_mpt::X::n { \BNVSStar: } { }
2216         \BNVS_skip_no_B::oc::w { } { }
2217     } {

```

❗ $:: \langle L \rangle$ range.

```

2218         \BNVS_ast_plc_init_mpt::X::n { } { #####1 }
2219         \BNVS_xprted_brace:
2220         \BNVS_scaNning_LZ:w
2221     }
2222 } {
2223     \BNVS_error:n { Unexpected~`:::',~replaced-by~`::'. }
2224     \BNVS_scaNned_O:w
2225 }
2226 \BNVS_scaN_no_B:w
2227 } {

```

❗ What has been scanned so far is $\langle A \rangle$, put it into braces and go for a $\langle A \rangle : \langle L \rangle$ range.

```

2228         \BNVS_xprted_brace:
2229         \BNVS_scaNning_AL:w
2230     }
2231 } {
2232     \BNVS_end_scaN: \BNVS_scaNned_M:w
2233 } {
2234     \BNVS_end_scaN: \BNVS_scaNned_L:w
2235 } {
2236     \BNVS_end_scaN: \BNVS_scaNned_S:
2237 } {

```

By default, we assume a $\langle V \rangle$.

```

2238     \BNVS_ast_plc_init:X { \BNVS::V }
2239     \BNVS_scaN_no_B:w
2240 }
2241 }
```

\BNVS_scaNning_ZL:w **\BNVS_scaNning_ZL:w**

Utility. Scanning a $: \langle Z \rangle$ or a $: \langle Z \rangle :: \langle L \rangle$ range, the $\langle Z \rangle$ is already exported between braces.

```

2242 \cs_new:Npn \BNVS_scaNning_ZL:w {
```

Create a subscan process to scan the $\langle L \rangle$ component. Only the **\BNVS_scaNned_M:w**, **\BNVS_scaNned_L:w** or **\BNVS_scaNned_S:** branch functions are expected, the other ones raise.

```

2243   \BNVS_subscan::RTN::w { } {
2244     \BNVS_will_end:n {
2245       \BNVS_if_xprted_empty:TF {
2246         \BNVS_did_end:n {
2247           \BNVS_ast_pfx_set:n { \BNVS::Z }
2248         }
2249       } {
2250         \BNVS_ast_plc_init_grp:n { ##1 }
2251         \BNVS_did_end:n {
2252           \BNVS_ast_pfx_set:n { \BNVS::ZL }
2253         }
2254       }
2255     }
2256     \BNVS_scaNned_set::OC {
2257       \BNVS_error:n { Unexpected~`:... ',~ignored. }
2258       \BNVS_end_scaN:
2259       \BNVS_skip_no_B::oc::w { } { }
2260     } {
2261       \BNVS_error:n { Unexpected~`:... ',~ignored. }
2262       \BNVS_end_scaN:
2263       \BNVS_skip_no_B::oc::w { } { }
2264     }
2265   } {
2266     \BNVS_scaN_no_B:w
2267   }
2268 }
```

\BNVS_scaNning_LZ:w **\BNVS_scaNning_LZ:w**

Utility. Scanning a $:: \langle L \rangle$ or a $:: \langle L \rangle : \langle Z \rangle$ range, the $\langle L \rangle$ is already exported between braces.

```

2269 \cs_new:Npn \BNVS_scaNning_LZ:w {
```

Create a subscan process to scan the $\langle Z \rangle$ component. Only the **\BNVS_scaNned_M:w**, **\BNVS_scaNned_L:w** or **\BNVS_scaNned_S:** branch functions are expected, the other ones raise.

```

2270   \BNVS_subscan::RTN::w { } {
```

```

2271 \BNVS_will_end:n {
2272   \BNVS_if_xprted_empty:TF {
2273     \BNVS_did_end:n {
2274       \BNVS_ast_pfx_set:n { \BNVS::L }
2275     }
2276   } {
2277     \BNVS_ast_plc_init_grp:n { ##1 }
2278     \BNVS_did_end:n {
2279       \BNVS_ast_pfx_set:n { \BNVS::LZ }
2280     }
2281   }
2282 }
2283 \BNVS_scaNned_set::OC {
2284   \BNVS_error:n { Unexpected~`:... ',~ignored. }
2285   \BNVS_end_scaN:
2286   \BNVS_skip_no_B::oc::w { } { }
2287 } {
2288   \BNVS_error:n { Unexpected~`:... ',~ignored. }
2289   \BNVS_end_scaN:
2290   \BNVS_skip_no_B::oc::w { } { }
2291 }
2292 } {
2293   \BNVS_scaN_no_B:w
2294 }
2295 }

```

\BNVS_scaNning_AZ:w [\BNVS_scaNning_AZ:w](#)

Utility. Scanning a $\langle A \rangle : \langle Z \rangle$ or a $\langle A \rangle : \langle Z \rangle :: \langle L \rangle$ range, only the $\langle A \rangle$ is already exported, between braces.

```

2296 \cs_new:Npn \BNVS_scaNning_AZ:w {

```

❗ Create a subscan process to catch the emptiness of $\langle Z \rangle$.

```

2297   \BNVS_subscan::RTN::w {
2298     \BNVS_ast_plc_brc:
2299     \BNVS_scaNned_set::OCMLS {
2300       \BNVS_error:n { Unexpected~`:',~replaced~by~`::'. }
2301       \BNVS_scaNned_C:w
2302     } {
2303       \BNVS_end_scaN:
2304       \BNVS_scaNning_AZL:w
2305     } {
2306       \BNVS_end_scaN:
2307       \BNVS_ast_plc_init::X::n { \BNVS::AZ } { ####1 }
2308       \BNVS_scaNned_M:w
2309     } {
2310       \BNVS_end_scaN:
2311       \BNVS_ast_plc_init::X::n { \BNVS::AZ } { ####1 }
2312       \BNVS_scaNned_L:w
2313     } {
2314       \BNVS_end_scaN:
2315       \BNVS_ast_plc_init::X::n { \BNVS::AZ } { ####1 }
2316       \BNVS_scaNned_S:

```

```

2317     }
2318     \BNVS_scaN_no_B:w
2319 }
2320 }

```

\BNVS_scaNning_AZL:w \BNVS_scaNning_AZL:w

Utility. Scanning a $\langle A \rangle : \langle Z \rangle$ range or a $\langle A \rangle : \langle Z \rangle :: \langle L \rangle$ range, the $\langle A \rangle$ and $\langle Z \rangle$ are already exported between braces. Only the `\BNVS_scaNned_M:w`, `\BNVS_scaNned_L:w` or `\BNVS_scaNned_S:` are allowed, the other ones raise.

```

2321 \cs_new:Npn \BNVS_scaNning_AZL:w {

```

❗ Create a subscan process to catch the emptiness of $\langle L \rangle$.

```

2322   \BNVS_subscan::RTN::w {
2323     \BNVS_will_end:n {
2324       \BNVS_if_xprted_empty:TF {
2325         \BNVS_ast_plc_none:
2326         \BNVS_did_end:n {
2327           \BNVS_ast_plc_init_mpt::X::n { \BNVS::AZ } { ##1 }
2328         }
2329       } {
2330         \BNVS_ast_plc_brc:
2331         \BNVS_did_end:n {
2332           \BNVS_ast_plc_init_mpt::X::n { \BNVS::AZL } { ##1 }
2333         }
2334       }
2335     }
2336     \BNVS_scaNned_set::OC {
2337       \BNVS_error:n { Unexpected~`:... ',~ignored. }
2338       \BNVS_end_scaN:
2339       \BNVS_skip_no_B:w
2340     } {
2341       \BNVS_error:n { Unexpected~`:... ',~ignored.. }
2342       \BNVS_end_scaN:
2343       \BNVS_skip_no_B:w
2344     }
2345     \BNVS_scaN_no_B:w
2346   }
2347 }

```

\BNVS_scaNning_AL:w \BNVS_scaNning_AL:w

Utility. Scanning a $\langle A \rangle :: \langle L \rangle$ or a $\langle A \rangle :: \langle L \rangle : \langle Z \rangle$ range, only the $\langle A \rangle$ is already exported, between braces.

```

2348 \cs_new:Npn \BNVS_scaNning_AL:w {

```

❗ Create a subscan process to catch the emptiness of $\langle L \rangle$.

```

2349   \BNVS_subscan::N::w {
2350     \BNVS_ast_plc_brc:
2351     \BNVS_scaNned_set::OCMLS {
2352       \BNVS_end_scaN:
2353       \BNVS_scaNning_ALZ:w
2354     } {

```

```

2355     \BNVS_error:n { Unexpected~`::',~replaced-by~`:'.' }
2356     \BNVS_scaNned_0:w
2357   } {
2358     \BNVS_end_scaN:
2359     \BNVS_ast_plc_init::X::n { \BNVS::AL } { ####1 }
2360     \BNVS_scaNned_M:w
2361   } {
2362     \BNVS_end_scaN:
2363     \BNVS_ast_plc_init::X::n { \BNVS::AL } { ####1 }
2364     \BNVS_scaNned_L:w
2365   } {
2366     \BNVS_end_scaN:
2367     \BNVS_ast_plc_init::X::n { \BNVS::AL } { ####1 }
2368     \BNVS_scaNned_S:
2369   }
2370   \BNVS_scaN_no_B:w
2371 }
2372 }

```

\BNVS_scaNning_ALZ:w \BNVS_scaNning_ALZ:w

Utility. Scanning a $\langle A \rangle :: \langle L \rangle$ or $\langle A \rangle :: \langle L \rangle : \langle Z \rangle$ range, the $\langle A \rangle$ and $\langle AL \rangle$ are already exported between braces. Only the `\BNVS_scaNned_M:w`, `\BNVS_scaNned_L:w` or `\BNVS_scaNned_S:` are allowed, the other ones raise.

```

2373 \cs_new:Npn \BNVS_scaNning_ALZ:w {

```

❗ Create a subscan process to catch the emptiness of $\langle Z \rangle$.

```

2374   \BNVS_subscan::N::w {
2375     \BNVS_will_end:n {
2376       \BNVS_if_xprted_empty:TF {
2377         \BNVS_ast_plc_none:
2378         \BNVS_did_end:n {
2379           \BNVS_ast_plc_init_mpt::X::n { \BNVS::AL } { ##1 }
2380         }
2381       } {
2382         \BNVS_ast_plc_brc:
2383         \BNVS_did_end:n {
2384           \BNVS_ast_plc_init_mpt::X::n { \BNVS::ALZ } { ##1 }
2385         }
2386       }
2387     }
2388     \BNVS_scaNned_set::OC {
2389       \BNVS_error:n { Unexpected~`:... ',~ignored. }
2390       \BNVS_end_scaN:
2391       \BNVS_skip_no_B:w
2392     } {
2393       \BNVS_error:n { Unexpected~`:... ',~ignored.. }
2394       \BNVS_end_scaN:
2395       \BNVS_skip_no_B:w
2396     }
2397     \BNVS_scaN_no_B:w
2398   }
2399 }

```

1.12.12 Lists

Lists between rounded brackets are just used to gather material that may contain colons or commas separating balanced material. It is used to collect function arguments. Here is a preliminary grammar for such lists:

```
14 <VAZLRnd> ::= <RndOpen> <VAZLLst> <RndClose>
15 <VAZLLst> ::= <spc> ( <VAZL> ( <comma> <VAZL> )* )? <spc>
```

and the associate AST :

```
<VAZLRnd>* ::= { <VAZLLst>* }
<VAZLLst>* ::= ( { <VAZL>* } )*
```

```
\BNVS_scaN_VAZLLst_LS:w <input stream>
\BNVS_scaN_VAZLLst_LS::B:w <B:w>
<input stream>
```

Scan the *<input stream>* for a comma separated list of *<VAZL>* entries. Terminates branching to *\BNVS_scaNned_S:* in stop situation, branching to *\BNVS_scaNned_L:w* otherwise.

```
2400 \cs_new:Npn \BNVS_scaN_VAZLLst_LS:w {
2401   \BNVS_scaN_csl::RlsTN::w { } { } { } { } {
2402   } {
2403     \BNVS_scaN_VAZL_MLS:w
2404   }
2405 }
```

```
\BNVS_scaN_VAZLRnd::BST::Fw \BNVS_scaN_VAZLRnd::BSTN::Fw {<B:w>} {<S:>} {<T:>} {<scaN:w>} <input stream>
\BNVS_scaN_VAZLRnd::BSTN::Fw \BNVS_scaN_VAZLRnd::BST::Fw {<B:w>} {<S:>} {<T:>} <input stream>
\BNVS_scaN_VAZLRnd_BS::T::Fw \BNVS_scaN_VAZLRnd_BS::T::Fw {<T:>} {<F:w>} <input stream>
```

Scan *<VAZLRnd>* if any, otherwise execute *<F:w>* instructions.

```
2406 \cs_new:Npn \BNVS_scaN_VAZLRnd::BST::Fw #1 #2 #3 {
2407   \BNVS_scaN_Rnd::RBSTN::Fw { } { #1 } { #2 } { #3 } {
2408     \BNVS_scaN_VAZLLst_LS:w
2409   }
```

⊘ There must be absolutely no code here. No hooking allowed either.

```
2410 }

2411 \cs_new:Npn \BNVS_scaN_VAZLRnd_BS::T::Fw {
2412   \BNVS_scaN_VAZLRnd::BST::Fw {
2413     \BNVS_scaNned_B:w
2414   } {
2415     \BNVS_scaNned_S:
2416   }
```

⊘ There must be absolutely no code here. No hooking allowed either.

```
2417 }
```

1.12.13 ISR unit

Here is a partial grammar for identifiers,

```

3  <ISR>      ::= ( <IS> | <dot_P> ) <R>*
4  <IS>       ::= ( <I>? '!' )? <S>
5  <I>        ::= <S>
6  <S>        ::= <alpha> <alnum>*
7  <dot_P>    ::= '.' <P>
8  <P>        ::= <digit>* <S>
9  <R>        ::= <dot_P> | <dot_N> | <n>
10 <dot_N>    ::= '.' <N>
11 <n>        ::= <SqrOpen> <n_expr> <SqrClose>
12 <n_expr>   ::= <blnc>

```

and the associate AST :

```

<IS>* ::= \BNVS::IS { <I> } { <S> }
<ISR>* ::= \BNVS::ISR { <I> } { <S> } { <R>* }
<R>* ::= ( <P>* | <N>* | <n>* ) *
<P>* ::= { \BNVS::P { <P> } }
<N>* ::= { \BNVS::N { <N> } }
<n>* ::= { \BNVS::n { <n_expr>* } }

```

\c__bnvs_IKS_regex The start of a qualified dotted name, with three capture groups.

(End of definition for \c__bnvs_IKS_regex.)

```

2418 \regex_const:Nn \c__bnvs_IKS_regex {

```

1: the optional frame <id> with

2: the optional exclamation mark

```

2419 (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?

```

3: followed by a non empty short name.

```

2420 ( \ur{c__bnvs_S_regex} )

```

```

2421 }

```

```

\BNVS_scaNned::BIKS::w \BNVS_scaNned::BIKS::w { <scaN::B:w> } { <I> } { <K> } { <S> } <input stream>

```

Only called by \BNVS_scaN_IS::B::Fw. Execute <scaN::B:w> instructions, which terminate by \BNVS_scaNned_B:w.

```

2422 \cs_new:Npn \BNVS_scaNned::BIKS::w #1 #2 #3 #4 {

```

```

2423 \tl_if_empty:nTF { #3 } {

```


- ❗ No frame identifier given, use the current value. Store the short name, store in the appropriate variable.

```

2424     \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
2425     \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #4 }
2426 } {

```

- ❗ Frame identifier given. Proceed as before mutatis mutandis.

```

2427     \__bnvs_tl_set:cn I { #2 }
2428     \__bnvs_tl_clear_new:c { S(#2) }
2429     \__bnvs_tl_set:cn { S(#2) } { #4 }
2430 }
2431 \__bnvs_tl_set:cn S { #4 }

```

- ❗ Create a scan group to manage the $\langle pfX:w \rangle$.

```

2432 \BNVS_subscan::RTN::w {

```

- ❗ By default we start on a standalone identifier,

```

2433     \BNVS_ast_plc_init:n { ##1 }
2434     \BNVS_ast_pfX_set:n { \BNVS::IS }
2435 } {

```

- ❗ Export $\langle I \rangle$ and $\langle S \rangle$.

```

2436     \BNVS_tl_use:Nv \BNVS_xprt_braced:n I
2437     \BNVS_xprt_braced:n { #4 }
2438 } {

```

- ❗ Execute the $\langle scaN::B:w \rangle$ instructions including a call to the terminating branch function $\backslash\text{BNVS_scaNned_B:w}$. The $\langle pfX:w \rangle$ will possibly change.

```

2439     \cs_set:Npn \BNVS_scaN::B:w ##1 { #1 }
2440     \BNVS_scaN::B:w { \BNVS_scaNned_B:w }
2441 }
2442 }

```

```

\BNVS_scaN_IS::B::Fw \BNVS_scaN_IS::B::Fw { \langle scaN::B:w \rangle } { \langle F:w \rangle } \langle input stream \rangle

```

Scan an $\langle IS \rangle$ unit if any, stores the $\langle I \rangle$, $\langle S \rangle$ between braces, and calls the branch function $\backslash\text{BNVS_scaNned::BIKS::w}$ that will execute $\langle scaN::B:w \rangle$ instructions (see above). The $\langle F:w \rangle$ instructions are executed when no $\langle IS \rangle$ unit is found.

```

2443 \cs_new:Npn \BNVS_scaN_IS::B::Fw #1 #2 {
2444     \peek_regex_replace_once:NnTF \c__bnvs_IKS_regex {
2445         \cB\{ \1 \cE\} \cB\{ \2 \cE\} \cB\{ \3 \cE\}
2446     } {
2447         \BNVS_scaNned::BIKS::w { #1 }
2448     } {

```

- ❗ Choose the false branch.

```

2449         #2
2450     }
2451 }

```

```
\BNVS_scaNning_PNn::P::w \BNVS_scaNning_PNn::P::w {<P>} <input stream>
```

Export the $\langle P \rangle$ component then scan another $\langle PNn \rangle$ component.

```
2452 \cs_new:Npn \BNVS_scaNning_PNn::P::w #1 {
2453   \BNVS_xprt_braced:n { \BNVS::P { #1 } }
2454   \BNVS_scaN_PNn:w
2455 }
```

```
\BNVS_scaNning_PNn::N::w \BNVS_scaNning_PNn::N::w {<N>} <input stream>
```

Export the $\langle N \rangle$ component then scan another $\langle PNn \rangle$ component.

```
2456 \cs_new:Npn \BNVS_scaNning_PNn::N::w #1 {
```

❏ Normalize $\langle N \rangle$.

```
2457   \exp_args:Ne \BNVS_xprt_braced:n { \exp_not:N \BNVS::N { \int_eval:n { #1 } } }
2458   \BNVS_scaN_PNn:w
2459 }
```

```
\BNVS_scaNning_PNn_n:w \BNVS_scaNning_PNn_n:w <input stream>
```

Only called by `\BNVS_scaN_PNn:w`.

```
2460 \cs_new:Npn \BNVS_scaNning_PNn_n:w {
2461   \BNVS_scaN:N::RBSTN::w %[
2462   ] {
2463     \BNVS_ast_plc_init:n { { \BNVS::n { ##1 } } }
2464   } {
2465     \BNVS_scaNned_B:w
2466   } {
2467     \BNVS_scaNned_S:
2468   } {
2469
```

❏ On cLose branch, scan $\langle PNn \rangle$ after $\langle n \rangle$.

```
2470     \BNVS_scaNned_set::L { \BNVS_end_scaN_no_I_change: \BNVS_scaN_PNn:w }
2471   } {
2472     \BNVS_scaN_n_expr_LS:w
2473   }
2474 }
```

```
\BNVS_scaN_PNn:w \BNVS_scaN_PNn:w <input stream>
```

Scan all the individual $\langle R \rangle$ components, exporting them one after the other, then execute the `\BNVS_scaNned_B:w` branch function. It calls the auxiliary functions `\BNVS_scaNning_PNn::P::w` for a path component, `\BNVS_scaNning_PNn::N::w` for a litteral index or `\BNVS_scaNning_PNn_n:w`.

```
2475 \cs_new:Npn \BNVS_scaN_PNn:w {
2476   \peek_charcode_remove:NTF . {
2477     \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
2478       \BNVS_scaNning_PNn::P::w
2479     } {
2480       \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
2481         \BNVS_scaNning_PNn::N::w
2482       } {
```

Terminates the block and reinject the dot ‘.’.

```

2483         \BNVS_scaNned_B:w .
2484     }
2485 }
2486 } {
2487     \BNVS_peek_regex_remove_once:NTF \c__bnvs_Sqr_open_regex {
2488         \BNVS_scaNning_PNn_n:w
2489     } {

```

Also terminates the block.

```

2490         \BNVS_scaNned_B:w
2491     }
2492 }
2493 }

```

\BNVS_scaNning_R::BP::w **\BNVS_scaNning_R::BP::w** {*<scan_B:w>*} {*<P>*} *<input stream>*

Only called by **\BNVS_scaN_R::B::w** when a heading *<P>* has just been scanned. Calls **\BNVS_scaN_PNn:w**.

```

2494 \cs_new:Npn \BNVS_scaNning_R::BP::w #1 #2 {

```

New scan group to gather the *<R>* relative path.

```

2495     \BNVS_subscan:RTN::w {
2496         \BNVS_ast_plc_init_grp:n { ##1 }
2497         \BNVS_xprt_braced:n { \BNVS::P { #2 } }
2498     } {
2499         \BNVS_scaNned_set::B {
2500             \BNVS_end_scaN_no_I_change:
2501             #1
2502         }
2503     } {
2504         \BNVS_scaN_PNn:w
2505     }
2506 }

```

\BNVS_scaNning_R::BN::w **\BNVS_scaNning_R::BN::w** {*<scan_B:w>*} {*<N>*} *<input stream>*

Only called by **\BNVS_scaN_R::B::w** when a heading *<N>* has just been scanned. Calls **\BNVS_scaN_PNn:w**.

```

2507 \cs_new:Npn \BNVS_scaNning_R::BN::w #1 #2 {

```

New scan group to gather the *<R>* relative path.

```

2508     \BNVS_subscan:RTN::w {
2509         \BNVS_ast_plc_init_grp:n { ##1 }
2510         \exp_args:Ne \BNVS_xprt_braced:n {

```

Normalize *<N>*.

```

2511         \exp_not:N \BNVS::N { \int_eval:n { #2 } }
2512     }
2513 } {
2514     \BNVS_scaNned_set::B {

```

```

2515     \BNVS_end_scaN_no_I_change:
2516     #1
2517   }
2518 } {
2519   \BNVS_scaN_PNn:w
2520 }
2521 }

```

\BNVS_scaNning_R_n:B:w **\BNVS_scaNning_R_n:B:w** {*<scan_B:w>*} *<input stream>*

Only called by **\BNVS_scaN_R::B::w** when a heading '[' has just been scanned. Terminates by **\BNVS_scaN_PNn:w** to scan another component. See above for *<scan::B:w>*.

```

2522 \cs_new:Npn \BNVS_scaNning_R_n:B:w #1 {

```

¶ New scan group to gather the *<R>* relative path.

```

2523   \BNVS_subscan::RTN::w {
2524     \BNVS_ast_plc_init_grp:n { ##1 }
2525   } {

```

¶ On block branch, close the scan group and execute the *<scan_B:w>* instructions.

```

2526     \BNVS_scaNned_set::B {
2527       \BNVS_end_scaN_no_I_change:
2528       #1
2529     }
2530   } {

```

¶ Export material up to the closing square bracket as one single argument group.

```

2531     \BNVS_scaN:N::RBSTN::w %[
2532   ] {
2533     \BNVS_ast_plc_init:n { { \BNVS::n { ##1 } } }
2534   } {
2535     \BNVS_scaNned_B:w
2536   } {
2537     \BNVS_scaNned_S:
2538   } {

```

¶ On close branch, scan *<PNn>* after *<n>*.

```

2539     \BNVS_scaNned_set::L { \BNVS_end_scaN_no_I_change: \BNVS_scaN_PNn:w }
2540   } {
2541     \BNVS_scaN_n_expr_LS:w
2542   }
2543 }
2544 }

```

\BNVS_scaN_R::B::w **\BNVS_scaN_R::B::w** {*<scan::B:w>*} *<input stream>*

Scan a *<R>* component and execute *<scan_B:w>* instructions. The scanning stops exactly when some regular expression matches, or when squared material is scanned. The function **\BNVS_scaN_R::B::w** calls

- **\BNVS_scaNning_R::BP::w**,
- **\BNVS_scaNning_R::BN::w** or
- **\BNVS_scaNning_R_n:B:w**

with *<scan::B:w>* as argument.

```

2545 \cs_new:Npn \BNVS_scaN_R::B::w #1 {
2546   \peek_charcode_remove:NTF . {
2547     \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
2548       \BNVS_scaNning_R::BP::w { #1 }
2549     } {
2550       \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
2551         \BNVS_scaNning_R::BN::w { #1 }
2552       } {

```

❗ Terminates the block and reinject the dot.

```

2553       \BNVS_xprt_braced:n { }
2554       #1
2555       .
2556     }
2557   }
2558 } {
2559   \BNVS_peek_regex_remove_once:NTF \c__bnvs_Sqr_open_regex {
2560     \BNVS_scaNning_R_n:B:w { #1 }
2561   } {

```

❗ Terminates the block too.

```

2562     \BNVS_xprt_braced:n { }
2563     #1
2564   }
2565 }
2566 }

2567 \tl_new:N \l__bnvs_other_X_tl
2568 \tl_set:Nx \l__bnvs_other_X_tl { \tl_to_str:n { X } }

```

<code>\BNVS_scaNned_dot:::BP::w</code> <code>\BNVS_scaNned_dot:n:::BP::w</code>	<code>\BNVS_scaNned_dot:::BP::w {⟨B:w⟩} {⟨P⟩} ⟨input stream⟩</code> <code>\BNVS_scaNned_dot:::BP::w {⟨before⟩} {⟨B:w⟩} {⟨P⟩} ⟨input stream⟩</code>
--	---

Only called by `\BNVS_scaN_dot_P::B::Fw`. The second functions calls rhe first one with an empty `⟨before⟩`.

```

2569 \cs_new:Npn \BNVS_scaNned_dot:n:::BP::w #1 #2 #3 {

```

❗ No frame identifier given, no short name given.

```

2570   \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {

```

It defaults to X.

```

2571     \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
2572     \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
2573   }
2574   \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I ) }

```

❗ Create a scan group to manage the `⟨pfX:w⟩`. It will be closed by a call to `\BNVS_scaNned_B:w`.

```

2575   \BNVS_subscan::RTN::w {

```

By default we start on a standalone identifier,

```

2576     \BNVS_ast_pfX_set:n { \BNVS::ISR }
2577     \exp_args:Ne \BNVS_ast_plc_init_mpt:n {
2578       { \__bnvs_tl_use:c I }
2579       { \__bnvs_tl_use:c S }
2580       { \exp_not:n { ##1 } }
2581     }
2582     \BNVS_xprt_put:n { { \BNVS::P { #3 } } }
2583   } {
2584     \BNVS_scaNned_B_wrap:n:B { #1 } { #2 }

```

Create a scan group to manage the $\langle pfX:w \rangle$.

```

2585   } {

```

Find the next components.

```

2586     \BNVS_scaN_PNn:w
2587   }
2588 }

2589 \cs_new:Npn \BNVS_scaNned_dot:::BP::w {
2590   \BNVS_scaNned_dot:n:::BP::w {}
2591 }

```

$\backslash\text{BNVS_scaN_dot_P}::\text{B}::\text{Fw}$ $\backslash\text{BNVS_scaN_dot_P}::\text{B}::\text{Fw}$ $\{ \langle \text{scaN}::\text{B:w} \rangle \} \{ \langle \text{F:w} \rangle \}$ $\langle \text{input stream} \rangle$

Scan a $\langle \text{dot_P} \rangle$ unit and execute $\langle \text{scaN}::\text{B:w} \rangle$ instructions that must end with $\backslash\text{BNVS_scaNned_B:w}$. The $\langle \text{F:w} \rangle$ instructions are executed when no $\langle \text{dot_P} \rangle$ unit is found.

```

2592 \cs_new:Npn \BNVS_scaN_dot_P:::B::Fw #1 #2 {
2593   \peek_charcode_remove:NTF . {
2594     \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
2595       \BNVS_scaNned_dot:::BP::w { #1 }
2596     } {

```

Choose the false branch and reinject the dot.

```

2597       #2
2598     .
2599   }
2600 } {

```

Choose also the false branch.

```

2601       #2
2602     }
2603 }

```

$\backslash\text{BNVS_scaNned_dot}::\text{n}::\text{B}::\text{N:w}$ $\backslash\text{BNVS_scaNned_dot}::\text{n}::\text{B}::\text{N:w}$ $\{ \langle \text{before} \rangle \} \{ \langle \text{scaN}::\text{B:w} \rangle \} \{ \langle \text{N} \rangle \}$ $\langle \text{input stream} \rangle$
 $\backslash\text{BNVS_scaNned_dot}::\text{B}::\text{N:w}$ $\backslash\text{BNVS_scaNned_dot}::\text{B}::\text{N:w}$ $\{ \langle \text{scaN}::\text{B:w} \rangle \} \{ \langle \text{N} \rangle \}$ $\langle \text{input stream} \rangle$

Only called by $\backslash\text{BNVS_scaN_dot_N}::\text{B}::\text{Fw}$.

```

2604 \cs_new:Npn \BNVS_scaNned_dot:::B::N:w #1 #2 #3 {

```

No frame identifier given, no short name given.

```

2605   \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {

```

It defaults to X.

```

2606     \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
2607     \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
2608   }
2609   \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I ) }

```

❗ Create a scan group to manage the $\langle pfX:w \rangle$. It will be closed by a call to `\BNVS_scaNned_B:w`.

```

2610   \BNVS_subscan::RTN::w {

```

❗ By default we start on a standalone identifier,

```

2611     \BNVS_ast_pfX_set:n { \BNVS::ISR }
2612     \exp_args:Ne \BNVS_ast_plc_init_mpt:n {
2613       { \__bnvs_tl_use:c I }
2614       { \__bnvs_tl_use:c S }
2615       { \exp_not:n { ##1 } }
2616     }
2617     \exp_args:Ne \BNVS_xprt_put:n {
2618       { \exp_not:N \BNVS::N { \int_eval:n { #3 } } }
2619     }
2620   } {
2621     \BNVS_scaNned_B_wrap:n:B { #1 } { #2 }

```

❗ Create a scan group to manage the $\langle pfX:w \rangle$.

```

2622   } {

```

❗ Find the next components.

```

2623     \BNVS_scaN_PNn:w
2624   }
2625 }

2626 \cs_new:Npn \BNVS_scaNned_dot_N::B::Fw {
2627   \BNVS_scaNned_dot:n:B:N:w { }
2628 }

```

❗ Create a scan group to manage the $\langle pfX:w \rangle$.

`\BNVS_scaN_dot_N::B::Fw` `\BNVS_scaN_dot_N::B::Fw { $\langle scaN::B:w \rangle$ } { $\langle F:w \rangle$ }  $\langle input stream \rangle$`

Scan a $\langle dot_N \rangle$ unit and execute $\langle scaN::B:w \rangle$ instructions that must end with `\BNVS_scaNned_B:w`. The $\langle F:w \rangle$ instructions are executed when no $\langle dot_N \rangle$ unit is found.

```

2629 \cs_new:Npn \BNVS_scaN_dot_N::B::Fw #1 #2 {
2630   \peek_charcode_remove:NTF . {
2631     \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
2632       \BNVS_scaNned_dot:N:B:N:w { #1 }
2633     } {

```

❗ Choose the false branch and reinject the dot.

```

2634     #2
2635     .

```

```

2636     }
2637   } {

```

❗ Choose also the false branch.

```

2638     #2
2639   }
2640 }

```

```

\BNVS_scaN_ISR::B::Fw \BNVS_scaN_ISR::B::Fw {⟨scaN:b:⟩} {⟨F:w⟩} ⟨input stream⟩

```

Scan an $\langle ISR \rangle$ unit if any, $\langle scaN:b: \rangle$ instructions terminated by a call to the branch function $\backslash BNVS_scaNned_B:w$. The $\langle F:w \rangle$ instructions are executed when no $\langle ISR \rangle$ unit is found.

```

2641 \cs_new:Npn \BNVS_scaN_ISR::B::Fw #1 #2 {
2642   \BNVS_scaN_IS::B::Fw {
2643     \BNVS_scaN_R::B::w { #1 }
2644   } {
2645     #2
2646   }
2647 }

```

1.12.14 ISR extension units

Either for assignments or function calls. Here is the grammar

```

7  ⟨ISRMOR⟩ ::= ⟨ISR⟩ ⟨MOR⟩
8  ⟨MOR⟩    ::= ⟨ModOpEq⟩ ⟨Rhs⟩
9  ⟨ModOpEq⟩ ::= ⟨spc⟩ ( ⟨Mod⟩ ⟨Op⟩ '=' ) ⟨spc⟩
10 ⟨Mod⟩    ::= ' ' | '?'
11 ⟨Op⟩     ::= ' ' | '+' | '-' | '/' | '*'
12 ⟨Rhs⟩     ::= <see below>
13 ⟨ISRC⟩    ::= ⟨ISR⟩ ⟨call⟩
14 ⟨call⟩    ::= ⟨RndOpen⟩ ⟨blnc⟩ ⟨RndClose⟩

```

with the associate AST

```

⟨ISRMOR⟩* ::= \BNVS::ISRMOR {⟨I⟩} {⟨S⟩} {⟨R⟩*} {⟨Mod⟩} {⟨Op⟩} {⟨Rhs⟩*}
⟨ISRC⟩*   ::= \BNVS::ISRC {⟨I⟩} {⟨S⟩} {⟨R⟩*} {⟨RndLst⟩*}

```

$\backslash c_bnvs_ModOpEq_regex$ Assignment operators, two capture groups.

(End of definition for $\backslash c_bnvs_ModOpEq_regex$.)

```

2648 \regex_const:Nn \c__bnvs_ModOpEq_regex {
2649   \s*

```

1: an optional modifier.

```

2650   ( \? )?

```

2: an optional operator.

```

2651   ( [-+/*] )?

```



```

2652   =\s*
2653 }

```

```

\BNVS_scaNned_ModOpEq::MLS::nnw \BNVS_scaNned_ModOpEq::MLS::nnw {<M:w>} {<L:w>} {<S:>} {<Mod>} {<Op>}
<input stream>

```

Only called by \BNVS_scaN_MOR::MLS::Fw. Calls \BNVS_scaN_MLS:w.

```

2654 \cs_new:Npn \BNVS_scaNned_ModOpEq::MLS::nnw #1 #2 #3 #4 #5 {
2655   \BNVS_xprt_braced:n { #4 }
2656   \BNVS_xprt_braced:n { #5 }
2657   \BNVS_scaN::RBOCMLSTN::w {
2658     \BNVS_begin_scaN:
2659     \BNVS_ast_plc_init_grp:n { ##1 }
2660   } { \BNVS_error:n { Unexpected } \BNVS_scaNned_L:w }
2661   { \BNVS_error:n { Unexpected } \BNVS_scaNned_L:w }
2662   { \BNVS_error:n { Unexpected } \BNVS_scaNned_L:w }
2663   { \BNVS_end_scaN_no_I_change: #1 }
2664   { \BNVS_end_scaN_no_I_change: #2 }
2665   { \BNVS_end_scaN_no_I_change: #3 }
2666   { }
2667   { \BNVS_scaN_MLS:w }
2668 }

```

```

\BNVS_scaN_MOR::MLS::Fw \BNVS_scaN_MOR::MLS::Fw {<M:w>} {<L:w>} {<S:>}
{<F:w>} <input stream>

```

Scan a <ModOpEq> unit, if any, and execute one of <comMa:w>, <L:w> or <S:> instructions, otherwise execute <F:w> instructions.

```

2669 \cs_new:Npn \BNVS_scaN_MOR::MLS::Fw #1 #2 #3 #4 {
2670   \peek_regex_replace_once:NnTF \c__bnvs_ModOpEq_regex {
2671     \cB\{ \1 \cE\} \cB\{ \2 \cE\}
2672   } {
2673     \BNVS_ast_pfx_will_set:n { \BNVS::ISRMOR }
2674     \BNVS_scaNned_ModOpEq::MLS::nnw { #1 } { #2 } { #3 }
2675   } {
2676     #4
2677   }
2678 }

```

```

\BNVS_scaN_ISRMOR::MLS::Fw \BNVS_scaN_ISRMOR::MLS::Fw {<M:w>} {<L:w>} {<S:>}
{<F:w>} <input stream>

```

Scan a <ISRMOR> unit, if any, and execute one of <M:w>, <L:w> or <S:> instructions, otherwise execute <F:w> instructions.

```

2679 \cs_new:Npn \BNVS_scaN_ISRMOR::MLS::Fw #1 #2 #3 #4 {
2680   \BNVS_scaN_ISR::B::Fw {
2681     \BNVS_scaN_MOR::MLS::Fw {
2682       \BNVS_ast_pfx_will_set:n { \BNVS::ISRMOR }
2683       #1
2684     } {
2685       \BNVS_ast_pfx_will_set:n { \BNVS::ISRMOR }
2686       #2
2687     } {

```

```

2688     \BNVS_ast_pfx_will_set:n { \BNVS::ISRMOR }
2689     #3
2690   } { #4 }
2691 } {
2692   #4
2693 }
2694 }

```

\BNVS_scaN_ISR::B::Fw **\BNVS_scaN_ISR::B::Fw** {*B:w*} {*F:w*} *<input stream>*

Scan a *<ISRC>* unit, if any, and execute *<B:w>* instructions, otherwise execute *<F:w>* instructions.

```

2695 \cs_new:Npn \BNVS_scaN_ISR::B::Fw #1 #2 {
2696   \BNVS_scaN_ISR::B::Fw {
2697     \BNVS_scaN_VAZLRnd::BST::Fw {
2698       #1
2699     } {
2700       \BNVS_scaNned_S:
2701     } {
2702       \BNVS_did_end:n {
2703         \BNVS_ast_pfx_will_set:n { \BNVS::ISRC }
2704       }
2705     } { #2 }
2706   } { #2 }
2707 }

```

\BNVS_scaN_ISRCMOR::BMLS::Fw **\BNVS_scaN_ISRCMOR::BMLS::Fw** {*B:w*} {*M:w*} {*L:w*} {*S:*} {*F:w*} *<input stream>*

Scan a *<ISRC>* or a *<ISRMOR>* unit, if any, and execute the branching instructions accordingly. Otherwise execute *<F:w>* instructions.

```

2708 \cs_new:Npn \BNVS_scaN_ISRCMOR::BMLS::Fw #1 #2 #3 #4 #5 {
2709   \BNVS_scaN_ISR::B::Fw {
2710     \BNVS_scaN_VAZLRnd::BST::Fw {
2711       \BNVS_ast_pfx_will_set:n { \BNVS::ISRC }
2712       #1
2713     } {
2714       #4
2715     } { } {
2716       \BNVS_scaN_MOR::MLS::Fw {
2717         \BNVS_ast_pfx_will_set:n { \BNVS::ISRMOR }
2718         #2
2719       } {
2720         \BNVS_ast_pfx_will_set:n { \BNVS::ISRMOR }
2721         #3
2722       } {
2723         \BNVS_ast_pfx_will_set:n { \BNVS::ISRMOR }
2724         #4
2725       } { #5 }
2726     }
2727   } { #5 }
2728 }

```

1.12.15 Running an AST

Storage Once data has been turned into an AST, we run it to perform the action. The heading of the AST is `\ast:n{<type>}`, where `<type>` uniquely identifies the type of the AST.

```
\BNVS_ast_run_clear: \BNVS_ast_run_clear:
\BNVS_ast_run_set:n \BNVS_ast_run_set:n {<something>}
\BNVS_ast_run_put:n \BNVS_ast_run_use:n {<instructions:w>}
\BNVS_ast_run_use:n Clear, set or use the ast_run tl variable.

2729 \tl_new:N \l__bnvs_ast_run_tl

2730 \cs_new:Npn \BNVS_ast_run_clear: {
2731   \__bnvs_tl_clear:c { ast_run }
2732 }

2733 \cs_new:Npn \BNVS_ast_run_set:n {
2734   \__bnvs_tl_set:cn { ast_run }
2735 }

2736 \cs_new:Npn \BNVS_ast_run_put:n {
2737   \__bnvs_tl_put_right:cn { ast_run }
2738 }

2739 \cs_new:Npn \BNVS_ast_run_use:n #1 {
2740   \BNVS_tl_use:nv { #1 } { ast_run }
2741 }
```

Policy Running the same AST depends on the expected result. For example, the result might be a range or might be limited to a single integer value. A policy makes the difference.

```
\BNVS_ast_plc_set:n \BNVS_ast_plc_def:nn {<plc>} {<instructions>}
\BNVS_ast_plc_def:nn \BNVS_ast_plc_set:n {<plc>}

\BNVS_ast_plc_def:nn associates the set of <instructions> to the <plc> identifier.
\BNVS_ast_plc_set:n executes the set of instructions associated to <plc>.

2742 \cs_new:Npn \BNVS_ast_plc_set:n #1 {
2743   \use:c { BNVS_ast_plc = #1 }
2744 }

2745 \cs_new:Npn \BNVS_ast_plc_def:nn #1 #2 {
2746   \cs_set:cpn { BNVS_ast_plc = #1 } {
2747     #2
2748   }
2749 }
```

Run The policy must be set prior to the run.

\BNVS_ast_run:cn \BNVS_ast_run:n \BNVS_ast_run_xprted:n	\BNVS_ast_run:cn {⟨var⟩} {⟨ast⟩} \BNVS_ast_run:n {⟨ast⟩} \BNVS_ast_run_xprted:n {⟨ast⟩}
---	---

Execute ⟨ast⟩ and saves the result to the right of ⟨var⟩. When ⟨var⟩ is not provided, it defaults to ast_run.

```

2750 \cs_new:Npn \BNVS_ast_run:cn #1 #2 {
2751   \group_begin:
2752   \BNVS_ast_run_clear:
2753   #2
2754   \BNVS_ast_run_use:n {
2755     \group_end:
2756     \__bnvs_tl_put_right:cn { #1 }
2757   }
2758 }

2759 \cs_new:Npn \BNVS_ast_run:n {
2760   \BNVS_ast_run:cn { ast_run }
2761 }

2762 \cs_new:Npn \BNVS_ast_run_xprted:n {
2763   \BNVS_ast_run:cn { xprted }
2764 }
```

1.12.16 Run units

Here is the grammar

$${}_3 \langle \text{Run} \rangle ::= \langle \text{spc} \rangle \text{'?'} \langle \text{VAZLRnd} \rangle$$

with the associate AST

$$\begin{aligned}
\langle \text{Run} \rangle^* &::= \text{\textcolor{blue}{\backslash BNVSRun:n}} \{ \langle \text{RunLst} \rangle^* \} \\
\langle \text{RunLst} \rangle^* &::= \langle \text{RunItem} \rangle^* (, \langle \text{RunItem} \rangle^*)^* \\
\langle \text{RunItem} \rangle^* &::= \langle \text{int} \rangle? (\langle \text{int} \rangle? \text{'-'} \langle \text{int} \rangle)?
\end{aligned}$$

\BNVS_scaN_Run_S::B::Fw \BNVS_scaN_Run_no_B:Fw	\BNVS_scaN_Run_S::B::Fw {⟨B:w⟩} {⟨F:w⟩} ⟨input stream⟩ \BNVS_scaN_Run_no_B:Fw {⟨F:w⟩} ⟨input stream⟩
---	---

Scan a ⟨Run⟩ unit, if any, and execute ⟨B:w⟩ instructions including the corresponding branch functions, otherwise execute ⟨F:w⟩ instructions.

```

2765 \cs_new:Npn \BNVS_scaN_Run_no_B:Fw {
2766   \BNVS_scaN_Run_S::B::Fw { \BNVS_scaN_no_B:w }
2767 }

2768 \cs_new:Npn \BNVS_scaN_Run_S::B::Fw #1 #2 {
2769   \peek_charcode_remove:NTF ? {
```

```

2770     \BNVS_scaN_VAZLRnd::BST::Fw {
2771         #1
2772     } {
2773         \BNVS_scaNned_S:
2774     } {
2775         \BNVS_xprted_converter_set:n {
2776             \BNVS_ast_run_clear:
2777             \tl_map_function:nN { ##1 } \BNVS_run_item:n
2778             \BNVS_ast_run_use:n { \BNVS_xprt_put:n }
2779         }
2780         \cs_set:Npn \BNVS_run_item:n ##1 {
2781             \BNVS_ast_run:n { ##1 }
2782             \cs_set:Npn \BNVS_run_item:n #####1 {
2783                 \BNVS_ast_run_put:n { , }
2784                 \BNVS_ast_run:n { #####1 }
2785             }
2786         }
2787         \BNVS_ast_plc_init::X::n { \BNVSRun:n } { { ##1 } }
2788     } {
2789         #2 ?
2790     }
2791 } {
2792     #2
2793 }
2794 }

2795 \cs_new:Npn \BNVS_scaN_Run::B::Fw #1 {
2796     \peek_charcode_remove:NTF ? {
2797         \BNVS_scaN_VAZLRnd::BST::Fw {
2798             #1
2799         } {
2800             \BNVS_scaNned_S:
2801         } {
2802             \BNVS_xprted_converter_set:n {
2803                 \BNVS_ast_run_clear:
2804                 \tl_map_function:nN { ##1 } \BNVS_run_item:n
2805                 \BNVS_ast_run_use:n { \BNVS_xprt_put:n }
2806             }
2807             \cs_set:Npn \BNVS_run_item:n ##1 {
2808                 \BNVS_ast_run:n { ##1 }
2809                 \cs_set:Npn \BNVS_run_item:n #####1 {
2810                     \BNVS_ast_run_put:n { , }
2811                     \BNVS_ast_run:n { #####1 }
2812                 }
2813             }
2814             \BNVS_ast_plc_init::X::n { \BNVSRun:n } { { ##1 } }
2815         } {
2816             #1 ?
2817         }
2818     } {
2819         #1
2820     }
2821 }

```

1.13 Definition scanner

The definition is parsed as a $\langle \text{VAZLLst} \rangle$ AST, then the abstract command are defined, and finally the AST is executed.

```
\BNVS_ast_parse_defs:cn \BNVS_ast_parse_defs:cn {<var>} {<defs>}
```

Parse the $\langle \text{defs} \rangle$ AST into $\langle \text{var} \rangle$.

```
2822 \BNVS_new:cpn { ast_parse_defs:cn } #1 #2 {
2823   \BNVS_xprt_clear:
2824   \BNVS_tl_to_str_unbraced:Nn \BNVS_scaN_VAZL_MLS:w { #2 }
2825   \BNVS_xprted_use:n { \_bnvs_tl_set:cn { #1 } }
2826 }
```

1.13.1 AST

Here are all the nodes for the AST of the definitions.

```
<cs1(<item>)>*<:=> { ( {<item>*<}<*<}< }
<IS>*<:=> \BNVS::IS {<I>*<}<{<S>*<}<
<ISR>*<:=> \BNVS::ISR {<I>*<}<{<S>*<}<{<R>*<}<
<R>*<:=> ( {<P>*<}<{<N>*<}<{<n>*<}<*<
<P>*<:=> { \BNVS::P {<P>*<}< }
<N>*<:=> { \BNVS::N {<N>*<}< }
<n>*<:=> { \BNVS::n {<n_expr>*<}< }
<Run>*<:=> \BNVSRun:n {<VAZLLst>*<}<
<ISMOR>*<:=> \BNVS::ISMOR {<I>*<}<{<S>*<}<{<R>*<}<{<Mod>*<}<{<Op>*<}<{<Rhs>*<}<
<ISRC>*<:=> \BNVS::ISRC {<I>*<}<{<S>*<}<{<R>*<}<{<VAZLLst>*<}<
<VAZL>*<:=> {<V>*<}<{<A>*<}<{<Z>*<}<{<L>*<}<{<AZ>*<}<{<AL>*<}<{<ZL>*<}<{<LZ>*<}<{<AZL>*<}<{<ALZ>*<}<
  <=> | \BNVSStar:
<V>*<:=> \BNVS::V {<blnc>*<}<
<A>*<:=> \BNVS::A {<blnc>*<}<
<Z>*<:=> \BNVS::Z {<blnc>*<}<
<L>*<:=> \BNVS::L {<blnc>*<}<
<AZ>*<:=> \BNVS::AZ {<blnc>*<}<{<blnc>*<}<
<AL>*<:=> \BNVS::AL {<blnc>*<}<{<blnc>*<}<
<ZL>*<:=> \BNVS::ZL {<blnc>*<}<{<blnc>*<}<
<LZ>*<:=> \BNVS::LZ {<blnc>*<}<{<blnc>*<}<
<AZL>*<:=> \BNVS::AZL {<blnc>*<}<{<blnc>*<}<{<blnc>*<}<
<ALZ>*<:=> \BNVS::ALZ {<blnc>*<}<{<blnc>*<}<{<blnc>*<}<
<star>*<:=> \BNVSStar:
<VAZLRnd>*<:=> \BNVSRnd:n {<VAZLLst>*<}<
<VAZLSqr>*<:=> \BNVSSqr:n {<VAZLLst>*<}<
<VAZLCr1>*<:=> \BNVSCr1:n {<VAZLLst>*<}<
<VAZLLst>*<:=> ( {<VAZL>*<}<*<
<Raw>*<:=> \BNVS:n {<other>*<}<
<other>*<:=> <characters of category code other>
<blnc>*<:=> ' ' | ( {<VAZL>*<}<{<Rnd>*<}<{<Cr1>*<}<{<Sqr>*<}<{<Run>*<}<{<Raw>*<}<{<blnc>*<}<*<
<Rhs>*<:=> <blnc>*<
```

1.13.2 Next section